CS 225

Data Structures and Algorithms

Spring 2025

Anirudh Konidala

Table of Contents

1 Introduction to CS 225 4
2 C++ Review 6
3 List ADT and Linked List Implementation 9
4 ArrayList: An Array Implementation of a List 22
5 Amortized Analysis 27
6 Stacks and Queues 32
7 Iterators 38
8 Trees 43
9 Tree Traversals 48
10 Tree Searching and Dictionaries 54
11 BST and BST Implementation 58
12 BST Balance 70
13 BST Rotations 74
14 Adelson-Velsky Landis (AVL) Trees 77
15 Lambdas 90
16 BBST Range Search and KD Trees 93
17 BTrees 96
18 BTree Analysis 98
19 Priority Queues 99
20 Heaps 100
21 Disjoint Sets 101
22 Graphs 102
23 Graph Implementations 103
24 Graph Traversals 104
25 MST: Minimum Spanning Tree (Kruskal's Algorithm) 105
26 MST: Minimum Spanning Tree (Prim's Algorithm) 106
27 Dijkstra's Algorithm 107
28 Floyd Warshall's Algorithm 108
29 Probability in Computer Science 109
30 Hashing, Hash Collisions, and Hash Tables 110

31 Bloom Filters 111
32 Cardinality 112
33 MinHash Sketch Strategy 113
34 Wrapping up CS 225! 114
Appendix A: Midterm Reviews 115
A.1 Midterm 0: Review of Prerequisite Material 115
A.2 Midterm 1: Lists 142

1 Introduction to CS 225

> **CS 225** - Covers the fundamental **data structures** (list, stack, queue, tree, etc.) and their implementations using C++ and object oriented programming along with an **introductory analysis to algorithms**

> **Prereqs** - Must have credit in both **CS 173/MATH 213** (Discrete Math) and **CS 128** (Introduction to Computer Science II in C++)

> **ECE Students** - Electrical/Computer Engineering (ECE) students may substitute CS 128 credit for **ECE 220** (Intro to Computing Systems), as both earn the same credit

As stated in the prereqs, **a strong knowledge in C++ is required** to be successful in this course! In other words, please ensure you have a **solid grasp** in CS 128/ECE 220 course material. Please see **Course Info and Resources** below for more info

1.1 Background

This course is crucial as it provides the foundation for writing efficient software, a skill highly sought after companies like **Google**, **Amazon**, and **Meta**. Mastery in this course will open pathways into working at such companies as they and many other employers rigorously evaluate your skill in writing software before prior to hiring you as a software developer.

1.2 Grading

- **36%** - Programming Assignments (MPs) (each MP is worth 6%)
- **12%** - Laboratory Assignments (each Lab is worth 10%)
- **36%** - Exams/Midterms (each Midterm is worth 6%)
- **16%** - Final Exam

1.3 Course Info and Resources

- CS 225 Lectures
- Full Syllabus is available here
- Review C++ Here
- C++ Reference Links
  - Online C++ Reference
  - Tutorial Based Approach of C++

- Textbook C++ Reference

2 C++ Review

Contacting the CS 225 course - cs225admin@lists.cs.illinois.edu

2.1 Encapsulation

Encapsulation in C++ is primarily distinguished via two things: **header** and **code**. In C/C++, code is separated into two files, one which we call the **header** and one which we call the **code segment**

- **header files** - Describes how the code is used, or how people should use this specific data structure
  - has a `.h` or `.hpp` extension. Mostly `.h`, but `.hpp` is used for **templates**, which help write **generic** code that supports any data type
- **code files** - Describes how the code works, or how the data structure is implemented!
  - has a `.cpp` file extension

Object Oriented languages also provide with two main things, a **constructor**, which initializes the necessary data for us in a object and a **destructor** which gets rid of any necessary data for us in an object

2.2 Memory Management - Ownership

Someone or some code owns everything, which simply means that **one part of the program is responsible** for

- Creating an object in memory (**Constructor**)
- Keeping track of it
- Deleting (freeing) it when it's no longer needed (**Destructor**)

This essentially means that the programmer is responsible for managing memory, which can both be powerful and dangerous. Forgetting to free allocated memory can cause **memory leaks**, or program crashes. In contrast, **freeing** memory too early can cause **dangling pointers** and **undefined behavior (segfaults)**. You can see both mistakes in **Figure 2.1**, where `h1` is leaked on the left and `p` is left dangling on the right.

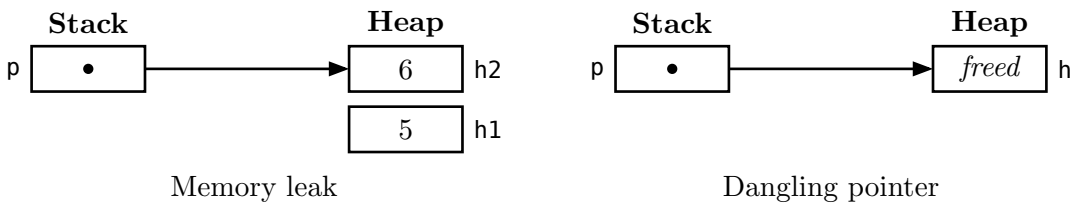


Figure 2.1: Example: after `p = new int(6);` the old block `h1` is unreachable (left); after `delete p;` the pointer still holds the freed address (right)

Ownership is tracked carefully using `new` and `delete`, or use smart pointers like `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`

2.3 Rule of Zero/Three/Five

When your class manages resources (**dynamic memory**, file handles, etc.), you need to define how it is **copied**, **moved**, and **destroyed** in order to avoid issues like memory leaks or double deletes

The rule of zero/three/five are rules widely adopted in C++ related to memory/resource management and ownership, specifically about when and how you should define or avoid defining special member functions in a class

Rule of Zero - If your class doesn't manage resources directly, do not define the **big five**

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Rule of Three - If you define one of a **destructor**, **copy constructor**, or **copy assignment operator**, **you should define ALL three of them!**

- This is because handling one implies your class is managing a resource that needs careful copying and cleanup
- Notice in **Figure 2.2** what goes wrong when only the destructor is defined: `a` and `b` point to the same block, so it gets deleted twice

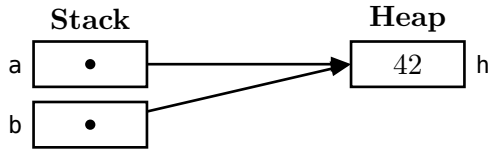


Figure 2.2: Example: with a destructor but the implicit copy constructor, `Foo b = a;` copies only the `data` pointer, so both destructors delete the same block

Rule of Five - Applies to modern C++, where if you plan on managing resources with move semantics, define **all 5** (this is the opposite of the **rule of zero**)

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Use this rule when you need to **avoid deep copying** and allow your class to be efficiently moved, like the move in **Figure 2.3**, where `b` takes over the heap block of `a` and nothing gets copied.

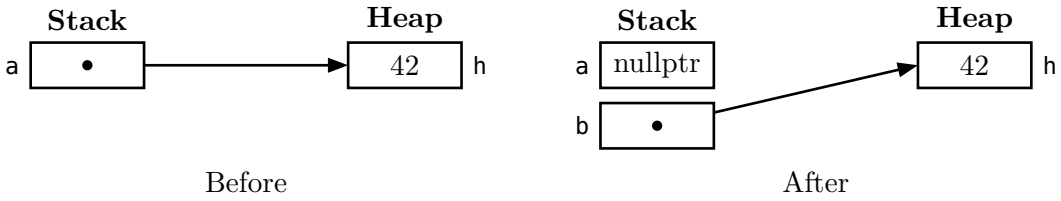


Figure 2.3: Example: `Foo b = std::move(a);` hands the heap block of `a` to `b` without copying it and leaves `a.data` as `nullptr`

3 List ADT and Linked List Implementation

ADT - Abstract Data Type
- ADTs are conceptual models that define a set of operations and behaviors for a data structure

List ADT Goals
- `insert` - Done via front/back, at index, or at pointer
- `delete` - Done via front/back, at index, or at pointer
- `getData` - Done via front/back, at index or at pointer
- `isEmpty`
- `size`
- **Store any necessary constructor, destructors, copy constructors, etc. so that it behaves like any basic variable**

The goal is to have every data structure behave like a fancy interface, but at a fundamental level act exactly the same as a basic data type! They should also behave like **templates** so that they can store anything (integers, pictures, imaginary numbers, etc.). We just want one generic data structure!

This is done via generics and templates!

There are two main implementations of a list
1. vector or array (store a list by allocating a chunk of data!)
2. linked list (allocate every element in a list separately and link them together via pointers or references)

Data structures can be best shown as diagrams, like the linked list in **Figure 3.1**

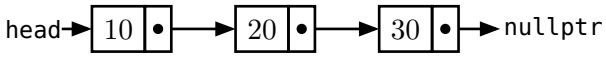


Figure 3.1: A linked list holding 10, 20, 30

Looking at **Figure 3.1**, the **head** is the **first node** in the linked list, and is in fact a **pointer** that points to the first node in the linked list.

Similarly, the **tail** is the **last node** in the linked list and it points to **nothing** (null pointer)!

This means that our List ADT must contain
- `head` - a pointer to a `ListNode`

Intuitively, this also shows that the linked list that contains several nodes, which begin with the value `10`, where then points to the next node of the linked list with value `20`, and so on

- From a C++ standpoint, this means that our List ADT must chain nodes together via **pointers**
  - Each node contains its own data along with a **pointer** that points to the next node in the List!

This is shown in the code below, which represents a generic `ListNode`, `ListNode<T>`. In this `ListNode` representation, each `ListNode` holds a generic type of data notated as `T`

Additionally, there is a pointer to the next `ListNode` in the List along with a copy constructor that can be used to copy data into the List!

```
#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};
```

3.1 `insertAtFront`

The `.hpp` extension is used to define functionality of templates defined in a `.h`. It typically contains the full definitions of template functions, enabling the compiler to generate code for each type instantiation based on the defined type

One of the functions that a good List data structure must implement is `insertAtFront`, which as the name suggests inserts a node to the front of the List!

To understand this better, let's use a concrete example, where `3` will be inserted at the front of the List in **Figure 3.2**

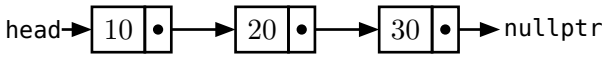


Figure 3.2: The list before inserting `3` at the front

1. In order to insert `3` at the front of the List, we need to create a new `ListNode` type! However, the data cannot get destroyed after the function ends because the List type needs to store this value for as long as it needs to
   1. Therefore, we must dynamically allocate this newly created `ListNode` via the heap, where we will receive a pointer to that data on the stack!
2. Once we dynamically allocate the new `ListNode`, we must reassign the `next` pointer to the head of the List because it will be inserted at the front!

1. In other words, the pointer to the next node is no longer null, like it was when newly created. Now it will point to the `head`!

3. Make `head` point to the newly created `ListNode`

After these steps, the List looks like **Figure 3.3**, where the new node (shaded) now sits in front of `10`

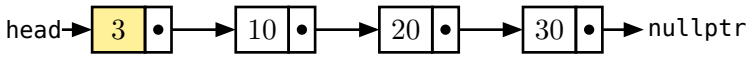


Figure 3.3: The list after `insertAtFront(3)`

In C++, this is what the `insertAtFront` algorithm would look like

```
// List.hpp

#include "List.h"

/** Inserts data at the front of a List
  * Runtime: O(1)
 */
template <typename T>
void List<T>::insertAtFront(const T& data) {
    // Dynamically allocate the data to the heap
    ListNode<T>* newNode = new ListNode<T>(data);

    // Reassign the next pointer to the head
    newNode->next = head;

    // Reassign the head pointer to the newly created node
    head = newNode;
}
```

This algorithm has a runtime of $O(1)$, due to three simple operations and doesn't loop or scale through a list size!

Recall what `->` (arrow operator) does in C++. It's a nice operator that dereferences the pointer and then calls an attribute/method on that pointer! `newNode->next` is the mere equivalent of `(*newNode).next`

3.2 `insert`

The previous algorithm only inserted nodes at the front of the List. However a more general insert algorithm is recommended to insert nodes in more places than just the front!

To solve this problem, the best thing to do is to create a `insert` algorithm that is able to insert data anywhere at a given index of the list. This is shown in our updated `LinkedList` ADT code below

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template <typename T>
class List {
private:
    // a pointer to the head (front) of the linked list
    ListNode<T>* head = nullptr;

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);
};
```

When the index is 0, `insert` would behave exactly how `insertAtFront` works

To understand this much trickier algorithm, we can use the same concrete List example from `insertAtFront`, with example function call `insert(3, 1)`

To make this call take place, we need to find the node at the first index and insert the newly created node before that. To do this, our insert algorithm must do two following steps

1. Allocate the given data onto the heap
2. Find the place to insert the newly created node (typically the code semantics are before)
    1. Therefore, we need a helper `find` function to return the node by reference at the given index. Our updated `LinkedList` ADT is represented in the code below

```
// List.h
```

```
#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template <typename T>
class List {
private:
    // a pointer to the head of the linked list
    ListNode<T>* head = nullptr;

    /** Helper for the recursive version of `find`
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index, ListNode<T>*& head);

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);

    /** Finds the `ListNode` at the given index!
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index);
};
```

3.2.1 `find`

1. To implement `find`, it should return a `ListNode*&`, which is **a `ListNode` pointer by reference**
    1. Recall that with return type semantics in C++, returning just a `ListNode*` won't actually allow us to change the original `ListNode*`. Doing so will create a copy of the `ListNode*` and instead modify the **copy** instead of the original pointer. Therefore, we need to return a **reference** to the original pointer, which is a type `ListNode*&`

2. Recall that `insertAtFront` changed our `head` member variable. Therefore, `head` needs to be the pointer pointing at the given index needed to be inserted
3. Implement this **iteratively**. Implementing **recursively** will cause a **Stack Overflow Error**, where no more stack frames can be placed on the call stack, resulting in **stack memory overflow**
    1. Regardless, thinking about the recursive solution can still be a good thing to do
        1. When `index == 0`, we know that this is the `head` of the List, or simply the first node, otherwise we return a recursive function call, which subtracts the index by 1, and then passes in the next node of the List. This recursive algorithm is shown in the first code snippet below
    2. To implement find iteratively, we can follow a fixed algorithmic step
        1. Return the `head` ListNode if the index is `0`. We can do this because we know the `head` is always at index `0`. We also return `head` if the List is empty, since there's nothing to walk through
            1. Recall that this method returns a `ListNode*&`, which is a **pointer reference**. `head` is in fact a **reference to a pointer**!
        2. Create a temporary `ListNode`, `curr` that is assigned to head. This represents the current node being referenced in the `LinkedList`
        3. Otherwise, loop while `x` is less than `index - 1`, and reassign `curr` to the next node in the `LinkedList`
            1. We also stop early if `curr->next` is null, so `curr` never walks off the end of the List and becomes null. An index past the end just lands at the end of the List
        4. Then return `curr->next` because we know that once `x` becomes `index - 1`, we can stop and just return the a reference to the next node in the `LinkedList`, hence `curr->next`
            1. Recall that `curr` is a **copy** of head! We don't want to return a copy `ListNode*`, but rather a reference `ListNode*`, which is simply a type `ListNode*&`
            2. Therefore, we return `curr->next` and loop from `x` to `index - 1`, instead of looping from `x` to `index` and returning just `curr`
        5. This process is shown in the second code snippet below. You can trace it for `find(2)` on the List from **Figure 3.2** in **Figure 3.4**, where `curr` stops at the shaded node and returns its `next`

The code below shows implementing `find` **recursively**

```
// List.hpp

#include "List.h"

// RECURSIVE IMPLEMENTATION OF FIND!

template <typename T>
ListNode<T>*& List<T>::find(int index) {
    return find(index, head);
}

template <typename T>
```

```
ListNode<T>*& List<T>::find(int index, ListNode<T>*& head) {
    // we know that the head is always the starting point at index 0 of the list,
so we can just simply return the head
    if(index == 0) { return head; }

    // Return a recursive method call passing in the next node and decrement the
given index!
    return find(index - 1, head->next);
}
```

The code below shows implementing `find` **iteratively**

```
// List.hpp

#include "List.h"

// ITERATIVE IMPLEMENTATION OF FIND!

/** Finds the ListNode at the given index!
 *  Runtime: O(n)
 */
template <typename T>
ListNode<T>*& List<T>::find(int index) {
    // Return the head if the index is 0
    // We know that the first node in the list is always the head!
    if(index == 0 || head == nullptr) {
        return head;
    }

    // Create a copy pointer of the head to traverse through the list!
    ListNode<T>* curr = head;

    // Traverse through the list to find the node right before the given index
    // Stop early at the last node so `curr` never becomes null
    int x = 0;
    while(x < index - 1 && curr->next != nullptr) {
        curr = curr->next;
        x += 1;
    }

    // Return the next node in the list, since that matches the node at the given
index
    return curr->next;
}
```

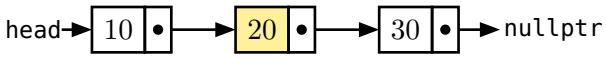


Figure 3.4: `find(2)` stops with `curr` at the shaded node (index 1, holding `20`) and returns its `next` pointer by reference, which points to the node at index 2 (holding `30`)

We can also update our current implementation of the `LinkedList` ADT, shown below

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template<typename T>
class List {
private:
    // a pointer to the head of the linked list
    ListNode<T>* head = nullptr;

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);

    /** Finds the `ListNode` at the given index!
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index);
};
```

Runtime for our `find` function is $O(n)$ because we have to traverse up to n nodes in the `LinkedList` to find the node at the given `index`

Now that the `find` function has been coded, we can continue with finishing our `insert` function. Recall that this is a generic function, which will insert data into the `LinkedList` at a given index. Recall that this algorithm is done in three steps

1. Call `find` to find the `ListNode` at the given index
    1. Recall that `find` returns a reference of a pointer to a `ListNode`, written as a type `ListNode*&`
2. Allocate the given data onto the heap
    1. Returned pointer should not be a reference pointer! Make just a regular pointer
    2. Recall that a reference pointer refers to an existing pointer! There is no existing pointer here, we make a new one when allocating onto the heap
3. Make the newly created node point to the found `ListNode` at the given index
4. Set the node found at the given index's pointer to the newly created node

This algorithm is shown in the code below

```
// List.hpp

#include "List.h"

template <typename T>
void List<T>::insert(const T& data, int index) {
    // Call `find` to find the `ListNode` at the given index
    ListNode<T>*& nodeAtIndex = find(index);

    // Dynamically allocate the data to the heap
    ListNode<T>* newNode = new ListNode<T>(data);

    // Reassign the next node of the newly created node to be the `ListNode` found
at the given index
    newNode->next = nodeAtIndex;

    // Reassign the found `ListNode` at the given index's pointer to the newly
created node from the given data
    nodeAtIndex = newNode;
}
```

Let's go back to the List from **Figure 3.2** and call `insert(3, 1)`. As you can see in **Figure 3.5**, the shaded node holding `20` is the node at index 1 before the insert. Afterwards, the new node holding `3` (shaded) takes over index 1 and points to `20`, and the `next` pointer returned by `find` (the one inside `10`) now points to the new node

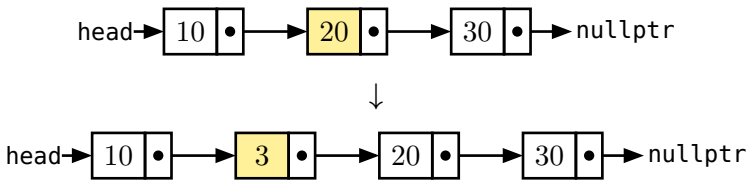


Figure 3.5: Inserting `3` at index 1 of the List from **Figure 3.2**

`insert` has a $O(n)$ runtime, since it needs to traverse to the given position/index and then insert the newly created node of the given data

3.3 `operator[]`

The next function our `LinkedList` ADT must have is the operator `[ ]`. Recall that in C++ we can operator overload with a function that changes the behavior of the `[ ]` operator.

The brackets operator will behave very similarly to the `find` function, but rather return the `data` by reference that the `ListNode` at the given index contains. This updates our `LinkedList` class like this (shown below)

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    // the current data in the list
    T data;

    // a pointer to the next node in the linked list
    ListNode* next;

    // a copy constructor for the list
    ListNode(const T& data) : data(data), next(nullptr) { }
};

template <typename T>
class List {
private:
    // a pointer to the head (front) of the linked list
    ListNode<T>* head = nullptr;

public:
    /** Inserts data at the front of a List
    * Runtime: O(1)
    */
    void insertAtFront(const T& data);

    /** Inserts data in the given index of a list
    *  Runtime: O(n)
    */
    void insert(const T& data, int index);

    /** Finds the `ListNode` at the given index!
    *  Runtime: O(n)
    */
    ListNode<T>*& find(int index);
```

```
    /** Returns the data by reference of the `ListNode` at the given index
    *   Runtime: O(n)
    */
    T& operator[](int index);
};
```

This algorithm, therefore, works very simple

1. Find the `ListNode` at the given index (we already did this with the `find` function!)
2. Return that `ListNode`'s data (Every `ListNode` instance holds its own data!)

And the code for this function is below

```
// List.hpp

#include "List.h"

/** Returns the data by reference of the `ListNode` at the given index
 *  Runtime: O(n)
 */
template <typename T>
T& List<T>::operator[](int index) {
    // Call `find` to find the `ListNode` at the given index
    ListNode<T>* loc = find(index);

    // Return the data that the `ListNode` at the given index holds
    return loc->data;
}
```

The runtime for `operator[]` is also $O(n)$ because it takes n times to traverse through the list to find the `ListNode` at the given index to then return its data!

3.3.1 `remove`

The next function is to remove data from our `LinkedList` at a given index. This updates our `LinkedList` class structure, as shown below!

```
// List.h

#pragma once

template <typename T>
struct ListNode {
    T data; // the current data in the linked list
    ListNode<T>* next; // a pointer to the next node in the linked list
    ListNode(const T& data) : data(data), next(nullptr) { }
};
```

```
template<typename T>
class List {
private:
    ListNode<T>* head = nullptr; // a pointer to the head of the linked list

public:
    void insertAtFront(const T& data);
    void insert(const T& data, int index);
    ListNode<T>*& find(int index);
    T& operator[](int index);
    T remove(int index);
};
```

The algorithm for this is once again simple

1. Find the `ListNode` at the given index, with our helper `find` function, let's call it `toRemove`
2. Create a `temp` `ListNode` containing the `ListNode` to be removed, let's call it `temp`
3. Create a variable containing `temp`'s `data`, can call this `val`
4. Set toRemove to the next `ListNode` in the `LinkedList`
5. delete the `temp` `ListNode` from memory (`temp` was what was removed)
6. return `val`

The code below shows this algorithm in C++

```
// List.hpp

#include "List.h"

/** Removes a `ListNode` by index and returns its associated data!
 *  Runtime: O(n)
 */
template <typename T>
T List<T>::remove(int index) {
    // Find the `ListNode` to be removed
    ListNode<T>*& toRemove = find(index);

    // Create a temporary `ListNode` containing what should be removed
    ListNode<T>* temp = toRemove;

    // Create a `val` variable to store temp's data
    T val = temp->data;

    // Reassign `toRemove` to the next node in the `ListNode`
    toRemove = toRemove->next;

    // Delete `temp` from memory since we no longer need it
    delete temp;
```

```
    return val;
}
```

If we call `remove(1)` on that same List from **Figure 3.2**, we get **Figure 3.6**. Notice how the node holding `10` now points past the shaded node holding `20` to `30`, the node holding `20` is deleted, and `20` is returned

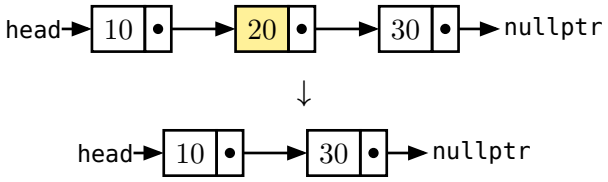


Figure 3.6: `remove(1)` removes the node holding `20`

A couple things about this algorithm

- You need a pointer to the node you're removing so you can later delete it, which is what we call `toRemove`
- Creates a copy pointer to the node being removed called `temp`. We keep a separate reference so that we can return its data in the function
  - Without `temp`, once `toRemove = toRemove->next`, the previous node that `toRemove` stored is now lost
  - We need the original node that was stored in `toRemove` in order to be able to return the data that was removed from the `LinkedList`, which is why we store it in `temp`!
- `delete temp`, ensures the memory allocated to `temp` is freed. We do this call because we know that the `LinkedList` no longer needs this `temp ListNode` (**`LinkedList` no longer needs to track it!**)

This wraps up the first implementation of a List, often referred to as a `LinkedList`

4 ArrayList: An Array Implementation of a List

There is another way to implement a `List`, which is by using an Array. This is simply implementing a C++ `std::vector`, or in broader programming terms, an `ArrayList`

The primitive foundation of an `ArrayList` is based on the idea of storing elements in a chunk of contiguous memory, allowing efficient access and manipulation. The list starts with a raw array and builds abstraction over it to support dynamic resizing, safe memory handling, and user-friendly operations, like the ones covered in `LinkedList`

We begin by figuring out what we need to store the list. In the `LinkedList`, we stored individual `ListNodes`, where they each held data along with a pointer to the next node, which allowed for easy chaining access between the nodes

However, in an `ArrayList` approach, we need to simply store a chunk of memory somehow, while this is typically a `std::vector` in C++, we are actually implementing this, so we need a primitive array, as mentioned previously. Therefore, a chunk of memory is allocated using a **pointer**, where we can call this a generic type `T`. This is reflected in our `ArrayList` class below

```
// ArrayList.h

#pragma once

template<typename T>
class ArrayList {
public:

private:
    // A pointer to the start of a chunk of memory
    T* data = nullptr;
};
```

`data` is a pointer to a chunk of memory that will be allocated to the heap

In addition to this, we need to store the **size** and **capacity**, where the **size** represents how many items are actually in the list and the **capacity** represents how much space is available in the memory block

These both could be `ints`, or they could be pointers, where `size` represents a pointer to the end of the used portion of the array and `capacity` represents a pointer to the end of the allocated chunk of memory, or buffer. This approach is reflected in our code below,

```
// ArrayList.h

#pragma once

template<typename T>
class ArrayList {
public:

private:
    T* data = nullptr;
    T* size = nullptr;
    T* capacity = nullptr;
};
```

For example, the list in **Figure 4.1** holds 3, 4, 5, so it has room for three more elements before `size` catches up to `capacity`.

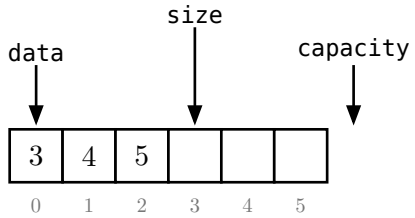


Figure 4.1: Example: an `ArrayList` holding 3, 4, 5 in a buffer of 6 slots. `size` points one past the last element and `capacity` points one past the end of the buffer

Now, the next idea to think about this `ArrayList` implementation is how do we add more space to the buffer? If the capacity is full, meaning that size and capacity both point to the same area in the buffer, then how would I resize the buffer to add additional data into the buffer?

The solution is to allocate some new buffer somewhere with a bigger space. Then, copy the data from the previous buffer along with its pointers (`size`, `capacity`, and `data`). This approach will be done with a private method, `addSpace()`. Our `ArrayList` ADT is updated below. Recall that this is a helper method that resizes the buffer, so therefore, it should be private.

```
// ArrayList.h

#pragma once

template<typename T>
class ArrayList {
public:

private:
    T* data = nullptr;
```

```
    T* size = nullptr;
    T* capacity = nullptr;

    void addSpace();
};
```

Notice that this is very expensive, which is why the `LinkedList` approach is often much more recommended

Our helper `addSpace()` function must add data to the end of the allocated chunk of memory. However, this causes issues when the allocated chunk of memory is full with data. Recall that this occurs when `size == capacity`.

- To solve this problem, we simply need to allocate a new buffer with a bigger capacity and then copy over the previous buffer's data along with the respective pointers to the buffer including `data`, `size`, and `capacity`
  - You can see this copy in **Figure 4.2**, where the four elements of the full buffer (highlighted) move into a buffer twice as big

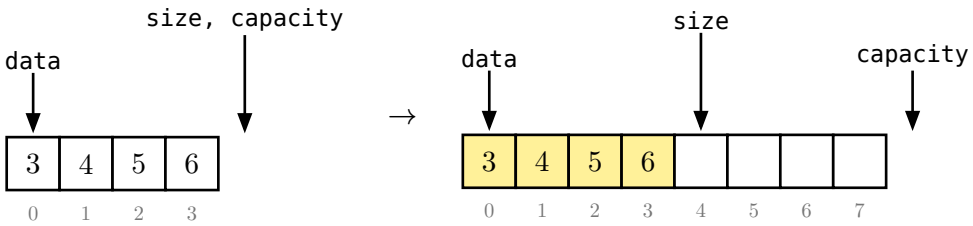


Figure 4.2: Example: a full buffer (`size == capacity`) is copied into a bigger buffer. Highlighted cells are the copied elements

We will come back to this `addSpace()` helper function, but for now we will talk about the main public function that will use `addSpace()` called `pushBack()`, which is one very common function in a `std::vector`! Recall, that it simply inserts an element at the end of the list

We can update our `ArrayList` ADT like this,

```
// ArrayList.h

#pragma once

template<typename T>
class ArrayList {
public:
    void pushBack(const T& value);
private:
    T* data = nullptr;
    T* size = nullptr;
    T* capacity = nullptr;
```

```
    void addSpace();
};
```

4.1 `pushBack()`

The algorithm for `pushBack()` becomes simple with our helper `addSpace()` function

1. Check if `size == capacity`. If so, then call helper function `addSpace()` to add more space into the function
2. Then, dereference the `size` pointer and set it equal to the new `value` parameter variable
   - we do this because this is how we place the given element at the end of the current end of the buffer
   - Remember that `size` tracks the end of filled data in the buffer and `capacity` tracks the end of the entire buffer!
3. Add 1 to the `size` pointer to set the pointer to the new current end of the buffer

The code for this algorithm is shown below

```
// ArrayList.hpp

#include "ArrayList.h"

/** Inserts data to the back of this `ArrayList`
 *  Runtime: O(1)
 */
template <typename T>
void ArrayList<T>::pushBack(const T& value) {
    // Add additional space to the buffer if `size` == `capacity`
    if(size == capacity) {
        addSpace();
    }

    // Place the new element at the current end
    *size = value;

    // Increment the `size` pointer to now point to the new end of the filled
buffer
    ++size;
}
```

Let's trace a single call in **Figure 4.3**. There's still room in the buffer, so `pushBack(6)` skips `addSpace()` and just drops 6 into the highlighted slot.

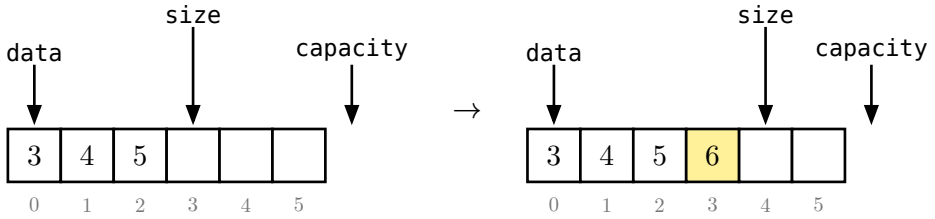


Figure 4.3: Example: `pushBack(6)` writes 6 at `*size`, then moves `size` forward one slot

5 Amortized Analysis

Amortized Analysis is a technique commonly used in Computer Science to analyze the **average running time** per operation over a sequence of operations in an algorithm. Rather than focusing on the worst case cost for a single operation (this is simply **Big-O**), amortized analysis evaluates the overall cost when spread out evenly across all operations performed.

This is useful because even though some operations in an algorithm can cost a lot more than others, the average cost per operation is low when many operations are performed consecutively.

5.1 Resizing by adding 2 elements every time

One way to implement our `addSpace()` function is to resize the buffer by increasing the `capacity` by 2 each time the array is full. This means that when more space is needed, `addSpace()` allocates a new array with a capacity of `capacity + 2`. Recall that after this, `addSpace()` then needs to copy all the data from the previous buffer to the new buffer and then update the buffer's internal pointers.

Since `addSpace()` needs to copy all the data from the previous buffer to the new buffer when updating its capacity by `capacity + 2`, this means that the algorithm has a runtime of $O(n)$, where n represents the number of elements currently in the array. This is because each element must be copied one by one into the new buffer, as you can see from the highlighted cells in **Figure 5.1**, where each resize copies 2 more elements than the last.

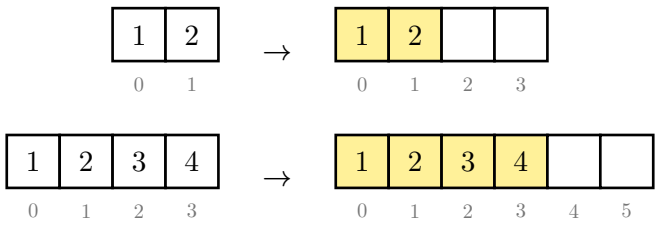


Figure 5.1: Example: pushing 1, 2, 3, 4, … with `capacity + 2` resizing. Highlighted cells are copied: 2 copies, then 4, then 6, and so on

Let r represent the number of resizes, where a resize occurs when the ArrayList is full (or simply every time two elements are added).

Then, the total runtime cost of this algorithm, where there are 2 copies made every 2 iterations, can be represented through the summation

$$\sum_{k=1}^{r} 2k$$

Since r represents the number of resize operations every two iterations, the summation is equivalent to $2 \times \frac{r(r+1)}{2} = r(r+1) = r^2 + r$. Therefore,

$$\sum_{k=1}^{r} 2k = r^2 + r$$

However, r is the number of resize operations every two iterations. When analyzing algorithmic runtime, it is important to express it in terms of n rather than r because an algorithm is expressed in terms of its input size, which is simply n.

If there were 5 resizes that occurred every 2 elements, then the total number of elements, n, would be 10, which can be mathematically expressed as $n = 2r$, or simply $r = \frac{n}{2}$

Substituting n for r becomes

$$= \left(\frac{n}{2}\right)^2 + \frac{n}{2}$$
$$= \frac{n^2 + 2n}{4}$$

Therefore, the total time for n inserts is $\frac{n^2+2n}{4}$.

The total amortized cost is then calculated by dividing by n, the total number of operations performed on an algorithm

$$\frac{\frac{n^2+2n}{4}}{n} = \frac{n^2 + 2n}{4n} = \frac{n(n+2)}{4n} = \frac{n+2}{4} = \frac{1}{4}n + \frac{1}{2}$$

making the total amortized cost $O(n)$ (since constants are dropped)

5.2 Resizing by doubling the elements every time

Now suppose, we resized the `ArrayList` by doubling the capacity every time. Each time we resize, the time it takes to do so doubles from 1, 2, 4, 8, 16, etc. This is because each resize involves copying all the elements from the old array into a new array that is twice as large.

Let r represent the number of resizes, where a resize occurs when the `ArrayList` is full (or simply every time two elements are added)

When the `ArrayList` is full, its capacity will resize by double the original capacity.

Recall that each time we resize, the time it takes to do so doubles from 1, 2, 4, 8, 16, etc. This is because each resize involves copying all the elements from the old array into a new array that is twice as large. Therefore, this results in 2^r copies per resize. Notice in **Figure 5.2** how the highlighted copies go 1, 2, 4 as the number of resizes grows

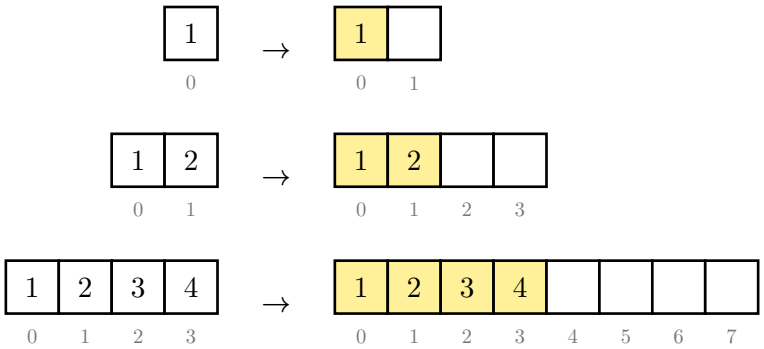


Figure 5.2: Example: pushing 1, 2, 3, 4, … with doubling. Highlighted cells are copied: 1, then 2, then 4, so each resize buys as many free slots as it copied

Therefore, the summation that represents the runtime of this algorithm is

$$\sum_{k=0}^{r} 2^k$$

This summation is equivalent to the sequence $1 + 2 + 4 + \cdots + 2^{r-1} = 2^r - 1$

However, in an amortized sense, we need to multiply this by the number of operations we are doing. We are doubling the capacity and inserting a new element into our buffer. This means we have to multiply this runtime cost by our insertions. The $+1$ at the end represents the initial operation and constant cost that isn't captured by the summation. Therefore,

$$\sum_{k=0}^{r} 2^k = 2(2^r - 1) + 1$$

where $2^r = n$ and $r = \log_2 n$. Therefore,

$$\sum_{k=0}^{r} 2^k = 2\big(2^{\log_2 n} - 1\big) + 1 = 2(n-1) + 1 = 2n - 1$$

where $2n - 1$ represents the total copies with n inserts, meaning that the amortized cost is represented by

$$\frac{2n-1}{n}$$

which simplifies to $2 - \frac{1}{n}$. When analyzing this behavior's function, the

$$\lim_{n \to \infty} \left(2 - \frac{1}{n}\right) = 2$$

which means that it approaches a constant value of 2. Therefore, its amortized cost is $O(1)$

Since the amortized cost when doubling the capacity each resize is $O(1)$, it makes more sense to implement the `addSpace()` function using this resizing method

Now to implement our `addSpace()` function into our data buffer, we need to do a series of steps following our resize approach

1. Reallocate the data buffer, typically done by doubling its capacity (the current capacity is `capacity - data`, since both are pointers)
2. Check if the original capacity is 0. If so, set the new capacity to 1. This is to ensure memory efficiency
3. Allocate a new data buffer equivalent to the new capacity (which was double the original capacity)
4. Calculate the number of elements currently in the `ArrayList` (recall that this can be done with our `size` and `data` pointers by doing `size - data`)
5. Copy all contents from the old data buffer into the new data buffer
6. Free all the contents from the old data buffer and update the pointer of the new data buffer
7. Update the `capacity` pointer to point to the end of the new buffer and update the size pointer to point to the correct position in the new buffer

The code for this algorithm that reflects these series of steps is shown below,

```
// ArrayList.hpp

#include "ArrayList.h"

/** Allocates additional space to the buffer in this `ArrayList`
 *  Runtime: O(n)
 */
template <typename T>
void ArrayList<T>::addSpace() {
    /* Reallocate the capacity of the buffer (similar to realloc in C)
     * Typically that of double the original capacity
     */
    int newCapacity = (capacity - data) * 2;

    // If the original capacity is 0, set it to 1 to be minimal and memory
efficient
    if(newCapacity == 0) {
        newCapacity = 1;
    }

    // Allocate a new data buffer that is set to the updated capacity
    T* newData = new T[newCapacity];

    // Calculate the number of elements currently in the `ArrayList`
    int currentSize = size - data;

    // Copy all contents from the old data buffer into the new data buffer
    for(int i = 0; i < currentSize; i++) {
        newData[i] = data[i];
    }

    // Free the contents from the old buffer from memory
    delete[] data;
```

```
    // Update the data pointer to point to the newly allocated buffer
    data = newData;

    // Point `capacity` to the end of the new buffer
    capacity = data + newCapacity;

    // Update the size pointer to point to the correct position in the new buffer
    size = data + currentSize;
}
```

This will effectively add more space to our buffer (double that the original) when its capacity is full

5.3 Amortized Costs of various operations on various `List` implementations

Operation	**Singly Linked List**	**Array**
Insert/Remove at front/back	`O(1)`	`O(1)`* (amortized)
Insert/Remove given a pointer	`O(1)`	`O(n)`
Insert/Remove at a given index	`O(n)`	`O(n)`
Find a value in a list	`O(n)`	`O(n)`
Find a value in a sorted list	`O(n)`	`O(log n)`

The * represents **amortized** cost, which means the average cost per operation over a sequence of operations. Whether to use the amortized case depends on the application you are building. Amortized analysis isn't ideal for explaining the cost of a single, individual operation, but rather useful for understanding the average cost spread across many operations.

6 Stacks and Queues

One of the biggest problems with both `ArrayList` and `LinkedList` is that accessing elements can sometimes be slow. Recall that the runtime complexity is `O(1)` to remove an element at the beginning and end and `O(n)` for elements not at the beginning/end.

Therefore, if we want `O(1)` insertion and removal, we need a faster data structure, which introduces Stacks and Queues.

This lecture will introduce Stacks, but will only fully cover the Queue implementation. The next lecture will cover the Stack implementation in depth along with introducing another new topic

6.1 Queues

Queues are everywhere and are used in an enormous amount of computer hardware. A Queue is like a list, but with a fixed order of **FIFO (First In First Out)**.

FIFO is a good mnemonic that helps remember how queues fundamentally work. **This means that the first element in the queue will be the first element out of the queue**. Some may also refer the FIFO mnemonic as LILO - Last In Last Out, but the idea behind remembering what a queue does is still fundamentally the same.

A queue is like a **line of people** (funnily enough a line in British English is called a queue!). Suppose people would line up to ride on the scariest rollercoaster in town. The first person who is in line will be the first person who will leave the line to go on the rollercoaster. Similarly, if someone joins the line for the rollercoaster they will be placed at the end of the line.

Queues have a simple implementation because they only really need two core functionalities, `enqueue()` and `dequeue()`.

- `enqueue()` will add elements to the end of the queue
- `dequeue()` will remove elements at the front of the queue

These functions are both `O(1)`, meaning adding and removing is constant time. This is because these functionalities happen at the back and front of the queue for `enqueue()` and `dequeue()` respectively, meaning we don't have to deal with the other elements in the queue. In **Figure 6.1**, for example, 5 joins at the back and 3 leaves from the front, and 4 never moves.

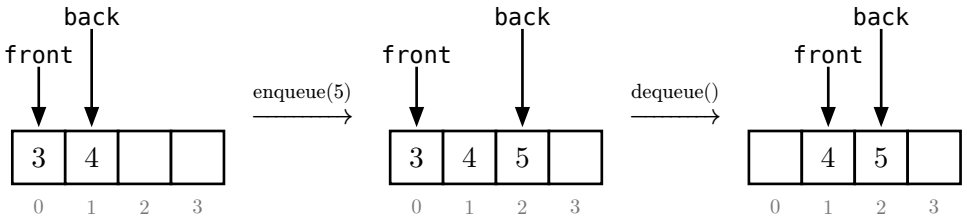


Figure 6.1: Example: `enqueue(5)` joins at the back, then `dequeue()` removes 3 from the front

6.2 Stacks

Like a Queue, a Stack also has a fixed order, but it works a little bit differently than a Queue. This order is referred to as **LIFO (Last In First Out)**.

LIFO is a good mnemonic that helps remember how stacks fundamentally work. **The last item put into the stack will be the first item out of the stack**. Some may also refer to the LIFO mnemonic as FILO - First In Last Out, but the idea behind remembering what a stack does/will do is still fundamentally the same.

A Stack is best described when thinking about a stack of plates in the kitchen. You add plates to the top, but remove plates from the top. You can't touch any other plate underneath until it appears at the top (well you could but the plates on top will topple over and break!).

Stacks also a simple implementation because they also only really need two core functionalities, `push()` and `pop()`.

- `push()` will add elements to the top of the stack
- `pop()` will remove elements from the top of the stack

If we store a stack in an array, both operations happen at `top`, as you can see in **Figure 6.2**: `push(5)` puts 5 on top and `pop()` takes that same 5 right back off.

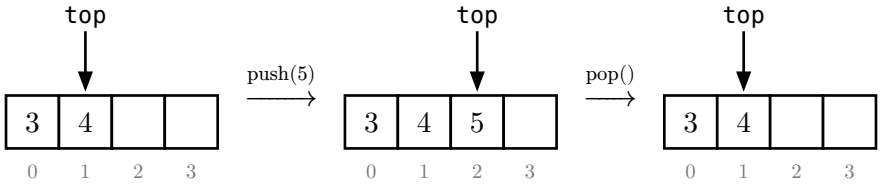


Figure 6.2: Example: a stack stored in an array. `push(5)` and `pop()` both work at `top`, so `pop()` returns 5, the last element pushed

These functions are also both `O(1)`, meaning adding and removing is constant time. This is because these functionalities both happen at the top of the stack, meaning we don't have to deal with the other elements in the queue.

6.3 Stacks vs. Queues

- Stacks operate upon the mnemonic LIFO - **Last In First Out** (last element in is the first element out), while Queues operate upon the mnemonic FIFO - **First In First Out** (first element in is the first element out)
- Insertion/Removal on a Stack happens at the same end (the top), while Insertion/Removal on a Queue happens at different ends, which are the back and front respectively.
  - Regardless of this, insertion/removal are `O(1)` because they happen at the ends of the Stack/Queue

6.4 `CircularQueue` Implementation - `enqueue` and `dequeue`

Take a look at the following `CircularQueue` implementation below

```
// CircularQueue.h

#pragma once

template <typename T>
class CircularQueue {
public:
    // Adds an element to the rear of the queue
    void enqueue(T e);

    // Removes and returns the element at the front of the queue
    T dequeue();

    // Returns `true` if the queue is empty; `false` otherwise
    bool isEmpty();
private:
    // Pointer to the dynamically allocated array holding the queue elements
    T* items;

    // Total capacity of the queue (number of elements it can hold)
    unsigned int capacity;

    // Current number of elements in the queue
    unsigned int size;
};
```

This queue is an Array implementation, where data is stored in an array that `items` points to.

Before implementing this queue, we need to understand how the functionality of `enqueue()` and `dequeue()` fully work.

- Recall that `enqueue()` adds elements to the back of a queue. From a code standpoint, we insert elements by writing `items[size]`, inserting at the next most empty place in our `items` array
  - Recall that `capacity` represents the total length of the `items` array, while `size` represents the total number of elements the array has

- Suppose we called `enqueue(3)`. This inserts 3 at the back of the array, where `items` now has a size of 1
  - This means that if we were to call `enqueue(4)`, this means that `items[1]` will in fact represent the proper position to place the next element in the queue, which in this case is 4.
- After both calls, `items` looks like **Figure 6.3**, with 3 at index 0, 4 at index 1, and two empty slots left over

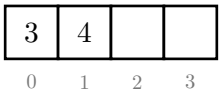


Figure 6.3: `items` after `enqueue(3)` and `enqueue(4)`, with a capacity of 4

- `dequeue()` removes elements from the front of a queue. From a code standpoint, we return the element at the front of a queue and then return it.
  - However, in order to properly implement this, we need to track where to remove elements from. This means adding a new variable, called `start`, where `items[start]` represents the element to remove from a queue. `size` would then decrement by 1 since an element was removed.
  - Since `items[start]` removes elements from a queue, `items[size]` will no longer work to insert additional elements from a queue.
    - This is because `items[size]` represents the last element in `items`, not the next index in the array to insert the new element.
    - To fix this issue, we can write `items[size + start]` to properly insert new elements into the queue. This is because start represents the index in `items` to remove an element, and size represents the number of elements `items` contains, so the next available spot in memory would be `start + size`
  - Remember that this doesn't free the data from memory, the data is still occupied in memory, it's just that the queue no longer considers the dequeued element to be part of the queue, which is done programmatically by simply updating `start`
  - Notice in **Figure 6.4** how `start` moves from 0 to 1 after a `dequeue()`, even though the 3 is still sitting in `items[0]`

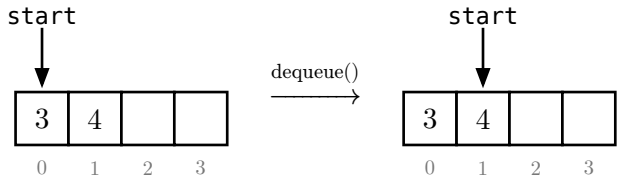


Figure 6.4: Example: `dequeue()` returns `items[0]` (3) and moves `start` to 1. The 3 is still in memory but is no longer part of the queue, and the next `enqueue` writes to `items[start + size]`, index 2

This is great, `enqueue()` and `dequeue()` work perfectly, until we come across another core issue. What if `items` is full? Since `items` is full and we were to insert elements using `items[start + size]`, our index would simply be out of bounds, since `start + size` exceeds the total size of `items`

- To fix this problem, we need a way to loop back from the beginning of `items`, and in programming this is referred to as the modulo operator `%`.
  - If we modulo `start + size` by capacity, we ensure that once the index reaches the end of the array, it wraps around back to the beginning, which effectively prevents an index out of bounds access into `items`
  - Therefore, `items[(size + start) % capacity]` will insert elements into `items` and `items[start % capacity]` will remove elements from `items`
    - This works because when we call `items[(size + start) % capacity]` and `items[start % capacity]`, `start` ensures that we overwrite memory that has already been removed by the queue.
  - In **Figure 6.5**, for example, `enqueue(7)` wraps around to index 0, right where the dequeued 3 used to be

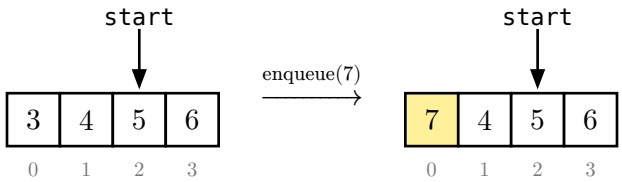


Figure 6.5: Example: with `start = 2`, `size = 2`, and `capacity = 4`, the queue is 5, 6. `enqueue(7)` writes to index `(2 + 2) % 4 = 0`, wrapping around over the already dequeued 3

So great, this queue implementation is amazing now, but there is still one more problem. What if the queue is actually full, where `size == capacity`? (recall that this typically happens when you enqueue a bunch of elements to the point where there is no more available allocated memory in `items` to place another element)

- To solve this problem, we use the same approach as the `ArrayList`.
  - Create a new container by doubling the `capacity` and then copy all elements from `items` into the new container
  - Update the `items` pointer to point to the newly made container.
- However, this implementation poses another problem because the `start` of the queue is no longer the `0`th element.
  - To solve this, when copying, we set the elements in the new container to correspond to the order of the queue by copying `items[start]`, then `items[start + 1]`, `items[start + 2]`, and so on, wrapping around as needed
  - You can see this copy in **Figure 6.6**: the wrapped queue 5, 6, 7, 8 starts at index 2, but after copying it starts back at index 0

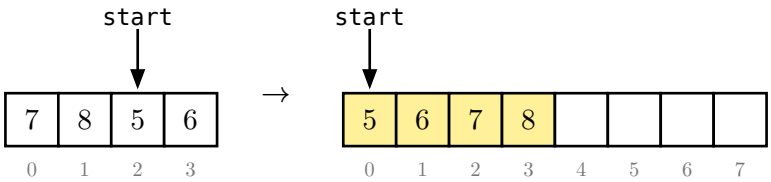


Figure 6.6: Example: a full queue 5, 6, 7, 8 that has wrapped around. Copying from `start` in queue order puts it back at index 0 of the doubled array

This effectively describes implementing a queue in depth via a `CircularQueue`. In a `CircularQueue`, we use a dynamically allocated array, like `items` that circles around when inserting an element goes outside the array's dynamically allocated memory!

Typically from a hardware standpoint, we don't resize a queue. In hardware, memory is already preallocated and doesn't change at runtime.

7 Iterators

One concept that is often in the scope of DSA is the **iterator**. An iterator is an object that traverses through all the elements/items in a data structure.

Suppose we had 3 data structures, `Array`, `LinkedList`, and a `Hypercube` (also known as a tesseract and is a 4D cube). How would we be able to traverse through every element in this data structure?

The solution is simply an iterator, which provides encapsulated access to our data in a data structure. Each data structure, `Array`, `LinkedList`, and `Hypercube` has its own way of iterating through its data

- For example, an `Array` typically uses an index to iterate, where `index++` traverses through each element in an array
- Similarly, a `LinkedList` uses a pointer to iterate, where `cur->next` represents a pointer to the next `ListNode` in a `LinkedList`. This is shown in the table below

7.1 Iterate through data in each data structure

Data Structure	Current Location	Current Data	Next
`LinkedList`	`ListNode*` (named `cur`)	`cur->data`	`cur = cur->next`
`Array`	`index`	`data[index]`	`++index`
`Hypercube`	`(x, y, z)` (a list or tuple of dimensions)	?? (cannot be determined)	?? (cannot be determined)

These intuitively make sense because we know how these data structures work. With an array, we can use its index to access an element, a `LinkedList` we can hold a pointer to the next node, etc.

But if we don't know how the data structure works, it will be very difficult to implement an iterator for it because we wouldn't know how to iterate through its elements.

7.2 Iterators with respect to Classes

Every class that implements an iterator must have the following **two pieces**.

1. Have two methods
   - `begin()` - Return iterator to the start item in a container (index 0)
   - `end()` - Returns iterator to one past the end item in a container
     - `end()` can come in handy a lot to check if an item exists in an iterator.
     - If I want to find string `milk` in a `std::vector<std::string>` named `groceries`, I can write `milk != groceries.end()` to see if `groceries` contains milk.

    - If `milk` is found, `end()` will return an iterator to what it found (in this case is `milk`) or will return an iterator to `end()` saying that there is no milk in the list.
    - This is a useful tool because we get an extra feature in our data structures to say that an element does/does not exist
2. Must have the base class `std::iterator`, which requires us to minimally implement
    - `operator*()` to get the data, where `*it` is the data at iterator `it`
    - `operator++()` to move/traverse through the data in a data structure
        - This standardized approach is what allows us to iterate via a data structure regardless of whether or not we iterate via a pointer (`LinkedList`) or index (`Array`)
    - `operator!=()`, which compares a constant reference to an iterator (`const Iterator& i`) to an element in the data structure and check whether they are equal or not

There are also some other nice features that can be implemented in an iterator, one of which is the arrow operator `->` which accesses members of a pointer.

- Recall that this is the shorthand for dereferencing a pointer and then calling its member, where `(*item).data` is equivalent to `item->data`.

A bidirectional iterator could also be implemented in a data structure, where it can traverse through elements **forwards** and **backwards**.

With these requirements, how would we implement `begin()` and `end()` with the example data structure mentioned earlier in this module?

7.3 Implementing `begin()` and `end()` for `Array` and `LinkedList`

Data Structure	`::begin`	`::end`
`LinkedList`	`Iterator(head_)`	`Iterator(nullptr)`
`Array`	`Iterator() { index = 0; }` or `Iterator(0)`	`Iterator(size)`

Let's explain the table above for an `Array` and `LinkedList`

- `LinkedList`
    - We would implement `begin()` by having our `Iterator` constructor take in the head of the `LinkedList`, which is `this->head_`, or simply `head_`
        - The reason why this intuitively makes sense is because the beginning of our `LinkedList` is a pointer to the first element of the `LinkedList`, which is simply `head_`
    - We would implement `end()` by returning our `Iterator` object, where it takes in a `nullptr`
        - This also intuitively makes sense because the end of a `LinkedList` occurs when a node no longer holds a pointer to its next node, when this happens its pointer is set to `nullptr` (see **Figure 7.2**)
- `Array`
    - We would implement begin by having our constructor set our `index` instance variable to `0`. This makes sense because the 0th index is always the first element in an array
        - You could also have the `Iterator` constructor take in a 0

- We would implement `end()` by returning `Iterator(size)`. This makes sense because size is the last element plus 1 in an array!
  - In **Figure 7.1**, for example, `end()` sits at index 3, one slot past the 5

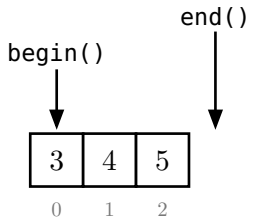


Figure 7.1: Example: an `Array` holding 3, 4, 5. `begin()` is index 0 and `end()` is index 3 (`size`), one past the last element, so it is never dereferenced

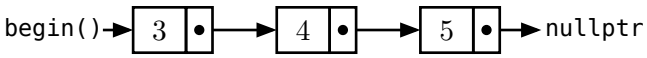


Figure 7.2: Example: a `LinkedList` holding 3, 4, 5. `begin()` wraps `head_` and `end()` wraps `nullptr`, which `++it` reaches after the last node

7.3.1 Iterator Usage Example with `std::vector`

Take a look at the following `Animal` class below

```
#include <iostream>
#include <string>
#include <vector>

struct Animal  {
    std::string name, food;
    bool big;
    Animal(std::string name = "blob", std::string food =  "you", bool big =
true) : name(name), food(food), big(big)  {  }
};

int main()  {
    Animal g("giraffe", "leaves", true), p("penguin", "fish", false), b("bear");
    std::vector<Animal> zoo;

    zoo.push_back(g);
    zoo.push_back(p);
    zoo.push_back(b);

    for(std::vector<Animal>::iterator it = zoo.begin();  it != zoo.end(); it++)  {
        std::cout << it->name << " " << it->food << std::endl;
    }
    return 0;
}
```

1. `Animal` stores 3 things, `name`, `food`, which describe the animal's name, food, and a boolean `big` that determines whether an `Animal` is big or not.
2. The `main` function creates 3 `Animal` objects and stores them in a `std::vector`. We then use an iterator to loop through the objects in the `std::vector`.

However, take a look at a snippet of the code above, specifically the part that utilizes an iterator

```
for(std::vector<Animal>::iterator it = zoo.begin();  it != zoo.end(); it++)  {
    std::cout << (*it).name << " " << (*it).food << std::endl;
}
```

one of the problems with this code is using the post-increment operator over the pre-increment operator. Luckily, `std::vector` implements this operator to work for its iterator, so this isn't an issue in our example code.

However, if we wanted to implement an iterator minimally, we would use the pre-increment operator (`++it`)

- If I were to do `it++`, then post increment would make a copy of the original iterator, move the original iterator forward, and then return the value of the original iterator that will appear there, which isn't what we want!

The rest of the code in our `for` loop prints the fields of the objects being traversed by the iterator.

- Notice that we access the members of `it` by dereferencing it and then calling a member via `(*it).name`. Recall that in C++, `(*it).name` is equivalent to `it->name`. Couldn't we use the arrow operator (`->`) instead as a shorthand of calling a member on a pointer?
  - To simply answer this question, `->` is typically implemented manually by several containers, but it is **not required**.
  - `std::vector` and many other containers do implement the `->`, so we could replace that snippet with `->`, as in the loop below, but the code snippet assumes that `->` is not implemented by the container

    ```
    for(std::vector<Animal>::iterator it = zoo.begin();  it != zoo.end(); ++it)  {
        std::cout << it->name << " " << it->food << std::endl;
    }
    ```

This code is now great, we utilize a `std::iterator` with a `std::vector`. We don't worry about how `std::vector` implemented the iterator.

We could add some other changes to this code. One is by using the `auto` keyword to define the iterator

```
auto it = zoo.begin();
```

`auto` is nice because the compiler infers the type of our iterator. This is good when we don't have to really remember what container we are working with. Whether to use `auto` is in fact debatable among the community.

Another way to write the code is by using a range based for loop to loop through constant `Animal` references, `Animal&`, in a `std::vector`

```
for(const Animal& animal : zoo) {
    std::cout << animal.name << " " << animal.food << std::endl;
}
```

In this for loop, I am introducing a constant `Animal` reference, `Animal&`, and this reference is going to take the value of every element in `zoo` one at a time using the iterator interface.

- The `animal` variable is the dereferenced iterator, so I would have to use `.` instead of `->`
- This approach completely abstracts/hides the fact I am even using an iterator. This code assumes that all the minimum iterator requirements are implemented.

And this brings us to the two approaches to use iterators

1. Using a for each loop, like the range-based `for` loop above

```
std::vector<Animal> zoo;

for(const Animal& animal : zoo) {
    std::cout << animal.name << " " << animal.food << std::endl;
}
```

2. Using a manually implemented iterator, like the loop in the `Animal` example above

```
std::vector<Animal> zoo;

for(auto it = zoo.begin(); it != zoo.end(); ++it) {
    std::cout << (*it).name << " " << (*it).food << std::endl;
}
```

Recall that the code above assumes that `->` is not implemented for the iterator

8 Trees

A **tree** comprises of two things:

1. It's an acyclic graph (has no cycles)
2. It is connected, meaning that there is a unique path from the root to every other node

A **binary tree**, T, is either

1. A tree composed of a tuple containing the **root**, **left subtree**, and **right subtree**, $T = (r, T_L, T_R)$, where r, T_L, and T_R represent the root, left subtree, and right subtree respectively
2. A **null** tree (there are no nodes in the tree)

A data structure should be able to contain nothing because that is how we tend to start evolving and adding elements/nodes into it. In the context of a tree, we would like to be able to start it off with nothing. Therefore, it makes sense to define a binary tree as a **null** tree, where it has no vertices

A tree's **height**, height(T), is the longest path from the root to a leaf. However, this changes a little bit for a binary tree

In a binary tree, it is equal to maximum subtree of the left and right plus 1, or simply

$$\text{height}(T) = \max(\text{height}(T_L), \text{height}(T_R)) + 1$$

This makes sense because if $T_L = 5$ and $T_R = 3$, then the whole tree can go at most 5 levels deep through the left. The maximum of the two tells you the **deepest path** from the current node to a leaf. We add 1 because we add the **edge of the root node**. Let's try this on the tree in **Figure 8.1**, where the left subtree rooted at B is the taller one, so it decides the height of the whole tree.

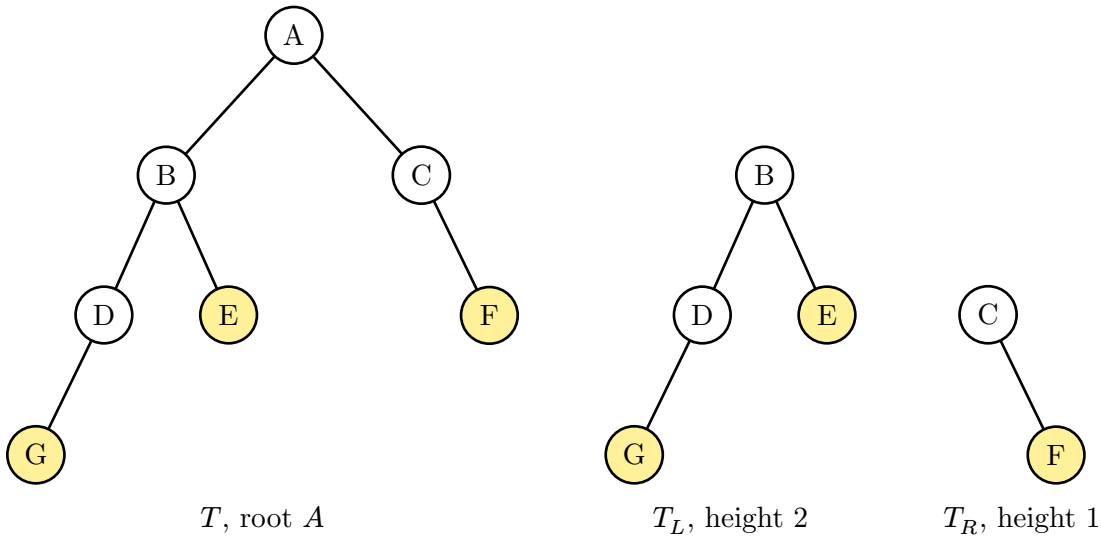


Figure 8.1: Example: the leaves (no children) are highlighted. The longest path from A is A, B, D, G, so $\text{height}(T) = \max(2, 1) + 1 = 3$

The height of a null tree is simply -1, because all the math and recursion will work out. Most of the time, the base case in tree recursion occurs when there are no nodes to choose from

A tree is **full** if and only if

1. The tree is empty (has no nodes, or simply $F = \varnothing$)
2. $F = (r, F_L, F_R)$ where either $F_L = F_R = \varnothing$ or $F_L \neq \varnothing$ and $F_R \neq \varnothing$
   - in code, this simply means checking whether the two nodes both have null pointers or neither have null pointers

A **perfect** tree, P, is defined in terms of a tree's height (simply because either everything exists at the tree's height or nothing exists).

If P_h represents a perfect tree with height h, then

1. $P_{-1} = \varnothing$
2. $P_h = (r, P_{h-1}, P_{h-1})$, meaning that the height of the left and right subtrees are **equal**

However, if I wanted to build a perfect tree with only 4 nodes, that wouldn't be possible since we have an odd number of nodes remaining after expending 1 for the root node

A **complete** tree is a binary tree where all levels are completely filled except for the last level, which is filled from left to right

More specially, complete trees

1. A perfect tree for every level except the last, where the last level is "pushed to the left"
2. For all levels k in $[0, h-1]$, k has 2^k nodes. For level h, all nodes are "pushed to the left"

A complete tree of height h, C_h

1. $C_{-1} = \{\}$
2. C_h (where $h \geq 0$) $= \{r, T_L, T_R\}$ and either:

- T_L is C_{h-1} and T_R is P_{h-2} - given a complete tree with a height h, its left subtree must be complete with height $h-1$ and its right subtree must be perfect with height $h-2$
- T_L is P_{h-1} and T_R is C_{h-1} - given a complete tree with a height h, its left subtree must be perfect with height $h-1$ and its right subtree must be complete with height $h-1$

This just mathematically explains the definition of a complete tree, which is simply a perfect tree where every level is filled except the last, which is filled with nodes towards its left subtree

If you compare the three trees in **Figure 8.2**, the complete tree is just the perfect tree with its rightmost leaf (7) missing.

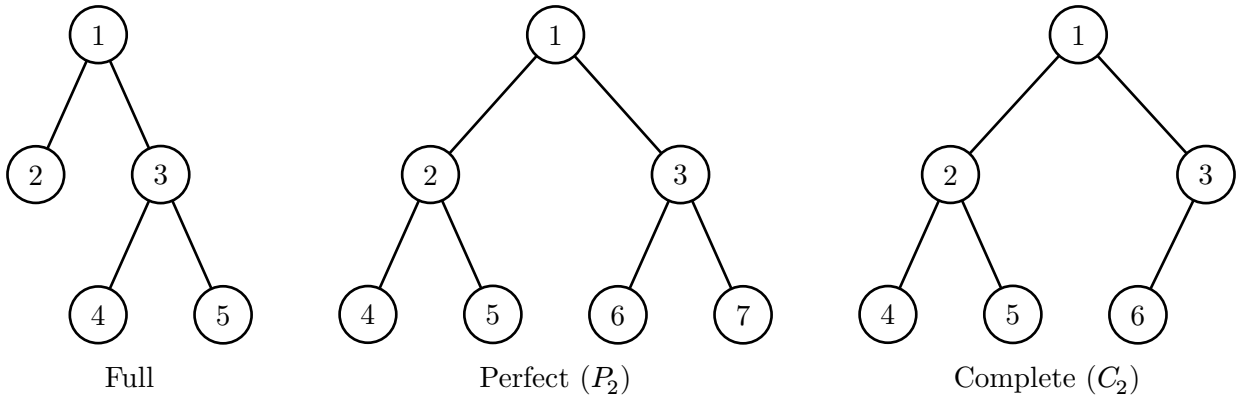


Figure 8.2: A full, a perfect, and a complete binary tree

8.1 Tree ADT

Tree ADT involves 3 primary functionalities

1. `insert`, inserts an element to the tree - into a location in structure
2. `remove`, removes an element from a tree - from a location in structure
3. `access`, access elements from a tree - from a location in structure

A list has a fixed structure and order, but a tree has varied structure depending on how it is used. The tree's structure may not matter because many algorithms may still work the same regardless of the tree's shape, as long as the structure still preserves the necessary relationships.

Therefore, let's think about how we code a `BinaryTree`. An example header file that defines a `BinaryTree` is shown below

```
#pragma once
template <class T>
class BinaryTree {
public:

private:

};
```

The first thing we need is a way to store the tree's root. In a `LinkedList`, the way we did it was by having a `ListNode` class to store each node of a list. Trees are similar to lists as well. They also have data stored in nodes, but are in hierarchical order rather than sequential

Therefore, we will also want to use a `struct` here to store each individual `TreeNode`, which will have 3 fields

1. `data` - the data the `TreeNode` holds
2. `left` - a **pointer** to another `TreeNode` that represents the left child (or `nullptr` if no left child)
3. `right` - a **pointer** to another `TreeNode` that represents the right child (or `nullptr` if no right child)

The code for a `TreeNode` is reflected below

```
struct TreeNode {
    // the data each tree node holds
    T data;

    // a pointer to the left child `TreeNode` (or `nullptr` if non-existent)
    TreeNode* left;

    // a pointer to the right child `TreeNode` (or `nullptr` if non-existent)
    TreeNode* right;
};
```

With this, it makes sense for our root to also be a **pointer** to a `TreeNode` as well. By making the left/right children nodes and root nodes pointers, it allows for efficient memory access and avoids copying data unnecessarily, allowing the tree to grow dynamically in memory without using too much overhead. The `BinaryTree` class is reflected below

```
template <class T>
class BinaryTree {
private:
    struct TreeNode {
        // the data each tree node holds
        T data;

        // a pointer to the left child `TreeNode` (or `nullptr` if non-existent)
        TreeNode* left;

        // a pointer to the right child `TreeNode` (or `nullptr` if non-existent)
        TreeNode* right;
    };

    // a pointer to the root node of a tree
    TreeNode* root;
};
```

In memory, each tree node is created on the heap, and the `root` pointer (on the stack) points to the first node. Each node's `left` and `right` fields store pointers to other nodes on the heap, linking them together without copying any data. This allows the tree to grow dynamically, using memory only when needed.

When we create a tree in memory, there is often wasted space because child pointers in many nodes can point to `null`, meaning they don't reference any actual node. Knowing **how many nodes store these null pointers** helps us understand the **memory overhead** and measure how **efficient** our tree data structure is.

In contrast, with a `LinkedList`, we only had to worry about one pointer per node possibly being null (the `next` pointer), so the overhead was much simpler and easier to quantify (it was always 1)

Therefore, it makes sense to **prove how many null pointers exist in a tree** to analyze and optimize its memory usage and better understand the structure's efficiency.

There will always be $n + 1$ `NULL` pointers for n nodes in our binary tree. But why?

Prove that if there are n data items in our representation of a binary tree, then there are $n + 1$ `NULL` pointers (Hint: use induction on n)

1. Let NULLS(n) be defined as the number of null pointers in a tree of size n
2. **Base Case(s)**
   - $n = 0 : 1$ null pointer; $(n + 1 = 0 + 1 = 1)$ (the tree is empty so the `root` pointer is null)
   - $n = 1 : 2$ null pointers; $(n + 1 = 1 + 1 = 2)$ (there is only the root node and both `left` and `right` child pointers are null)
   - $n = 2 : 3$ null pointers; $(n + 1 = 2 + 1 = 3)$ (One node has two children - one child node and one null child - and the child node has two null pointers, totaling 3)
3. **Induction Hypothesis**: Suppose NULLS(n) $= n + 1$ for all $0 \leq n < k$
4. **Inductive Step**: Prove for $n = k + 1$ nodes
   - Consider any binary tree with $k + 1$ nodes
   - Remove a leaf node (or any node that can be removed while keeping a valid binary tree structure), resulting in a tree with k nodes
   - By the inductive hypothesis, this smaller tree has $k + 1$ null pointers
   - When adding back the removed node:
     - One null pointer (where the node was attached) is replaced by a new node, so we lose 1 null pointer
     - The newly added node has two child pointers, both null, so we add two null pointers
   - Therefore, the **total null pointers** $= (k + 1) - 1 + 2 = k + 2$
   - Which equals: $(k + 1) + 1$
   - Therefore, by induction this statement holds true for $n \geq 0$

9 Tree Traversals

Previously, we discussed a new data structure: trees, where we discussed that there is a $O(n)$, linear space, overhead cost because n-space is required to store the data.

Here, we will continue discussing Trees and the Tree ADT. Previously, we failed to discuss how the implementation of `insert`, `remove`, and `access` would work. This is because the way they work is different for every tree.

Recall that previously we discussed that every data structure should be able to have its own way of getting everything it contains, which was done through an iterator. This means that if we want to achieve this functionality, we would need to implement an iterator interface. But **how**?

We need to know how to find everything in our tree. Otherwise, how would we know that we found everything. The simple answer to these questions is by implementing **tree traversals**.

Tree traversals are algorithms that will follow the structure of a tree in some fixed order that **visits** every node. They access **all the nodes** in a tree. So every data point is stored in some sort, and we are going to visit all of the nodes.

- By visit, we typically mean whatever the algorithm wants to do with the data, which for demonstration purposes, we just simplify to printing the data because that is an easy way to test what is going on
- So when a node is visited, it will then be printed and then at the end check whether we went through every node in the tree

While it makes sense that the goal of a tree traversal is to visit every node, but **how**? Should a tree visit a node **before** I traverse into the subtree, **in between** the subtree, or **after** the subtree?

The simple answer is that **it's up to you**. It depends on **when** you choose to visit a node in relation to its children. These three choices correspond to the three well-known tree traversal techniques:

- **Pre-order traversal** — visit the node **before** its subtrees
- **In-order traversal** — visit the node **between** its subtrees
- **Post-order traversal** — visit the node **after** its subtrees

By understanding how these traversals work, it makes coding them much more simple and straightforward.

1. Start from the root and do one of three of the following options (the order in which you do the tree depends on the traversal type)
   - Visit
   - Descend into the left subtree
   - Descend into the right subtree

2. Repeat step 1 for the **next** subtree (depending on its left or right) until there is no subtree available

The order in which you do the three options above depends on the traversal type

- **Preorder** - visit, descend left, descend right
- **Inorder** - descend left, visit, descend right
- **Postorder** - descend left, descend right, visit

We'll run every traversal in this section on the tree in **Figure 9.1**.

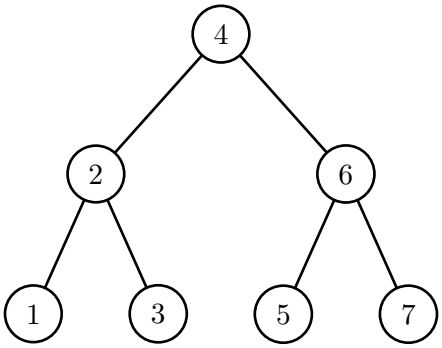


Figure 9.1: Example tree used for every traversal below

Traversals are **recursive** by nature because trees are recursive data structures. When we visit a node, we then descend into its left and right subtrees, and for each of those, we repeat the same process: visiting the node and then exploring its subtrees. This pattern continues until we reach nodes with no children, making recursion the best way to mimic this behavior.

9.0.1 Pre Order traversal

Preorder traversal visits a tree node, and then descends into the left and right subtrees. It's recursive since trees are recursive data structures in nature.

This makes coding pre order traversal very simple. This function will take in the current `TreeNode` being processed (`curr`): `void preOrderTraversal(TreeNode* curr);`

The code for `preOrderTraversal` is shown below

```
template <typename T>
void BinaryTree<T>::preOrderTraversal(TreeNode* curr) {
    // If the current node is a null pointer (`nullptr`), halt the recursion
    if(curr == nullptr) { return; }

    // Visit the current `TreeNode`
    std::cout << curr->data << " ";

    // Repeat the process for the left subtree
    preOrderTraversal(curr->left);
```

```
    // Repeat the process for the right subtree
    preOrderTraversal(curr->right);
}
```

Our **base case** occurs when the current node pointer (`curr`) is `nullptr`. This means we've reached a point in the tree where no further nodes exist (e.g. a leaf node's child) and there's nothing to process or recurse on, so we simply return.

If the node is not `nullptr`, we proceed with the pre-order traversal steps:

1. **Visit the current node** — we get its data (e.g., print it).
2. **Traverse the left subtree** — recursively call `preOrderTraversal` on `curr->left`.
3. **Traverse the right subtree** — recursively call `preOrderTraversal` on `curr->right`.

Each of these recursive calls follows the same pattern: check if the node is `nullptr`, visit it, and recurse on its children. This process naturally continues until all nodes have been visited, and both left and right child pointers are `nullptr`, causing the recursion to unwind.

Running pre-order on the tree in **Figure 9.1** visits 4, 2, 1, 3, 6, 5, 7.

9.0.2 In Order traversal

Inorder traversal descends into the left subtree, visits a tree node, and then descends into the right subtree

This also makes coding in order traversal very simple. This function will take in the current `TreeNode` being processed (`curr`): `void inOrderTraversal(TreeNode* curr)`

The code for `inOrderTraversal` is shown below

```
template <typename T>
void BinaryTree<T>::inOrderTraversal(TreeNode* curr) {
    // If the current node is a null pointer (`nullptr`), halt the recursion
    if(curr == nullptr) { return; }

    // Repeat the process for the left subtree
    inOrderTraversal(curr->left);

    // Visit the current `TreeNode`
    std::cout << curr->data << " ";

    // Repeat the process for the right subtree
    inOrderTraversal(curr->right);
}
```

Our **base case** is the same as pre order, when the current node pointer (`curr`) is `nullptr`. No further nodes exist anymore (e.g. a leaf node's child) and there's nothing to process or recurse on, so we simply return.

If the node is not `nullptr`, we proceed with the in-order traversal steps:

1. **Traverse the left subtree** — recursively call `inOrderTraversal` on `curr->left`.
2. **Visit the current node** — we get its data (e.g., print it).
3. **Traverse the right subtree** — recursively call `inOrderTraversal` on `curr->right`.

If we apply in-order traversal to the tree in **Figure 9.1**, we get 1, 2, 3, 4, 5, 6, 7 (sorted, because the example is a BST).

9.0.3 Post Order Traversal

Post Order traversal descends into the left subtree, descends into the right subtree, and then visits the current tree node.

This also makes coding post order traversal very simple, since we know its process. This function will take in the current `TreeNode` being processed (`curr`): `void postOrderTraversal(TreeNode* curr)`

The code for `postOrderTraversal` is shown below

```
template <typename T>
void BinaryTree<T>::postOrderTraversal(TreeNode* curr) {
    // If the current node is a null pointer (`nullptr`), halt the recursion
    if(curr == nullptr) { return; }

    // Repeat the process for the left subtree
    postOrderTraversal(curr->left);

    // Repeat the process for the right subtree
    postOrderTraversal(curr->right);

    // Visit the current `TreeNode`
    std::cout << curr->data << " ";
}
```

The base case is equivalent to pre and in order traversals.

If the node is not `nullptr`, we proceed with the post-order traversal steps:

1. **Traverse the left subtree** — recursively call `postOrderTraversal` on `curr->left`.
2. **Traverse the right subtree** — recursively call `postOrderTraversal` on `curr->right`.
3. **Visit the current node** — we get its data (e.g., print it).

Back on **Figure 9.1**, post-order visits 1, 3, 2, 5, 7, 6, 4, so the root 4 comes last.

The ideas behind these traversals are very similar. It's just that the order a `TreeNode` is visited and when to descend into subtrees varies based on the type of traversal

A good way to remember the traversals are the three mnemonics

- **NLR** - pre order traversal (node, left, right)
- **LNR** - in order traversal (left, node, right)

- **LRN** - post order traversal (left, right, node)

More specifically, these tree traversals fall under a broader category known as **depth first search (DFS)**. In DFS, we start at the root and recursively descend as far down each branch as possible before backtracking, where nodes are then visited in a way that explores one entire path before moving to another.

9.1 A different type of traversal - Level Order Traversal

There is another type of tree traversal that behaves differently than the three DFS traversals and it is known as a **Level Order Traversal**.

In this traversal, we visit each node **layer by layer**, or level by level, starting from the root node. This will help find nodes at each level.

This level order traversal method is more commonly known as **breadth first search** or simply **BFS**, which involves visiting the nodes of a tree level by level

In the three DFS traversals we used recursion on the **tree** itself to implement them. In BFS, we can use recursion, but it wouldn't make sense to do it on the **tree** itself since we don't need to recursively descend into each subtree.

Instead, we would need to use another data structure that the recursion would work with. Since BFS visits the nodes in horizontal, level order, a queue would make sense here. This is because queues are **First-In First-Out** and preserve the level ordering rule that BFS needs.

However, using recursion with BFS is not intuitive and not preferred, since it requires visiting all of a node's siblings before its children, which goes against the natural depth-first behavior of recursion.

We can still implement it iteratively with the help of a queue.

1. Create a queue of `TreeNode` pointers (`TreeNode*`)
2. Enqueue `root` to the queue
3. While the queue isn't empty, dequeue the front node and check if its pointer is `null`. If not, then print its data, and then enqueue its left and right child

Level-order on **Figure 9.1** goes one level at a time: 4, then 2, 6, then 1, 3, 5, 7.

The code for this process, which uses a queue, is shown below

```
template <typename T>
void BinaryTree<T>::levelOrderTraversal() {
    // Create a queue to store `TreeNode` pointers at every level
    std::queue<TreeNode*> q;

    // Add the root `TreeNode`, `root` to the queue
    q.push(root);

    while(!q.empty()) {
        // Get the `TreeNode` at the front of the queue and remove it
```

```
        TreeNode* n = q.front();
        q.pop();

        // Check if the front `TreeNode` is not null
        if(n != nullptr) {
            // visit the node
            std::cout << n->data << " ";

            // add the node's left and right children to the queue
            q.push(n->left);
            q.push(n->right);
        }
    }
}
```

10 Tree Searching and Dictionaries

A **traversal** is an algorithm that goes through every single node in a tree and visits it ($O(n)$ time)

A **search** is an algorithm that finds a node in a tree of value v if it exists (worst case is when $v \notin T$, where T represents any tree; this involves running the full traversal, so it is in $O(n)$ time)

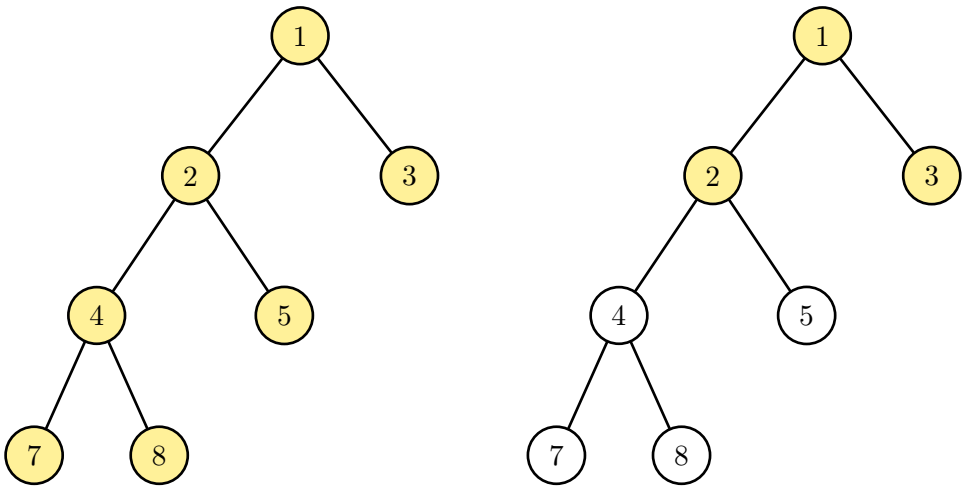
Previously, we discussed two types of search **Breadth First** and **Depth First** (**BFS** and **DFS**)

BFS is the same as level order traversal, where its overhead (or space complexity) is the maximum number of nodes in a queue (can also be seen as the width of a tree)

DFS involves any of the three traversals that descend into the tree's children: pre order traversal, in order traversal, and post order traversal. Its overhead is the maximum depth in the tree (or simply height)

BFS is more **reliable** than DFS. This is because it will eventually find a solution to a valid search if one exists, even if the tree is infinite in depth. Since BFS explores nodes level by level, this guarantees that all nodes at depth d are visited before any node at depth $d + 1$. So if the solution lies at a shallow depth, BFS will find it eventually.

In contrast, DFS can get **"stuck"** going down an infinite branch of a tree, especially if the solution lies on a different, shallower path. Therefore, DFS might never return a proper solution, even if it does exist. You can already see this cost on a finite tree in **Figure 10.1**: when searching for 3, DFS wanders through the whole left branch while BFS finds it after one level.



DFS (pre-order): 1, 2, 4, 7, 8, 5, 3 BFS: 1, 2, 3

Figure 10.1: Example: searching for 3, which sits at depth 1. DFS exhausts the whole left branch first; BFS reaches 3 after visiting only one level

BFS will always find a solution if one exists, even in infinite-depth trees, but it may require an infinite amount of memory. This is because BFS stores all nodes at a given depth (one full level of the tree) in a queue before moving on to the next level. If the tree is very wide, this means the queue can grow extremely large, consuming a lot of memory.

In contrast, DFS typically uses less memory, since it only needs to store the current path from the root to a leaf, which depends on the depth of the tree, not its width.

Therefore, in any practical application, memory is going to be the forefront problem. Since DFS uses less memory, that will often be the best approach. But, if there are infinite trees, DFS wouldn't work. There is a solution that solves this problem and it is known as **iterative deepening**

Iterative Deepening runs DFS with a **maximum depth limit** of k, meaning it will not explore any nodes beyond depth k (starting at $k = 0$). When DFS reaches that depth, it stops descending and backtracks up the tree. This approach is useful because the maximum memory used when doing DFS is proportional to k (simply the overhead cost - $O(k)$). If no solution is found at depth k, the algorithm restarts DFS with a new depth limit $k + 1$, and this process repeats until a solution is found.

This effectively simulates BFS for DFS and will always find the solution because it explores each node at every level by using bounded memory (as deep as depth k with overhead cost of $O(k)$).

These are the two basic searching strategies, but they aren't that effective, since they can be slow in both time and memory. But what if there was a data structure that put searching at its forefront, which is essentially what a **map** or **dictionary** does

10.1 Dictionaries

Dictionaries store **key-value** pairs, just like a regular dictionary, which stores words to definitions. If I had a word, I can use a dictionary to look up its definition! This could be done most optimally using binary search, where I start from the middle and then slowly go left or right based on whether the word I'm looking for comes before or after the middle word alphabetically.

This is just one example of a dictionary though, we can use it to look up students using their unique ID numbers (UIUC UIN) or by looking at a Flight number (**key**) for its departure time/ arrival time (**value**).

In fact, an array really is just a dictionary, because we use an index (**key**) to lookup the value of an element that is present in the array (**value**)

10.1.1 Dictionary Interface (ADT)

- `insert(key, value)` - insert a key and value into the dictionary
- `remove(key)` - remove a key value mapping from the dictionary via passing in the key
- `std::optional<value> find(key)` - return the value associated with the key (C++ has an `std::optional` type that you could use to return nothing if no value is found for the key!)

And then there's the obvious things like a constructor, destructor, and an iterator, but at the end of the day, this is what we want with a dictionary.

And with this, we solve our problems with searching in Computer Science. A lot of other data structures, like Hash Tables and Tries, will rely on dictionaries to provide fast and efficient key-based access, which is essential for everything from caching to autocomplete and spell checking

The code below is a sample header file for our dictionary, backed by a `std::unordered_map`

```
// Dictionary.h

#pragma once
#include <unordered_map>

template <class K, class V>
class Dictionary {
public:
    using Iterator = typename std::unordered_map<K, V>::iterator;

    void insert(const K key, const V value);
    V remove(const K& key);
    Iterator find(const K& key);
    Iterator begin();
    Iterator end();

private:
    // the underlying map that stores every key-value pair
    std::unordered_map<K, V> data;
};
```

Our custom dictionary implements an iterator by leveraging C++'s standard library iterator from the underlying data structure of `std::unordered_map`. This allows us to avoid writing a custom iterator from scratch, while still providing full iterator functionality like `begin()`, `end()`, and `find()`.

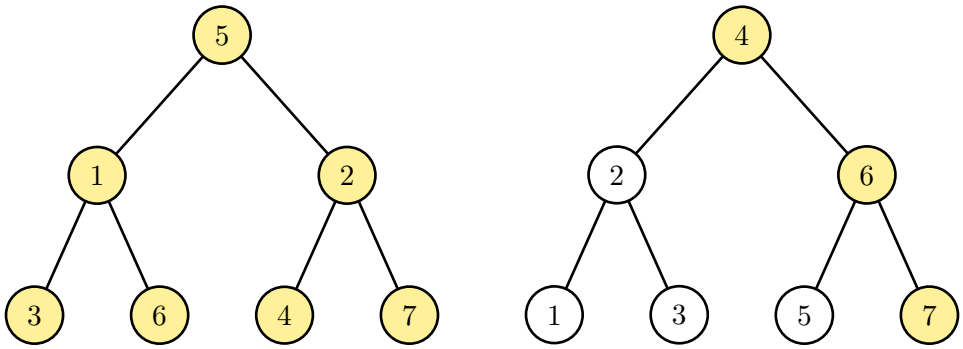
10.2 Using Binary Trees as a Search Structure

In the context of a tree, these three functionalities of a dictionary, `insert`, `remove`, and `find`, are reflected below

Dictionary Operation	Runtime	Explanation
`insert`	$O(1)$	Similar to a LinkedList; put a new pointer to head and point the tree's head to this new node, which is constant time
`remove`	$O(n)$	Need to first find the item by scanning the tree, and then shift elements to fill the gap
`find`	$O(n)$	Searches nodes one by one, traversing the tree until it finds the key or uses all nodes

This sets up the foundation for many other types of trees including, BSTs, BBSTs, and AVL trees, which will be covered later on

To preview why ordering the keys matters for `find`, compare the two trees in **Figure 10.2**: the plain binary tree visits all 7 nodes to find 7, while the BST only visits 3.



Plain binary tree: 7 nodes visited BST: 3 nodes visited

Figure 10.2: Example: finding 7 among the same keys. A plain binary tree gives no hint where 7 is, so the search scans every node; a BST orders its keys so the search follows a single path 4, 6, 7

11 BST and BST Implementation

Recall that all trees have **structure**. Fundamentally, a tree gives us **positions** that when we put nodes in a tree, we put them in a position. Previously, the trees we discussed were in a position that had no meaning whatsoever. Is there a way for us to put nodes in a specific position that has meaning?

For this to take place, we need to discuss a different type of tree: the **Binary Search Tree**, or BST. The BST has a property, in which a node's left subtree contains nodes with keys less than the node's key and a node's right subtree contains nodes with keys greater than the node's key.

When it comes to equality, we typically don't define it for a BST. We assume that there will not be any duplicates (every node is unique in value) in a BST, and therefore we don't place it anywhere. It still can be extended in a BST definition, but it ends up with odd decisions and semantics, so it's just better to assume that all nodes in a BST are **unique**.

Mathematically speaking, for tree T to be a BST:

- $T = \varnothing$
- $T = (r, T_L, T_R)$, where $\forall_x \in T_L$, $x < r$ and $\forall_x \in T_R$, $x > r$

x is a key in T_L, where $x < r$ and a key in T_R, where $x > r$

In **Figure 11.1**, the tree on the left is a valid BST, but the one on the right breaks this property because of the highlighted 12.

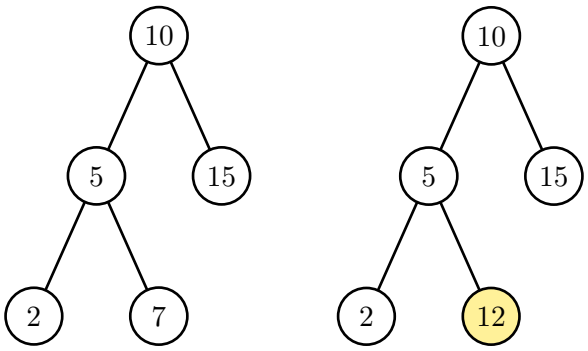


Figure 11.1: Example: on the right, 12 is a valid right child of 5, but it lies in the left subtree of 10 while $12 > 10$. The property must hold for the whole subtree, not just the parent

This creates structure in the tree, where if I wanted to find a node like 21, I would start searching from the root node, and then check if 21 is greater than or less than the root node and then descend to the right or left subtree respectively and repeat the process until I find the node of 21.

In this process, we rule out half one of the subtrees. Notice in **Figure 11.2** how the search for 21 only touches the highlighted path 15, 25, 21.

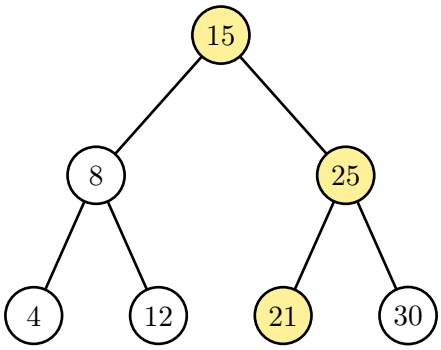


Figure 11.2: Example: finding 21 visits 15, 25, 21 ($21 > 15$, go right; $21 < 25$, go left). The left subtree of 15 is never touched

This is the unified process of searching that we have in a BST, which gives the tree a proper structure, giving each node's position a meaning.

Simply speaking, a BST is a binary tree that is organized in a way where all the nodes in its left subtree are smaller than it and all of the nodes in the right subtree are bigger than it.

Finding a node in a BST is $O(h)$, where h is the height of the tree. This is because the worst case occurs when we have to traverse all the way to the root of a leaf, which takes h steps. This means that $O(h) \equiv O(n)$ because the height h can be as large as n in the worst case.

Below is a good concrete implementation of a BST

```
// BST.h

#pragma once
#include <unordered_map>

template <class K, class V>
class BST {
public:
    using TreeIterator = typename std::unordered_map<K, V>::iterator;

    BST() : root(nullptr) { }
    void insert(const K key, const V value);
    void remove(const K& key);
    TreeIterator find(const K& key);
    TreeIterator begin();
    TreeIterator end();

private:
    struct TreeNode {
        TreeNode* left;
```

```
        TreeNode* right;
        K key;
        V value;

        TreeNode(const K& key, const V& value) : left(nullptr), right(nullptr),
key(key), value(value) { }
    };

    TreeNode* root;

    // a map that lets us use iterators for fast access without needing to
traverse the tree
    std::unordered_map<K, V> node_map;

    // internal helpers
    TreeNode*& _find(TreeNode*& root, const K& key);
    void _insert(TreeNode*& root, const K& key, const V& value);
    TreeNode*& _iop(TreeNode*& root);
    void _swap(TreeNode* a, TreeNode* b);
    void _remove(TreeNode*& node, const K& key);
};
```

This looks great, it shows the exact functionalities we need for our BST

Notice how we added an `std::unordered_map`, `node_map`. This helps us use the unordered map's iterator for fast access of our tree nodes, without having to constantly traverse the tree. It's a fast access cache alongside our BST

- `find` - Will be used to find a node in a BST. Returns a custom implemented `TreeIterator` that behaves like the STL iterator of a template library. This is because we want to return the iterator's `.end()` behavior when not found.
- `insert` - Will be used to insert a node into a BST
- `remove` - Will be used to remove a node from a BST

Let's first start with the simplest implementation, `find`

11.1 `find`

To implement find in a BST, the process is pretty straightforward

1. Visit the root node; if it's greater than the root node, descend into the right subtree and visit its node. otherwise, descend into the left subtree and visit its node.
2. Repeat step one for each node as you descend until you find the node and return it
3. Otherwise, return the iterator's `.end()` behavior if the node was not found in the BST

First we will begin with the internal find (`_find`). The external find function will just wrap the internal find with `root`, and then dealing with the case when the pointer is null and returning an iterator to the `end` rather than an iterator to that node. The signature of `_find` is shown below.

```
template <class K, class V>
typename BST<K, V>::TreeNode*& BST<K, V>::_find(TreeNode*& root, const K& key) {

}
```

This internal `_find` is a lot like `find` from a `LinkedList`. It returns a reference to a pointer to a `TreeNode`. This returns the pointer that points to the node that has the key that exists somewhere in my tree data structure. Or if not, the last pointer I found while trying to find it. And then we take in the `root`, which is our recursion and the `key`

Since this is a recursive problem we first need to figure out our base case, which is just checking if our root's key is equal to the key we want to find. If it is, we should just return the root, since it is a pointer as a reference to the root node!

However, this code will crash if `root` is null, so we need to add another base case to check if `root` is null.

Then, we need to check if the root's key is greater than the given key, and if so, we call `_find` and return the root's left node and the key

Otherwise, we can just call `_find` and then return the root's right node and the key

This full code for internal find, `_find`, is shown below

```
template <typename K, typename V>
typename BST<K, V>::TreeNode*& BST<K, V>::_find(TreeNode*& root, const K& key) {
    // if the root's pointer is null, return the root or
    // if the root's key equals the given key, return the root
    if(root == nullptr || root->key == key) { return root; }

    // if the root's key is greater than the given key, call find on the left
subtree
    if(root->key > key) {
        return _find(root->left, key);
    }

    // if the root's key is less than the given key, call find on the right
subtree
    return _find(root->right, key);
}
```

To implement the external find, `find`, we can just simply return `node_map.find(key)` as an iterator, since `node_map` always stores the same key-value pairs as the BST and supports fast lookup:

```
template <typename K, typename V>
typename BST<K, V>::TreeIterator BST<K, V>::find(const K& key) {
    // use `node_map` to quickly find the value without traversing the tree
```

```
    return node_map.find(key);
}
```

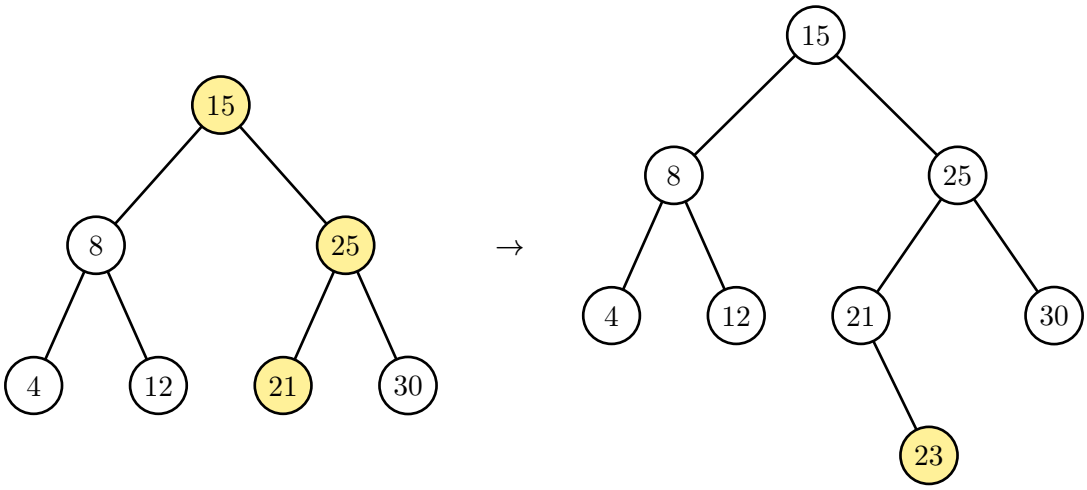
However, there are some questions to ask, especially since this was a recursive approach.

1. What if the tree is really tall? Because in a list, we don't want to write things recursively because the nodes will expend a lot of memory on the call stack.
   - The only valid solution would be to implement this iteratively, but I wouldn't want to do this because they work in a naturally recursive format. For now we can safely ignore this because there isn't much of a use case, but this is still a problem.

11.2 `insert`

We will now implement an internal insert function called `_insert`. The process is simple: to insert a node into a Binary Search Tree (BST), we search for the position where the new node should go, and then assign a new `TreeNode` with the given key and value at that location.

This works because, when we search for the correct position in the BST, we eventually reach a `null` pointer, since the key does not already exist in the tree. The `_find` function returns a **reference to that pointer**, giving us access to the exact spot where the new node should be inserted. With this reference, we can directly assign a new `TreeNode` to that location, effectively inserting the node into the tree. For example, inserting 23 in **Figure 11.3** follows the same path as a search (15, 25, 21), and 23 lands as the right child of 21.



Search visits 15, 25, 21 — 23 becomes the right child of 21

Figure 11.3: Example: inserting 23. `_find` stops at the null `right` pointer of 21, and the new node is assigned there

The code for internal insert, `_insert` is shown below

```
template <class K, class V>
void BST<K, V>::_insert(TreeNode*& root, const K& key, const V& value) {
    // Create a new `TreeNode` at the null pointer location returned by `_find`
    _find(root, key) = new TreeNode(key, value);
}
```

This code looks a little bit weird, but it makes sense

1. Recall that `_find(root, key)` returns a **reference to a pointer** (`TreeNode*&`), specifically the pointer in the tree where the node with `key` should be located
2. When the key doesn't exist, that pointer is **null**
3. Because `_find` returns a **reference to that pointer**, you can assign to it **directly**

This code can be done more clearly to understand what's going on, by storing the result of `_find` in an explicit `loc` reference

```
template <class K, class V>
void BST<K, V>::_insert(TreeNode*& root, const K& key, const V& value) {
    // grab the reference/location to the pointer where `key` should be inserted
    TreeNode*& loc = _find(root, key);

    // assign `loc` to point to a new `TreeNode` with the given `key` and `value`
    loc = new TreeNode(key, value);
}
```

There is a bug if you don't add the ampersand to the `TreeNode*` type when creating `loc`, since that then creates a copy pointer which changing it only changes the copy

It's a preference as to which method is better for writing `_insert`; the first one is slightly more clear as it expresses the entire insertion action in a single, direct statement without introducing intermediate variables or extra steps.

To then code the external `insert`, we just have to simply call the internal insert, `_insert` and then add the key to our `node_map`, which is an `std::unordered_map` that contains our `TreeNode`s

```
template <typename K, typename V>
void BST<K, V>::insert(const K key, const V value) {
    _insert(root, key, value);

    // also insert the key-value pair into the hashmap for fast lookup and
iterator support
    node_map[key] = value;
}
```

11.3 `remove`

The final step is to implement `remove` for a BST. This seems simple since I just need to find the location of the `TreeNode` that I want to remove and then set its pointer to null.

However, this actually will only work if the node is a **leaf node**. There are two other cases we have to consider when removing a node, which rely on the node having at least one child

1. **Removing a leaf node** - If the node is a leaf node (has no children), its parent's pointer just becomes null (see **Figure 11.4**)
2. **Removing a node with one child (left or right)** - If the node has exactly either one left or right child, you remove the node by connecting its parent directly to that child. This bypasses the node being removed, preserving the BST structure without losing any nodes. In **Figure 11.5**, for example, removing 8 lets 15 point straight at 4.
3. **Removing a node with two children** - If the node has two children, meaning it has both a left and right child, we need to instead find a replacement node to take the removed node's place in order to maintain the property of a BST. This is either:
   - The **inorder predecessor** (IOP) - the largest node in the left subtree
   - The **inorder successor** (IOS) - the smallest node in the right subtree
   - Swap the target node's key and value with the replacement node's, then recursively remove the replacement node, which then falls into the simpler cases of leaf or single child. You can see this in **Figure 11.6**, where the IOP of 15 is 12 (highlighted), so the two swap and 15 is then removed with the one child case

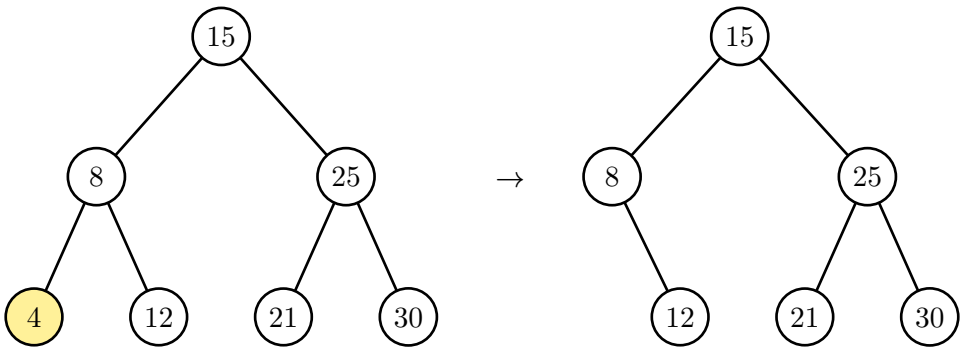


Figure 11.4: Example: removing the leaf 4. Its parent's `left` pointer simply becomes null

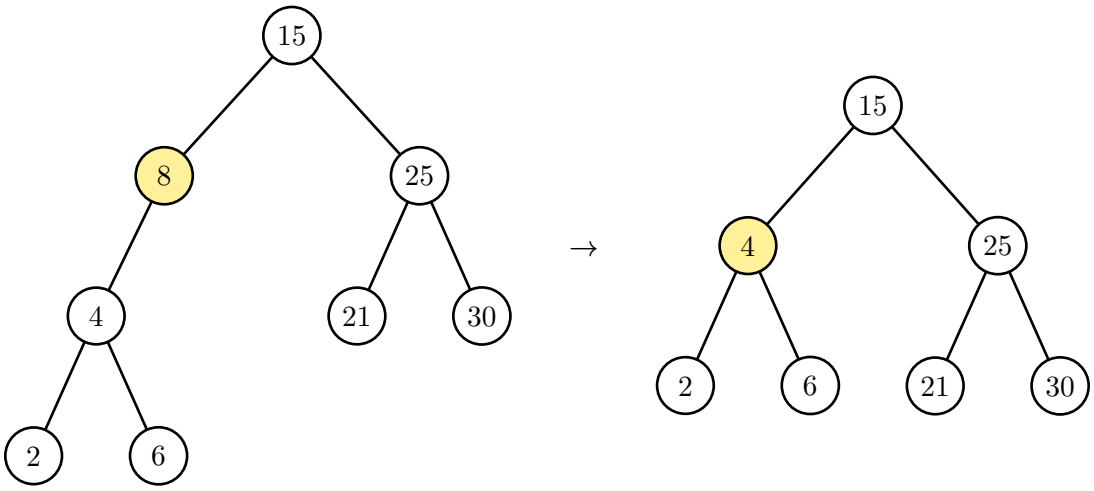


Figure 11.5: Example: removing 8, which has only a left child. 15 now points directly at 4, and the whole subtree under 4 moves up with it

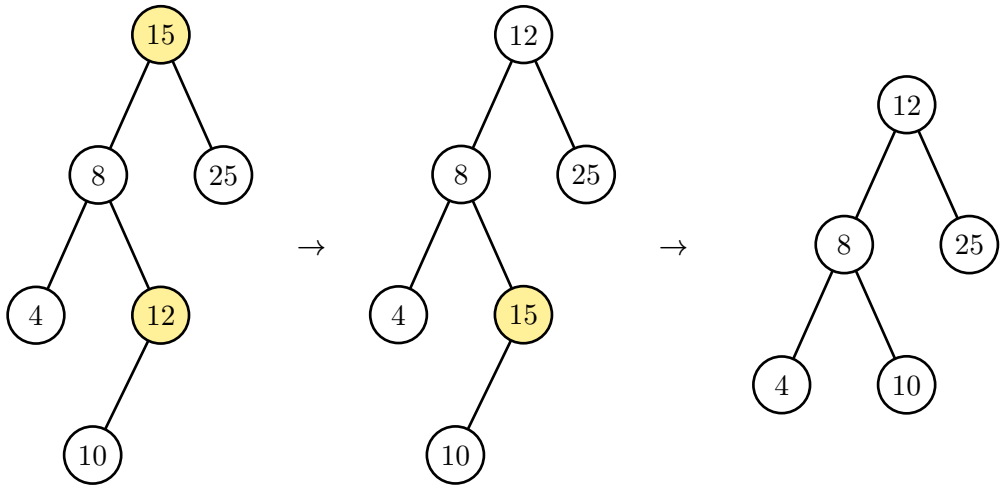


Figure 11.6: Example: removing 15, which has two children. Its IOP is 12, the rightmost node of the left subtree (8, then right to 12). After the swap, 15 has only a left child, so it is removed with the one child case

Finding either IOP and IOS (it doesn't matter which one) is a very effective approach when removing a node with both a left and right child because it always runs in $O(h)$ time, where h is the height of a tree

This is because when a left or right node follows a single path down a tree, the worst case occurs when the node travels through the entire path of h

First, we need a helper function that helps find either the IOP or IOS (will do IOP in this example). Recall that the IOP finds the largest node in the left subtree.

Therefore, to implement this, we need to find the rightmost value in the left subtree, since this will always be the largest value according to the BST property. To find the largest value, we just need to iterate through each node's right child until a node's right child no longer exists (null pointer). The code for finding the IOP is reflected below

```
template <typename K, typename V>
typename BST<K, V>::TreeNode*& BST<K, V>::_iop(TreeNode*& root) {
    // grab a pointer to the left subtree pointer of the root
    // (a pointer to the pointer, so moving `curr` never overwrites a link in the
tree)
    TreeNode** curr = &root->left;

    // iterate through each right child of the left subtree to find the rightmost
node
    // return the rightmost node (since that always contains the greatest value)
    while((*curr)->right != nullptr) {
        curr = &(*curr)->right;
    }
    return *curr;
}
```

The next thing we need to code is a helper swap function that swaps the internal data between two nodes. This is simple, it's just swapping the key and value of two nodes. We can use C++ STL swap function to achieve this. The code is shown below

```
template <typename K, typename V>
void BST<K, V>::_swap(TreeNode* a, TreeNode* b) {
    // swap the internal data of nodes `a` and `b`
    std::swap(a->key, b->key);
    std::swap(a->value, b->value);
}
```

Finally, we can top it off by coding the functionality of `remove`. We first need a helper to handle `remove`, `_remove`, for all kinds of nodes, and then we can call the external remove with `root`.

Recall that `_remove` involves handling the cases that vary based on the number of children (leaf node - no children, either a left or right child, both a left or right child). The code for `_remove` is shown below

```
template <typename K, typename V>
void BST<K, V>::_remove(TreeNode*& node, const K& key) {
    if(node == nullptr) { return; }

    if(key < node->key) {
        // descend down the left subtree because left subtree contains the smaller
values
        _remove(node->left, key);
```

```
    } else if(key > node->key) {
        // descend down the right subtree because right subtree contains the
larger values
        _remove(node->right, key);
    } else {
        // case 1: the leaf node case - free the node and set it to null
        if(node->left == nullptr && node->right == nullptr) {
            delete node;
            node = nullptr;
        }

        // case 2: the one child node case (either left or right child)
        else if(node->left == nullptr) {
            // remove the node by connecting its parent directly to the node's
only child (the right child)
            TreeNode* tmp = node;
            node = node->right;
            delete tmp;
        }
        else if(node->right == nullptr) {
            // remove the node by connecting its parent directly to the node's
only child (the left child)
            TreeNode* tmp = node;
            node = node->left;
            delete tmp;
        }

        // case 3: the two child node case (both left and right child)
        // swap the removed node with the iop and recursively call the function on
the left subtree
        else {
            K iop_key = node->key;
            TreeNode* iop = _iop(node);
            _swap(iop, node);

            // recursively call on the left subtree because the iop (the node to
remove) is located there
            _remove(node->left, iop_key);
        }
    }
}
```

A lot of things happen in this code, but most of it has already been previously explained. A couple things to clarify however,

1. If the key we need to remove is less than the node's key, it makes sense to descend down the left subtree, since that contains keys less than the key we need to remove
2. If the key we need to remove is greater than the node's key, it makes sense to descend down the right subtree, since that contains keys greater than the key we need to remove

3. We have to check which child is null for the one child case.
    - When we get to the else if statement, we know that at least one child has to point to some node since we already checked if both were null, and obviously wasn't the case
    - If only the right child is present (the left child is null), then we can simply remove the node and connect its parent directly to this right child. The same thing works with the left child: if only the left child is present, remove the node and connect its parent directly to this left child.
4. For the two child case, once we found the iop and swapped it with the current node, we have to recursively call `_remove` on the left subtree, since that is where the IOP will always be located. That is because the IOP is the greatest node in the left subtree, but smaller than its root (meaning it is the rightmost node of the left subtree)

This makes coding the exposed (external) version of `remove` simple. We just call `_remove` and pass in the root

```
template <typename K, typename V>
void BST<K, V>::remove(const K& key) {
    _remove(root, key);

    // remove the key from the map
    node_map.erase(key);
}
```

`begin()` and `end()` work by simply returning the begin and end iterators of `node_map`, our unordered map

```
template <typename K, typename V>
typename BST<K, V>::TreeIterator BST<K, V>::begin() {
    return node_map.begin();
}

template <typename K, typename V>
typename BST<K, V>::TreeIterator BST<K, V>::end() {
    return node_map.end();
}
```

11.4 BST-Analysis

The table below summarizes the runtime for all our important BST algorithms

Operation	BST Worst Case
`find`	$O(h) \equiv O(n)$
`insert`	$O(h) \equiv O(n)$
`remove`	$O(h) \equiv O(n)$

Operation	BST Worst Case
`traverse`	$O(n)$

For the operations `find`, `insert`, and `remove` on a BST, the worst-case time complexity depends on the height h of the tree. These operations require traversing from the root down to a specific node, which involves up to h recursive or iterative steps. Therefore, their worst-case time complexity is $O(h)$.

If the height h of a Binary Search Tree (BST) can be as large as the number of nodes n in the worst case, then any operation that takes $O(h)$ time can take up to $O(n)$ time. The worst-case scenario holds when $O(h) \equiv O(n)$

The height h of a BST is the length of the longest path from the root to a leaf, measured in edges

1. The worst case scenario occurs when the BST weakens into a `LinkedList`, where each node has only one child
2. The height of this worst case scenario BST, h, then becomes $n - 1$, where $h = n - 1$, since the longest path must visit all nodes except the root

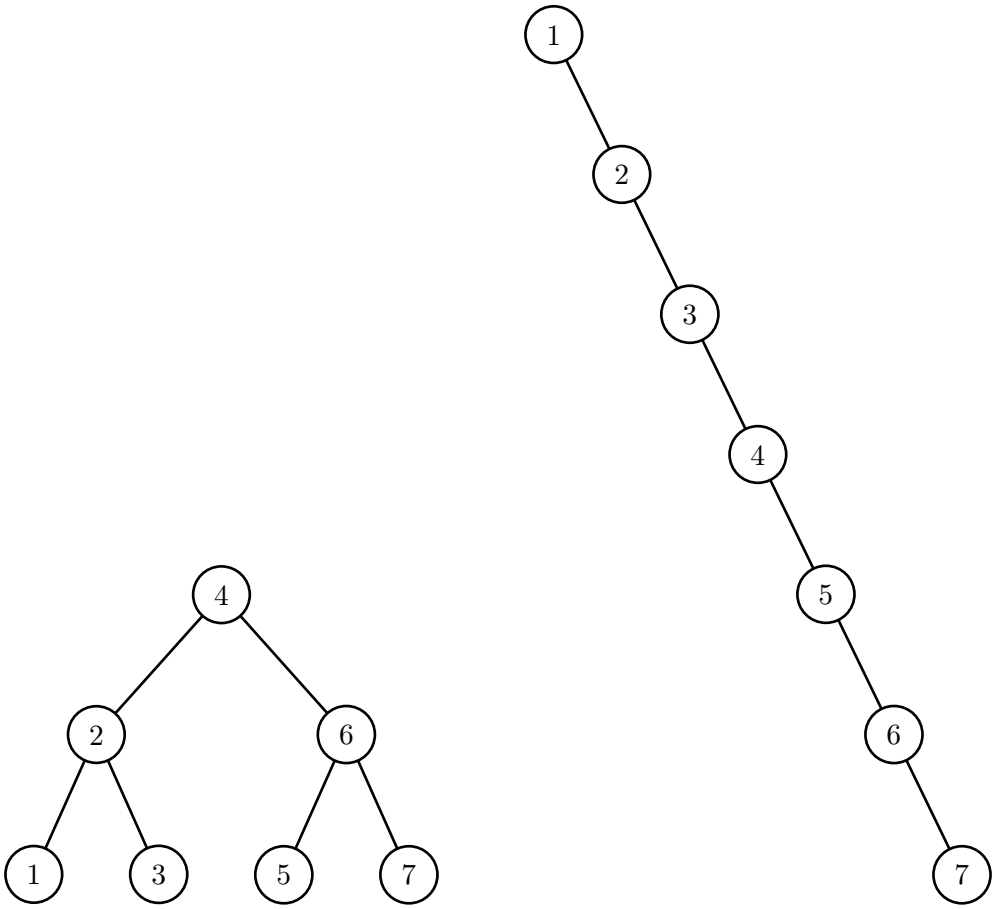
$h = n - 1$, so that means $O(h) = O(n - 1) = O(n)$. Since constants are insignificant in measuring an algorithm's runtime, meaning that $O(h) \equiv O(n)$

12 BST Balance

Previously, we discussed that $O(h) \equiv O(n)$ in the worst case scenario. When comparing our BST runtime analysis to our runtime analysis of a `LinkedList`, it's not actually any better, and is in fact almost the same.

The height of a BST depends on the order in which its nodes were inserted into it

Take a look at **Figure 12.1**, where the same seven keys are inserted in two different orders: the mixed order gives a tree of height 2, but the sorted order gives a stick of height 6.



Insert 4, 2, 6, 1, 3, 5, 7: height 2 Insert 1, 2, 3, 4, 5, 6, 7: height 6

Figure 12.1: Example: the same seven keys. Inserted in a mixed order they form a balanced tree of height $\approx \log_2 n$; inserted in sorted order every node gets only a right child, giving a "stick" of height $n - 1$ that behaves like a linked list

Therefore, a BST can still outperform a linked list because its best-case scenario occurs when the tree is **balanced**, resulting in a height that grows **logarithmically** with the number of nodes. In contrast, a `LinkedList`'s best-case scenario still involves traversing a length that grows **linearly** with the number of elements

This means BST operations can be done in $O(\log(n))$ time only when **balanced**, whereas a linked list requires $O(n)$ time even in the best case.

For all BST's:

- Its **lower bound** (best case scenario) occurs when $h \geq O(\log n) \equiv h \geq O(\log_2 n)$
- Its **upper bound** (worst case scenario) occurs when $h \leq O(n)$

Previously, we discussed why the upper bound must be at most $O(n)$. But, why is the lower bound at least $O(\log n)$?

To explain this, we must understand the structure of a perfect BST, which occurs when each level has twice the number of nodes as the previous level

- Level 0 (root) has 1 node
- Level 1 has 2 nodes
- Level 2 has 4 nodes
- $\vdots$
- Level h has 2^h nodes

This also means that level l has 2^l nodes

This forms a geometric series, where the number of nodes at level l, n_l, occurs as a sum of the geometric series of all nodes up to level l, where

$$n_l = 1 + 2 + 4 + \cdots + 2^l = \sum_{i=0}^{l} 2^i$$

Since $h = l$ at the maximum level of a tree, the total number of nodes in a BST, n, is 2^h, which can also be represented as a sum of the geometric series of all nodes up to level h, where

$$n = 1 + 2 + 4 + \cdots + 2^h = \sum_{i=0}^{h} 2^i$$

The sum of a geometric series is

$$1 + r + r^2 + \cdots + r^n = \sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}$$

Since $r = 2$,

$$\frac{2^{n+1} - 1}{2 - 1} = 2^{n+1} - 1$$

Solving for h,

$$
\begin{aligned}
n &= 2^{h+1} - 1 \\
n + 1 &= 2^{h+1} \\
\log_2(n+1) &= h + 1 \\
h &= \log_2(n+1) - 1
\end{aligned}
$$

Ignoring constants, the best case scenario of a BST occurs at a runtime of $O(\log n)$

Therefore, BSTs can run faster, only if their height remains close to $O(\log n)$

As discussed earlier, a BST's height depends on the number of nodes inserted into it.

If we assume that we have random numbers being inserted into our BST, and choose those random numbers, how many different ways are there?

The simple answer is that there is a lot. Mathematically speaking, if we put n random numbers into our tree, that means we have n choices to choose from for the first number to insert, $n-1$ for the second (since the first number is now gone), and so on. This means that we have $n!$ sequences to insert into our BST, which results in n unique numbers.

If we looked at every possible sequence and computed its height, its average is $O(\log n)$

This is because inserting keys in a random order splits the tree roughly in half at each step, where the tree's height grows the number of times you can halve the keys, which is about $\log_2(n) \equiv \log(n)$

This is delving into the average case runtime of a BST because the time complexity depends on the tree's height, and since the average height of a BST built from random insertions is about $O(\log n)$, the average case runtime for these operations is also $O(\log n)$

The table below summarizes BST analysis with the average case

Operation	**BST Average Case**	**BST Worst Case**	**Sorted List**	**Unsorted List**
`find`	$O(\log n)$	$O(h) \equiv O(n)$	$O(\log n)$	$O(n)$
`insert`	$O(\log n)$	$O(h) \equiv O(n)$	$O(n)$	$O(1)$
`remove`	$O(\log n)$	$O(h) \equiv O(n)$	$O(n)$	$O(n)$
`traverse`	$O(n)$	$O(n)$	$O(n)$	$O(n)$

We discussed the BST average cases, which make sense when thinking about it in context of permutations.

Some other things to note

1. `find` - Sorted List `find` is $O(\log n)$. This is basically performing binary search, since binary search is a more optimal search when a list is sorted. Similarly, unsorted list `find` is $O(n)$, which is simply linear search, which the worst case occurs when the element is found after traversing n times
2. `insert` - Unsorted is $O(1)$, since we can just append an element to the back of a list without maintaining an order, while sorted is $O(n)$ since we have to first insert the element to the back and then shift elements up to n times to make space for the new element

3. `remove` and `traverse` are $O(n)$ for both sorted and unsorted lists since the worst case scenario occurs when having to traverse a maximum of n times

This explains the notion of **balancing** a BST. We want to balance a BST so that its operations come with a cheaper cost of $O(\log n)$

To achieve this, we aim for a tree where the number of nodes in the left and right subtrees of every node is roughly equal. This ensures that the height of the tree remains logarithmic in terms of the number of nodes, preventing degeneration into a `LinkedList`

But how??

To **find the balance of a BST node**, you use the concept of a **balance factor** (b), where

$$b = \text{height}(T_R) - \text{height}(T_L)$$

When $b < 0$, the left subtree is taller; when $b > 0$, the right subtree is taller; and when $b = 0$, the tree is perfectly balanced

If $|b| > 1$, then the tree is unbalanced

Therefore, a tree is height balanced if $|b| \leq 1$ or $|b| < 2$ at every node. Comparing the two trees in **Figure 12.2**, every node on the left passes this check, but the highlighted 8 on the right has $b = -2$.

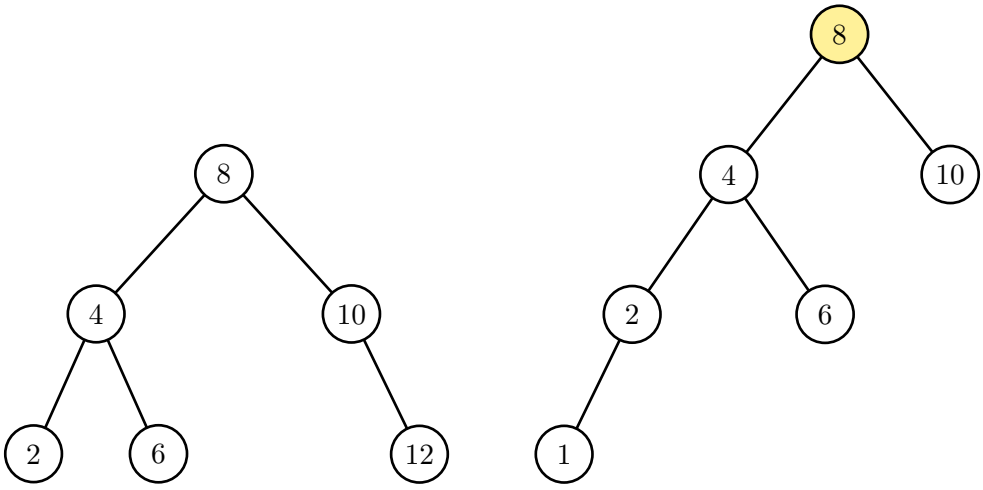


Figure 12.2: Example: on the left, 8 has $b = 1 - 1 = 0$ and 10 has $b = 0 - (-1) = 1$, so every $|b| \leq 1$. On the right, 8 has $b = 0 - 2 = -2$, so the tree is unbalanced even though every other node is fine

13 BST Rotations

A **BST rotation** is a strategy to help change the structure of a BST to help **rebalance** the tree without violating the BST property.

Two key details:

1. Maintain the BST property
   - `node->left < root`
   - `node->right > root`
2. Change a "stick" (every node basically has only one child) into a "mountain" (roughly equal subtrees)

When a BST becomes very imbalanced, where there are plenty of more nodes on one side than another, you can perform **rotations** to rebalance it by redistributing nodes so that the left and right subtrees are more evenly balanced (this is how "sticks" turn into "mountains")

Rotations:

- **Rotate nodes around a pivot** to "move" some nodes from the heavier side with more nodes to the lighter side with less nodes
- After rotation, the tree height decreases or the balancing improves
- The goal is to **evenly split nodes** (or get as close as possible) between left and right subtrees at that pivot node

For example, if the left subtree is much taller than the right, a right rotation can lift the left child up and move the original root down to the right subtree, balancing nodes between the two sides, like the R rotation in **Figure 13.1**, where 4 becomes the new root and its old right child 5 moves over to 6.

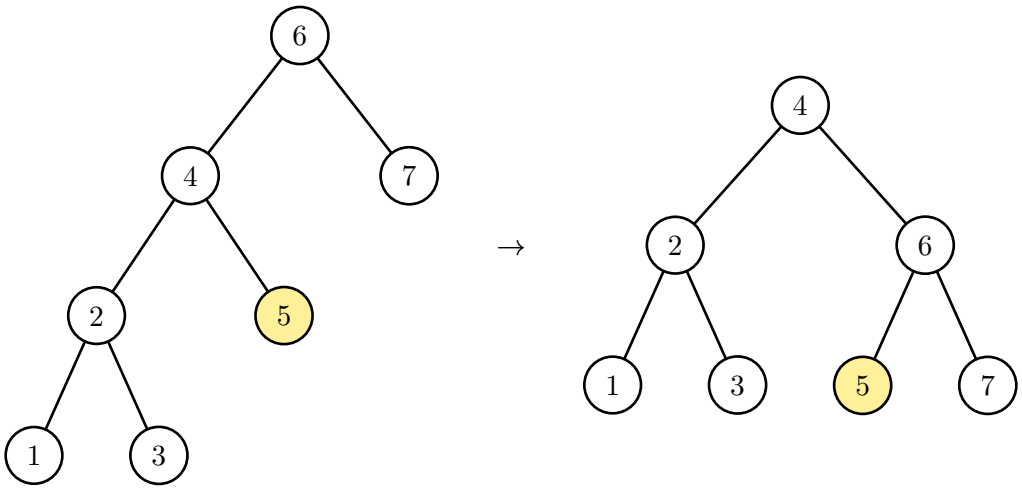


Figure 13.1: Example: an R rotation about 6. Node 4 moves up, 6 moves down to the right, and 4′s right subtree (5) is handed over to become 6′s left subtree

There are a total of **four** rotations: L (left), R (right), LR (left-right), RL (right-left)

- **L** (Left Rotation) - used when the right subtree is too tall and forms a **"right stick"** (due to its shape). This is known formally as the Right-Right case
- **R** (Right Rotation) - used when the left subtree is too tall and forms a **"left stick"** (due to its shape). This is known formally as the Left-Left case
- **LR** (Left-Right Rotation) - Used when there's imbalance in the left-right direction and forms a **"left elbow"** (due to its shape). This is known formally as the Left-Right case, where the left child has a taller right subtree
  - Perform a **left** and then **right** rotation
- **RL** (Right-Left Rotation) - Used when there's imbalance in the right-left direction and forms a **right elbow** (due to its shape). This is known formally as the Right-Left case, where the right child has a taller left subtree
  - Perform a **right** and then **left** rotation

If you're having trouble picturing these shapes, **Figure 13.2** draws all four with just three nodes, labeled with the rotation that fixes each one.

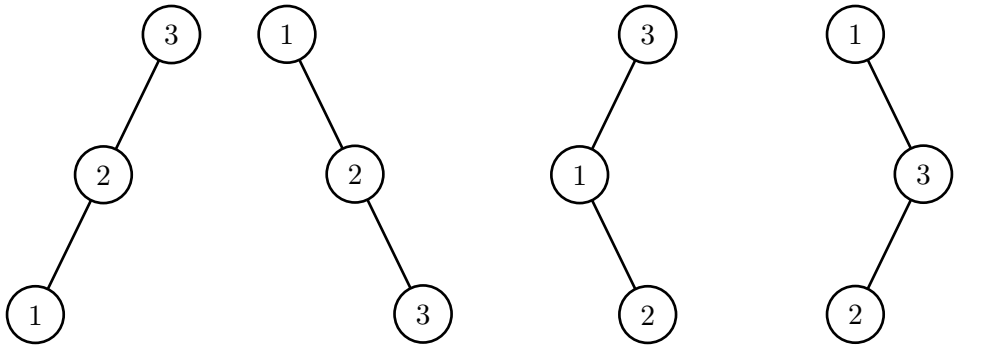


Figure 13.2: The four imbalanced shapes and the rotation that fixes each

Other things to know about rotations:

- All rotations are local, meaning subtrees aren't impacted
- All rotations are constant time, $O(1)$ because they only rearrange a **fixed, small number of pointers** (either a node, its child, and their subtrees); the amount of work doesn't grow with the size of the tree
- The BST property is maintained

The goal is to get $h \equiv O(\log n)$. These types of trees are called **Adelson-Velsky Landis trees** (or AVL trees for short), which are a specific type of **BBST** (Balanced Binary Search Tree)

14 Adelson-Velsky Landis (AVL) Trees

Adelson-Velsky Landis trees, or simply **AVL trees** are **self-balancing binary search trees** that automatically keep themselves balanced through rotations after insertions and deletions, ensuring efficient operations

This means that we have to consider 3 things:

- Rotations
- Maintaining Height
- Detecting Imbalance - check subtree height in $O(1)$ time and compute balance in $O(1)$ time

We can compute balance by having each subtree store its height, which means that I can retrieve it in constant time

Recognize the patterns to figure out which rotation to use

If insertion occurred on a subtree, t, where a **"right stick"** formed (two consecutive right children and no left children), meaning that its right subtree was heavier, we would need to do a **L** rotation about t to restore the balance of the tree. Notice in **Figure 14.1** how 2 becomes the new root once we rotate left about 1.

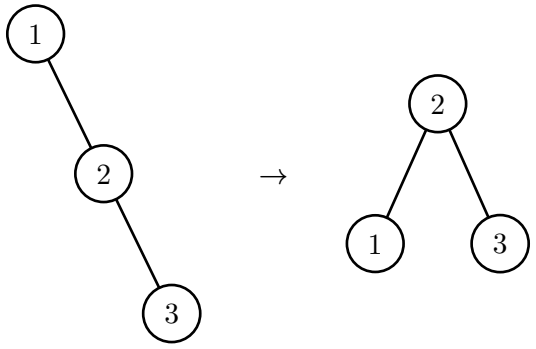


Figure 14.1: A right stick fixed by an L rotation about 1

$h = x$ prior to the left rotation and is now $h = x + 1$, where x is the height of the right subtree

We leverage this by noting that the balance factor, b of t is $+2$ and the balance factor of `t->right` is 1, which is greater than 0 (`t->right > 0`)

If insertion occurred on a subtree, t, where a **"right elbow"** formed (right child has taller left subtree), we would need to do a **RL** rotation about t to restore the balance of the tree (see **Figure 14.2**, where the R rotation about 3 first turns the elbow into a right stick).

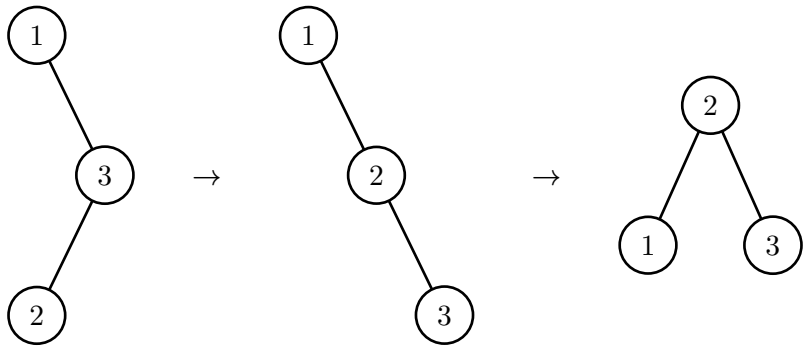


Figure 14.2: A right elbow fixed by an RL rotation: an R rotation about 3, then an L rotation about 1

We leverage this by noting that the balance factor, b of `t->right` is -1, which is less than 0 (`t->right < 0`)

The opposites are true for a **"left stick"** and **"left elbow"**, where we would need **R** and **LR** rotations respectively. If you compare **Figure 14.3** and **Figure 14.4** with the right side versions above, they're just mirror images

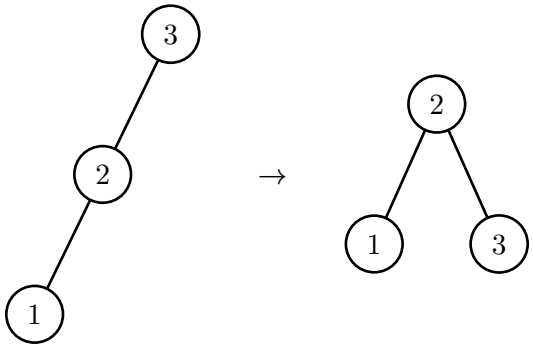


Figure 14.3: A left stick fixed by an R rotation about 3

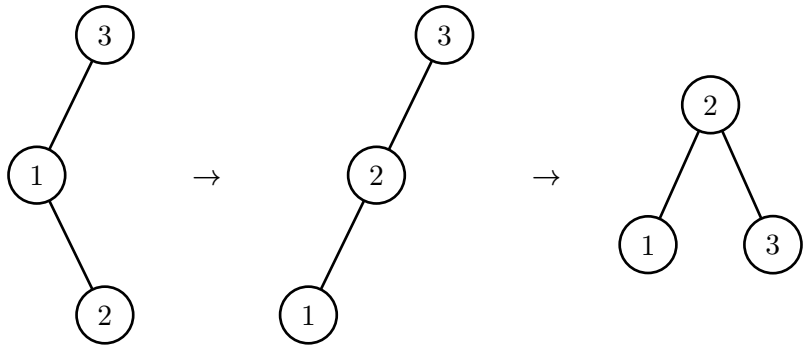


Figure 14.4: A left elbow fixed by an LR rotation: an L rotation about 1, then an R rotation about 3

14.1 Insertion into an AVL tree

With the knowledge in BST rotations, to insert a node into an AVL tree, we know we have to do the following steps

1. Insert at proper place
2. Check for imbalance
3. Rotate, if necessary
4. Update height

We already established that we need each `TreeNode` in an AVL tree to track its height. Therefore, we add a `height` member to each `TreeNode`

```
struct TreeNode {
    K key;
    V value;

    // have each `TreeNode` store the height in order to self balance
    unsigned int height;

    TreeNode* left;
    TreeNode* right;

    // a new node is a leaf, so it has height 0 and no children
    TreeNode(const K& key, const V& value)
        : key(key), value(value), height(0), left(nullptr), right(nullptr) {}
};
```

Inserting a node into an AVL tree is very similar to a BST (since it is in fact a type of BST!), as shown below

```
template <typename K, typename V>
void AVL<K, V>::_insert(const K& key, const V& value, TreeNode*& curr) {
    // found the empty spot where the new node belongs
    if(curr == nullptr) {
        curr = new TreeNode(key, value);
        return;
    }

    // maintain BST property
    if(curr->key > key) {
        _insert(key, value, curr->left);
    } else if(curr->key < key) {
        _insert(key, value, curr->right);
    }

    // we still need to ensure the balance of the AVL tree here
}
```

```
template <typename K, typename V>
void AVL<K, V>::insert(K key, V value) {
    _insert(key, value, root);
    node_map[key] = value;
}
```

The code above ensures the AVL tree maintains BST properties by descending into the left subtree if the key is smaller and descending into the right subtree if the key is greater

However, we are missing one crucial detail: ensuring that the AVL tree is **balanced**. To do this, we need to calculate the **balance factor** and then based on the balance factor, perform the appropriate rotation

The balancing function is outlined below

```
template <typename K, typename V>
void AVL<K, V>::_ensureBalance(TreeNode*& curr) {
    // calculate the balance factor
    int balance = height(curr->right) - height(curr->left);

    // check if the node is currently not in balance
    if(balance == -2) { // heavy left ("left stick" or "left elbow" shaped
subtree)
        int leftBalance = height(curr->left->right) - height(curr->left->left);
        if(leftBalance <= 0) {
            // perform right rotation
        } else {
            // perform left-right rotation
        }
    } else if(balance == 2) { // heavy right ("right stick" or "right elbow"
shaped subtree)
        int rightBalance = height(curr->right->right) - height(curr->right->left);
        if(rightBalance >= 0) {
            // perform left rotation
        } else {
            // perform right-left rotation
        }
    }

    _updateHeight(curr);
}
```

This is a lot of code, but what we are essentially doing is calculating the balance factor so we know which rotation to perform!

Compute the balance of the tree to determine the shape of a stick

- If the balance is -2, we know that the shape of the left subtree is either a **"left stick"** or **"left elbow"**

    - We then need to compute the balance of the left subtree to figure out specifically which one it is
    - A left subtree balance of -1 means that its a **"left stick"**, so we need to do a **right rotation**
    - A left subtree balance of $+1$ means that its a **"left elbow"**, so we need to do a **left-right rotation**
- Similarly, if the balance is $+2$, we know that the shape of the right subtree is either a **"right stick"** or **"right elbow"**
    - We then need to compute the balance of the right subtree to figure out specifically which one it is
    - A right subtree balance of $+1$ means that its a **"right stick"**, so we need to do a **left rotation**
    - A right subtree balance of -1 means that its a **"right elbow"**, so we need to do a **right-left** rotation
    - After a `remove`, the child's balance can also be 0. That case is fixed with a single rotation too, which is why the code checks `rightBalance >= 0` (and `leftBalance <= 0` on the left side) instead of exactly ± 1

We need a helper height function in order to avoid any crashes. While `curr->left->height` and `curr->right->height` do work, they may crash if `curr->left` and `curr->right` are null.

Therefore, a helper function to get the height of a node can help prevent any unnecessary crashes. It simply checks if the node is not null before calling its height property, and returns -1 for a null node, since the height of a null tree is -1. The code is shown below

```
template <typename K, typename V>
int AVL<K, V>::height(TreeNode* node) {
    return node != nullptr ? node->height : -1;
}
```

We also need a helper function to update the height after performing necessary rotations

The updated node's height is 1 plus the max height of its left and right subtrees

Height represents the number of edges in the longest path from that node down to a leaf. When calculating this longest path, you need to consider that it will go through either the left subtree or the right subtree, depending on whichever is taller. This is performed by taking the maximum height between the two subtrees

However, to get the total path length from the current node, you must add one additional edge, because that accounts for descending into the taller subtree, which `_updateHeight` does below

```
template <typename K, typename V>
void AVL<K, V>::_updateHeight(TreeNode* curr) {
    // calculate height as 1 + maximum height of children
    // the +1 accounts for the extra edge needed to descend into the subtree with
a greater height
    // this works after any tree modification because height depends on the
```

```
current tree structure
    if(curr != nullptr) {
        curr->height = 1 + std::max(height(curr->left), height(curr->right));
    }
}
```

From here, the rest is simple. We just have to simply code each rotation. The rotations intuitively make sense to code if you know what's going on

1. **Left Rotation**
   - The right child becomes the new root of the subtree
   - The new root's left subtree becomes the old root's right child
   - The old root becomes the left child of the new root of the subtree
   - Update the heights of the old root and then the new root, since only those two nodes moved
2. **Right Rotation**
   - The left child becomes the new root of the subtree
   - The new root's right subtree becomes the old root's left child
   - The old root becomes the right child of the new root of the subtree
   - Update the heights of the old root and then the new root, since only those two nodes moved
3. **Left-Right Rotation**
   - Perform a left rotation about the left child
   - Perform a right rotation about the root
4. **Right-Left Rotation**
   - Perform a right rotation about the right child
   - Perform a left rotation about the root

Coding these then becomes easy. Below are `_rotateLeft`, `_rotateRight`, `_rotateLeftRight`, and `_rotateRightLeft`, in that order

```
template <typename K, typename V>
void AVL<K, V>::_rotateLeft(TreeNode*& root) {
    // the right child becomes the new root
    TreeNode* newRoot = root->right;

    // move `newRoot`'s left subtree to old root's right subtree
    root->right = newRoot->left;

    // the old root becomes the left child of the new root
    newRoot->left = root;

    // the old root is now below the new root, so update its height first
    _updateHeight(root);
    _updateHeight(newRoot);

    // update pointer to point to the new root
    root = newRoot;
}
```

```
template <typename K, typename V>
void AVL<K, V>::_rotateRight(TreeNode*& root) {
    // the left child becomes the new root
    TreeNode* newRoot = root->left;

    // move `newRoot`'s right subtree to old root's left subtree
    root->left = newRoot->right;

    // the old root becomes the right child of the new root
    newRoot->right = root;

    // the old root is now below the new root, so update its height first
    _updateHeight(root);
    _updateHeight(newRoot);

    // update pointer to point to the new root
    root = newRoot;
}
```

```
template <typename K, typename V>
void AVL<K, V>::_rotateLeftRight(TreeNode*& root) {
    // rotate left about the left child and then rotate right about the root
    _rotateLeft(root->left);
    _rotateRight(root);
}
```

```
template <typename K, typename V>
void AVL<K, V>::_rotateRightLeft(TreeNode*& root) {
    // rotate right about the right child and then rotate left about the root
    _rotateRight(root->right);
    _rotateLeft(root);
}
```

This updates our balance function as follows

```
template <typename K, typename V>
void AVL<K, V>::_ensureBalance(TreeNode*& curr) {
    // calculate the balance factor
    int balance = height(curr->right) - height(curr->left);

    // check if the node is currently not in balance
    if(balance == -2) {
        int leftBalance = height(curr->left->right) - height(curr->left->left);
        if(leftBalance <= 0) { // heavy left ("left stick" shaped subtree)
            _rotateRight(curr);
        } else { // "left elbow" shaped subtree
            _rotateLeftRight(curr);
```

```
        }
    } else if(balance == 2) {
        int rightBalance = height(curr->right->right) - height(curr->right->left);
        if(rightBalance >= 0) { // heavy right ("right stick" shaped subtree)
            _rotateLeft(curr);
        } else { // "right elbow" shaped subtree
            _rotateRightLeft(curr);
        }
    }

    // update the height of `curr`, which is the root of this subtree after any
rotation
    _updateHeight(curr);
}
```

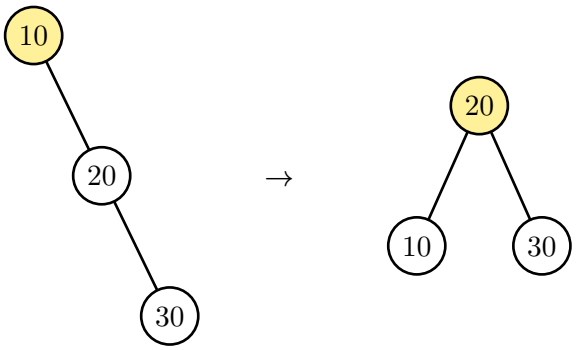
Finally, `_insert` calls `_ensureBalance` on every node along the path back up from the new node, so any imbalance is fixed right where it appears

```
template <typename K, typename V>
void AVL<K, V>::_insert(const K& key, const V& value, TreeNode*& curr) {
    // found the empty spot where the new node belongs
    if(curr == nullptr) {
        curr = new TreeNode(key, value);
        return;
    }

    // maintain BST property
    if(curr->key > key) {
        _insert(key, value, curr->left);
    } else if(curr->key < key) {
        _insert(key, value, curr->right);
    }

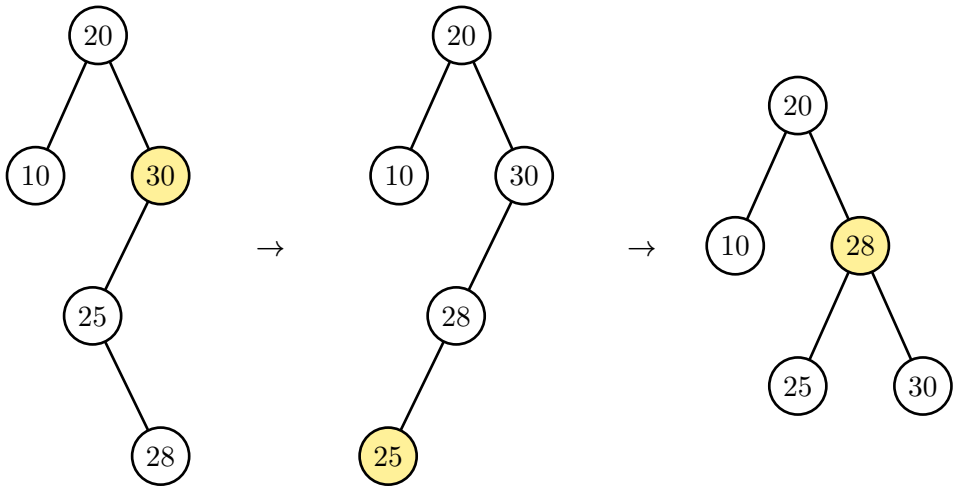
    // ensure the balance of the AVL tree (this also updates the height of `curr`)
    _ensureBalance(curr);
}
```

For example, inserting the keys 10, 20, 30, 25, 28 in that order triggers two rebalances. The first happens right after 30 goes in, when 10 becomes a right stick and gets an L rotation (see **Figure 14.5**). The second happens after 28, when 30 becomes a left elbow and needs an LR rotation, which you can follow step by step in **Figure 14.6**.



Insert 10, 20, 30 L rotation about 10

Figure 14.5: Example: inserting 10, 20, 30 forms a right stick at 10 (balance +2), fixed by an L rotation



Insert 25, then 28 L rotation about 25 R rotation about 30

Figure 14.6: Example: inserting 25 and then 28 forms a left elbow at 30 (balance −2, left child balance +1), fixed by an LR rotation

I will not code the remaining functionalities, including `find` and `remove`. `find` is identical to a regular BST, so please see the BST implementation for it. `remove` starts out exactly like BST `remove`, but it is **not** identical: removing a node can unbalance every ancestor on the path back up to the root. Just like `_insert`, `_remove` must call `_ensureBalance` on each node as the recursion unwinds, and a single `remove` may need a rotation at several of those nodes (up to $O(\log n)$ rotations), while an `insert` needs at most one.

14.2 AVL Tree Analysis

`insert`, `remove`, and `find` also all run in $O(h)$. Recall that this is because in the worst case, we may need to traverse from the root down to a leaf node, visiting $h + 1$ nodes.

However, we will now argue that $h = O(\log n)$, using the balance property to prove this

Recall from discrete math, the definition of big-O states that $f(n)$ is $O(g(n))$ iff $\exists_{c,k}$ such that $f(n) \leq c \cdot g(n)$ for all $n > k$

Or simply, the worst-case growth rate of a function as the input gets very large. When I give it more data, how does it slow down?

Let $h(n)$ be the height of an AVL tree as a function of n, the number of nodes. However, we can't directly compute this function, since tree height depends on not only the number of nodes, but also on the structure of the tree.

In other words, 2 AVL trees can have 4 nodes, but be fundamentally different in structure

Since we can't compute the exact height function, $h(n)$, we analyze the worst-case scenario: which is the tallest possible tree with n nodes, or simply $h_{\max}(n)$

But directly computing $h_{\max}(n)$ is also difficult. There are too many possible tree shapes to check them all. Instead, we approach the problem indirectly by taking the inverse and finding the **minimum number of nodes** required to build an AVL tree of height h

This makes sense because when I find the lower bound of the inverse, $h_{\min}(n)$, I essentially get the upper bound on $h_{\max}(n)$

The number of nodes in the tree, $f^{-1}(h)$ will always be **greater than** $c \times g^{-1}(h)$ for all values where $n > k$

This is because if one function grows slower than another, its inverse must grow faster, so to reach the same output, the slower-growing function requires a larger input, making its inverse larger (up to a constant factor) for sufficiently large values

To find the lower bound on n given h, we can begin by defining a function given h, which describes the **smallest number of nodes in an AVL tree** given height h

NODES(h) is the minimum number of nodes in an AVL tree of height h.

- When $h = -1$, NODES(h) $= 0$ because the tree is **empty**
- When $h = 0$, only one tree exists because only the root node exists in such tree, so NODES(h) $=$ 1
- When $h = 1$, there are three possible trees: a tree with only a **left child**, a tree with only a **right child**, and a tree with **both** a **left and right** child. NODES(h) $= 2$ since the minimum number of nodes required for height 1 is 2
- When we get to $h = 2$, however, I get a lot of possible trees and not all of them are balanced.
  - However, there is one possible balanced tree that can be formed with a minimum of four nodes, where the **root's left child also contains a right child**
  - This forms a minimum of 4 nodes, where NODES(h) $= 4$, as in **Figure 14.7** (the root's left child 1 has the right child 2)

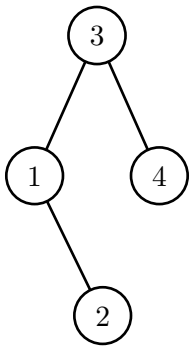


Figure 14.7: A minimum-size AVL tree of height 2

Assuming that $N(h)$ is an alias for NODES(h), we build a recurrence relation, where

$$N(h) = 1 + N(h-1) + N(h-2)$$

If we build the first few minimum-size AVL trees with this recurrence, we get **Figure 14.8**. Notice how the tree for $N(3)$ is just a root with the $N(2)$ tree on its left and the $N(1)$ tree on its right.

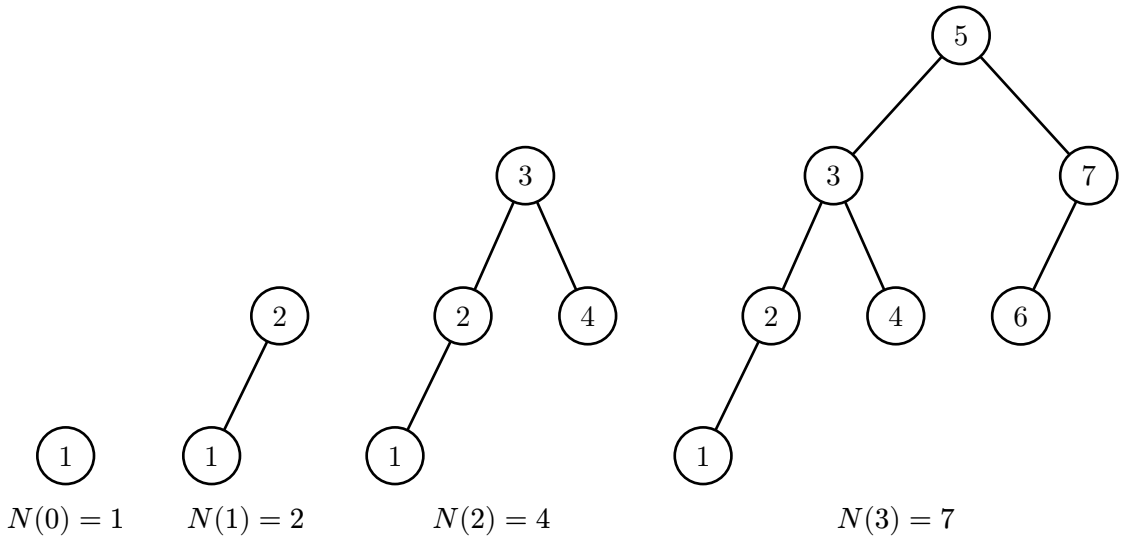


Figure 14.8: Minimum-size AVL trees of heights 0 to 3: each is a root, a minimum tree of height $h-1$ on the left, and a minimum tree of height $h-2$ on the right

We can first argue that $N(h) = 1 + N(h-1) + N(h-2)$, which means that $N(h) > 2N(h-2)$. This is important so that we establish a lower bound on the minimum required number of nodes for height h

We can argue that the minimum number of nodes $N(h)$ in an AVL tree of height h is equal to the sum of one root node plus the minimum nodes of its two subtrees: $N(h) = 1 + N(h-1) + N(h-2)$.

Since $N(h-1) > N(h-2)$, this implies $N(h) > 1 + 2N(h-2)$ because $N(h-1)$ is strictly greater than $N(h-2)$

For larger h, the constant 1 becomes insignificant, so we simplify this to $N(h) > 2N(h-2)$, showing that $N(h)$ grows exponentially as the height increases

With this, we can prove a closed form of $2N(h-2)$

Prove that an AVL tree of height h has at least $2^{\frac{h}{2}}$ nodes.

Consider an AVL tree and let h denote its height. Start with $h = 1$

$$N(1) = 2^1 = 2$$

$$2^{\frac{1}{2}} = \sqrt{2}$$

and $2 \geq \sqrt{2}$

This means that an AVL tree with height 1 has at least $\sqrt{2}$ nodes

Suppose $h = 2$

$$\begin{aligned} N(2) &= 1 + N(2-2) + N(2-1) \\ &= 1 + 1 + 2 \\ &= 4 \end{aligned}$$

$$2^{\frac{2}{2}} = 2^1 = 2$$

and $4 \geq 2$, which means that an AVL tree with height 2 has at least 2 nodes

By inductive hypothesis

$\forall_{j<h}$, $N(j) \geq 2^{\frac{j}{2}}$, where we will prove via inductive step that

$$N(h) = N(h-1) + N(h-2) + 1 > 2N(h-2)$$

$$2N(h-2) \geq 2 \times 2^{\frac{h-2}{2}} = 2^{\frac{h}{2}}$$

An AVL tree of height h has at least $2^{\frac{h}{2}}$ nodes.

via induction, we have shown that $n \geq N(h) \geq 2^{\frac{h}{2}}$, where inverting this results in

$$h \leq 2 \log n$$

meaning that

$$h \equiv O(\log n)$$

14.3 Summarizing AVL and BBST

Red-Black Trees

- BBST's where max height is equal to $2 \cdot \log n$
- Constant number of rotations on `insert`, `remove`, and `find`, $O(1)$

AVL Trees

- **Max Height**: $1.44 \cdot \log n$
- **Rotations**: $O(h)$

Red-Black trees outperform AVL trees in rotations. They are also laid out well in memory that improves cache performance.

Red-Black trees are much more complicated to understand how they work under the hood

Previously, when we were constructing BST's, we were using `std::unordered_map` to track our nodes. This was so that we had access to a fast iterator. But there is actually something even more interesting about this.

C++ STL maps in fact provide us with a balanced BST that is in fact a red-black tree. When we use `operator[]` on a map, what we are essentially doing is searching for the key in a tree and

- **If the key exists**, it returns a reference to the corresponding value.
- **If the key doesn't exist**, it inserts the key with a default-constructed value and then returns a reference to that value.

But from a big-O standpoint, all operations are $O(\log n)$, considering the trees are **balanced**, which is a huge improvement from $O(n)$ of a regular BST

This is great for specific applications when looking for close matches, such as range queries, nearest-neighbor lookups, or finding the next largest or smallest key, all of which benefit from the in-order structure and efficient traversal of balanced binary search trees like red-black trees

14.4 Runtime Analysis for all Data Structures Covered

	Unsorted Array	Sorted Array	Unsorted List	Sorted List	Binary Tree	BST	AVL
`find`	$O(n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$	$O(h)$	$O(\log n)$
`insert`	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(h)$	$O(\log n)$
`remove`	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(h)$	$O(\log n)$
`traverse`	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

15 Lambdas

> NOTE: This is a simple lesson reviewing iterators and lambda functions from CS 128!

15.1 Iterators

From 128, we know that there are **three** types of iterators:

1. **Forward**
   - Only moves in the **forward** direction,
   - **Operations**: `++it` (increment), `*it` (dereference), `==` and `!=` (comparison)
   - **Example**: Singly `LinkedList` iterator, where you can go through the list once from start to end, but can't go backwards
2. **Bidirectional**
   - Can move in **both directions** (forward and backward)
   - **Operations**: Everything from Forward, plus `--it` (decrement)
   - **Example**: Doubly linked list iterator and `std::set` iterator
   - You can traverse forward with `++` or backward with `--`
3. **Random Access**
   - Can **jump to any position** in constant time
   - **Operations**: Everything from Bidirectional, plus `it + n`, `it - n`, `it[n]`, `<`, `>`, etc.
   - **Example**: `std::vector` iterator, array pointers
   - You can do `it + 5` to jump 5 positions ahead, or `it[10]` to access element 10

15.2 Lambdas

In C++, a **lambda function** is an **anonymous function** you can define on the fly and inline. This means that I define and write the function right when I use it, instead of somewhere else

This is typically used for short snippets of code like callbacks, custom sorting, or quick operations that don't necessarily need a separate named function.

Lambda functions are so useful because they allow us to use functions as **data**. This means that the function itself is a value that can be stored, passed around, and manipulated!

Lambdas consist of 3 things

1. Capture, notated in `[ ]` - specifies which variables from the surrounding scope the lambda can access and whether it captures them by value (copy) or by reference
2. Parameter, notated in `( )` - The list of parameters with a type and name specified and separated with commas
3. Body, notated in `{ }` - The body of the lambda

For example,

```
auto isBig = [big](int num) { return num >= big; };
```

- `[ ]` contains the variables within scope that the lambda "captures" for it to use. In this example, this variable is `big`
- `( )` contains the parameters needed to pass into the function
- `{ }` contains the body of the function

Take a look at a popular function used in excel, `COUNTIF`. This function counts the number of cells in a range that meet a specific criteria

Suppose the criteria is that any integer, x, is less than 5. To code this in C++ we would do,

```
bool lessThan5(int x) {
    return x < 5;
}

int countIf(std::vector<int>& data) {
    return std::count_if(data.begin(), data.end(), lessThan5);
}
```

In **Figure 15.1**, for example, `count_if` calls `lessThan5` on every value from `begin()` up to `end()`, and only 3, 1, and 4 (highlighted) pass.

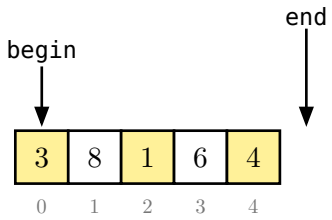


Figure 15.1: Example: `count_if` walks from `begin()` up to, but not including, `end()` and counts the highlighted values less than 5, returning 3

However, `lessThan5` is an unnecessary function when you can achieve the same result more concisely and directly using a lambda expression inline, avoiding extra named functions and keeping the code simpler and easier to read.

To solve this problem, we can use a **lambda**, like the one below

```
auto lessThan5 = [](int x) { return x < 5; };
```

this makes our `countIf` function much more readable

```
int countIf(std::vector<int>& data) {
    auto lessThan5 = [](int x) { return x < 5; };
```

```
    return std::count_if(data.begin(), data.end(), lessThan5);
}
```

16 BBST Range Search and KD Trees

Balanced BST's are useful for **range-based** and **nearest-neighbor** searches (e.g. finding out which nodes in a tree have values that fall between $[11, 42]$)

16.1 Upper and Lower Bound

Previously, we discussed how C++'s STL map is implemented via a red-black tree under the hood

`std::map` includes a `lower_bound` and an `upper_bound` function, which allow efficient logarithmic-time range queries by returning iterators to the first element not less than (`lower_bound`) and the first element greater than (`upper_bound`) a given key, respectively.

- `lower_bound` - The smallest element greater than or equal to the argument
- `upper_bound` - The smallest element strictly greater than the argument

16.2 Range-Based Searches

Since BBST's are balanced, its height is $O(\log n)$, so range queries can be efficient when combining with in-order traversal, since that maintains sorted order

Range queries are a search to find all keys that fall within a given range, typically written as $[x, y]$, where x is the lowest possible value (lower bound) and y is the highest possible value (upper bound). For example, a range search for $[11, 42]$ in **Figure 16.1** returns the highlighted keys and never even visits 4 or 60.

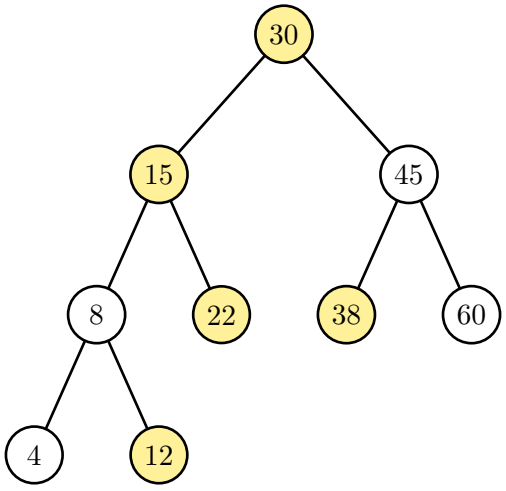


Figure 16.1: Example: a range search for $[11, 42]$ returns the highlighted keys. 8 and 45 are out of range, so only one of their subtrees is searched, and 4 and 60 are never visited

A ***k*-dimensional tree**, or **KD Tree** is a data structure used to store and search points in k-dimensional space efficiently for range queries and nearest neighbor searches

Range based searching has a lot of real world applications, involving **computer graphics**, **collision detection**, and **sensor data**. Each of these things relies on quickly identifying all objects, points, or events that fall within a specific region or interval, so that only relevant data is processed while ignoring everything outside the range, which greatly improves efficiency.

16.3 KD Trees

Range based searching becomes efficient with a type of tree known as a **KD Tree**, which organizes points so that large portions of space can be skipped entirely during a query

To build a KD tree from k-D space

1. Start with one dimension (doesn't matter which one)
2. Split dataset into two parts: points less than the median go on the left subtree and points greater than the median go on the right subtree
3. Create a node with the median point as the current root
4. Recursively build the left and right subtrees: move to the next dimension (e.g., if you split by x at this level, split by y at the next)
5. Repeat steps 2-4 for each subtree with their respective points

Following these steps on six points gives the KD tree in **Figure 16.2**, with (7,2) at the root.

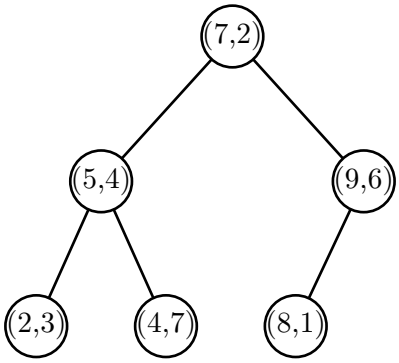


Figure 16.2: Example: the KD tree built from (2,3), (5,4), (9,6), (4,7), (8,1), (7,2). The root splits on x and its children split on y

A KD tree stays balanced simply because of the median because at each step you split the dataset using the **median** point along the current dimension, ensuring that half the points go on the left subtree and half the points go on the right subtree

16.4 k-Nearest Neighbor (k-NN)

***k*-Nearest Neighbors (k-NN)** is a simple algorithm used in machine learning and data analysis to find the k closest points to a given query point in a dataset

It is used very often in **Machine Learning**, specifically classification and regression

- **Classification** - Assign the query point the most common class among its k nearest neighbors.
- **Regression** - Predict the query point's value as the average of its k nearest neighbors' values.

It becomes very efficient with a constructed KD Tree because it avoids brute force checking each point. Instead, it narrows the search by recursively exploring only those regions of space that could contain points closer than the current best candidates, ignoring irrelevant subtrees, to make nearest neighbor searches much faster

To find the nearest neighbor in a KD Tree, given a query point

1. Start at the root and move down the tree, choosing the child subtree that contains the query point based on the splitting dimension
2. Once you reach a leaf, record that point as the current best candidate
3. Backtrack up the tree, at each node checking if there could be points in the other subtree closer than the current best (by comparing distance to the splitting plane)
4. If so, explore the other subtree to find potentially closer points
5. Continue this process until all promising subtrees have been checked
6. Return the closest point found as the nearest neighbor

17 BTrees

In Big-O, we assume that we have uniform time for all operations, but this is not always the case

Sometimes, searching, inserting, and deleting data in a AVL tree is not always efficient when working with disks, where access is much slower than in memory

A **B-Tree** is a self-balancing search tree designed for efficient storage and retrieval on disks. Unlike a binary search tree that stores only one key per node, each B-Tree node can hold multiple keys and have multiple children

- The keys inside a node are kept in **sorted order**, and the children split the key ranges between them
- All leaves appear at the same depth, so the tree remains balanced
- Because each node can hold many keys (often filling a disk block), the tree height is very small, which reduces disk accesses
- Operations like search, insertion, and deletion still take $\boldsymbol{O(\log n)}$ time, but with a much smaller constant factor, making B-Trees ideal for databases and file systems

Take the small B-tree in **Figure 17.1**: its root holds two keys, 20 and 40, which split its three children by range.

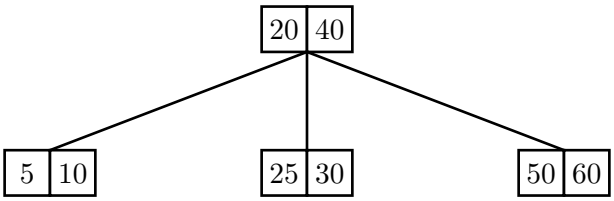


Figure 17.1: Example: a B-tree of order 3. Each node holds at most 2 sorted keys, the keys 20 and 40 split the children into < 20, between 20 and 40, and > 40, and every leaf is at the same depth

When an insertion fills a node past its capacity, the node splits, like in **Figure 17.2**, where 27 overflows the leaf and gets pushed up into the root.

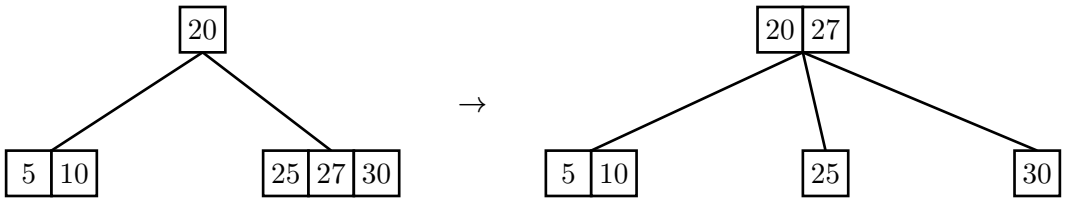


Figure 17.2: Example: inserting 27 into a B-tree of order 3. The leaf overflows to three keys, so its middle key moves up into the parent and the leaf splits in two

18 BTree Analysis

19 Priority Queues

20 Heaps

21 Disjoint Sets

22 Graphs

23 Graph Implementations

24 Graph Traversals

25 MST: Minimum Spanning Tree (Kruskal's Algorithm)

26 MST: Minimum Spanning Tree (Prim's Algorithm)

27 Dijkstra's Algorithm

28 Floyd Warshall's Algorithm

29 Probability in Computer Science

30 Hashing, Hash Collisions, and Hash Tables

31 Bloom Filters

32 Cardinality

33 MinHash Sketch Strategy

34 Wrapping up CS 225!

Appendix A: Midterm Reviews

A.1 Midterm 0: Review of Prerequisite Material

Midterm 0 is CS 225′s formal introduction to UIUC's Computer Based Testing Facility (CBTF)

Covers the info discussed in course prerequisites, including Graph Theory from **CS 173/MATH 213** and C++ principles from **CS 128/ECE 220**

A.1.1 Topics Covered

- Course Policies
- Mathematical Foundations
  - Be able to give a closed form solution to a recurrence relation
  - Be able to indicate the big-O running time of common algorithms (eg: binary search, merge sort, etc)
  - Be able to understand basic graph terminology
  - Be familiar with Trees
  - Be able to understand a standard proof and proof by induction
- C++ Topics
  - Classes in C++
    - Public member functions
    - Private helper functions
    - Private variables
    - Constructors
      - Automatic default constructor
      - Custom constructors (default and non-default)
    - Copy constructor
      - Automatic copy constructor
      - Custom copy constructor
    - Inheritance
  - Namespaces in C++
    - Creating a class that is part of a namespace (eg: `Cube` is part of the `cs225` namespace)
    - Using a class from a namespace (eg: `cs225::Cube`)
    - Purpose and usefulness of namespaces
  - Variables
    - Four properties: name, type, location (in memory), and value
    - Primitive vs. user-defined
  - Memory
    - Indirection in C++:
      - Reference variables
      - Pointers

    - Differences and trade-offs between each type
  - Stack memory
  - Heap memory
- Functions: Calling and Returning
  - Pass by value, by reference, and by pointer
  - Return by value, by reference, and by pointer

A.1.2 Recursive Definitions

- **Recursive definitions** contain formulas where each term is based on one or more preceding terms
  - Many recursive numerical formulas have something called a **closed form**, which is a closed expression that doesn't involve computing each term in the formula/sequence (similar to a summation)
    - Deriving this closed form is an act known as **unrolling**
      - When unrolling, we substitute a recursive definition into itself until there is an explicit pattern suggesting the closed form expression

For example with the expression,

$$\begin{aligned} T(n) &= 2T(n-1) + 3 \\ &= 2(2T(n-2) + 3) + 3 \\ &= 2(2(2T(n-3) + 3) + 3) + 3 \\ &= 2^3T(n-3) + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &= 2^4T(n-4) + 2^3 \cdot 3 + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &\vdots \\ &= 2^kT(n-k) + 2^{k-1} \cdot 3 + ... + 2^2 \cdot 3 + 2 \cdot 3 + 3 \end{aligned}$$

we can use summation notation to represent the kth unrolling step, where

$$\begin{aligned} T(n) &= 2^kT(n-k) + 2^{k-1} \cdot 3 + ... + 2^2 \cdot 3 + 2 \cdot 3 + 3 \\ &= 2^kT(n-k) + 3\big(2^{k-1} + ... + 2^2 + 2 + 1\big) \\ &= 2^kT(n-k) + 3\sum_{i=0}^{k-1}(2^i) \end{aligned}$$

The input of function T hits the base case when $n - k = 1$, or when $k = n - 1$. Recall that $T(1) = 1$ and substituting $n - 1$ value for k gives

$$
\begin{aligned}
T(n) &= 2^k T(n-k) + 3\sum_{i=0}^{k-1}(2^i) \\
&= 2^{n-1}T(1) + 3\sum_{i=0}^{n-2}(2^i) \\
&= 2^{n-1} + 3\sum_{k=0}^{n-2}(2^k) \\
&= 2^{n-1} + 3(2^{n-1} - 1) \\
&= 4(2^{n-1}) - 3 \\
&= 2^{n+1} - 3
\end{aligned}
$$

therefore making the closed form for function T, $2^{n+1} - 3$

1. Given that $T(0) = 5$, find the closed form solution of

$$T(n) = T(n-1) + 2$$

   a. Unroll the first couple of terms

$$
\begin{aligned}
T(n) &= T(n-1) + 2 \\
&= (T(n-2) + 2) + 2 \\
&= ((T(n-3) + 2) + 2) + 2 \\
&\vdots \\
&= T(n-k) + 2k
\end{aligned}
$$

   b. where $k = n$ and $T(n) = T(0) + 2n = 5 + 2n$
   c. Recall that $T(0)$ can represent any sort of numeric value, where in this case is 5. This numeric value of 5 is known as the **base case** and $2n$ is known as the **recursive case**

2. Given that $T(1) = 3$, find the closed form solution of

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

   a. Unroll the first couple terms of the sequence

$$
\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + n \\
&= 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 4T\left(\frac{n}{4}\right) + 2n \\
&= 2\left(2\left(2T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + \frac{n}{2}\right) + n = 8T\left(\frac{n}{8}\right) + 3n \\
&\vdots \\
&= 2^k T\left(\frac{n}{2^k}\right) + kn
\end{aligned}
$$

   b. where $\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log_2 n$ and

$$
\begin{aligned}
T(n) &= 2^{\log_2 n} \cdot T(1) + n \cdot \log_2 n \\
&= n \cdot T(1) + n \log_2 n \\
&= 3n + n \log_2 n
\end{aligned}
$$

A.1.3 Proofs by Induction

Induction is a technique used to prove recursive definitions/formulas, indicating that it does match its respective closed form solution

- Includes a **base case**, **inductive hypothesis**, and **inductive step**
  - **Base Case** - The starting point, where you prove that it holds true for the smallest value, typically $n = 0$ or $n = 1$
  - **Inductive Hypothesis** - An assumption that the statement being proven is true for some value k
  - **Inductive Step** - Prove the inductive hypothesis with the next step $k + 1$

1. Prove that $1 + 2 + 3 + ... + n = \frac{n(n+1)}{2}$ for $n \geq 1$
   a. Let $n = 1$, then $\frac{1(1+1)}{2} = 1$, where $1 = 1$ (both sides are equal)
   b. Assume the formula holds true for $k \geq 1$, where $1 + 2 + 3 + ... + k = \frac{k(k+1)}{2}$
   c. Prove the inductive hypothesis from b. for $k + 1$, the next recurring step

$$
\begin{aligned}
1 + 2 + ... + k + (k + 1) &= \frac{(k + 1)(k + 2)}{2} \\
(1 + 2 + ... + k) + (k + 1) &= \frac{k(k + 1)}{2} + (k + 1) \\
\frac{k(k + 1) + 2(k + 1)}{2} &= \frac{(k + 2)(k + 1)}{2}
\end{aligned}
$$

   where the inductive step does indeed hold

2. Prove $2^n \geq n + 1$
   a. Check if true for $n = 0$

$$
\begin{aligned}
2^0 &\geq 0 + 1 \\
1 &\geq 1
\end{aligned}
$$

   The base case does hold
   b. Assume the statement holds for $n = k$, where

$$
2^k \geq k + 1
$$

   c. Using this assumption, prove it holds for $n = k + 1$

$$
\begin{aligned}
2^{k+1} &\geq (k + 1) + 1 \\
2^{k+1} &\geq k + 2 \\
2^k \cdot 2 &\geq 2(k + 1) \geq k + 2
\end{aligned}
$$

A.1.4 Big-O

Big-O describes how fast a function grows as its input size, n, increases. It observes the asymptotic behavior as n gets large.

- measures the efficiency of algorithms
- compares runtime/space usage based on large inputs
- describes the worst case runtime of an algorithm
  - or the **upper bound** of an algorithm, which indicates how bad the algorithm's runtime can get

Big-O Notation	Name	Typical Algorithmic Behavior
$O(1)$	Constant Time	Accessing a single array element
$O(\log n)$	Logarithmic Time	Binary Search; algorithm halves the input per iteration
$O(n)$	Linear Time	Linear Search; looping over an array once
$O(n \log n)$	Log-Linear Time	Merge Sort, Quick Sort, Heap Sort; Divide and Conquer, where problems are broken into subproblems that are solved individually
$O(n^2)$	Quadratic Time	Bubble Sort, Selection Sort, Insertion Sort; Anything involving nested for loops
$O(2^n)$	Exponential Time	Typically many recursive algorithms, including brute force (attempts all possible solutions to a problem until one satisfies the problem's conditions)
$O(n!)$	Factorial Time	Permutation-Based Solutions

A.1.4.1 Big-O of Common Algorithms

Algorithm	Big-O Notation	Description
Linear Search	$O(n)$	Checks each element one by one from beginning to end looking for the given element
Binary Search	$O(\log n)$	Takes in a sorted array and iteratively halves the array that may contain the given value
Bubble Sort	$O(n^2)$	Repeatedly steps through an array and compares adjacent elements, swapping them if they are in the wrong order
Insertion Sort	$O(n^2)$	Iteratively inserts each element of an unsorted list into its correct position in a sorted portion of the list
Selection Sort	$O(n^2)$	Sorts an array by repeatedly selecting the **smallest** element from the

Algorithm	Big-O Notation	Description
		unsorted portion and swapping it with the first unsorted element
Merge Sort	$O(n \log n)$	Recursively divides an array/list into smaller sublists, where each sublist is sorted and then merged back together into one big list
Quick Sort	$O(n^2)$ with average case of $O(n \log n)$	Selects a pivot element from the array and partitions the other elements into two sub-arrays, based on whether they are less than or greater than the pivot
Heap Sort	$O(n \log n)$	Organizes elements in an array using a binary heap, by repeatedly swapping the root node (whether it's the maximum or minimum element for max/min heap respectively) with the last element of the heap

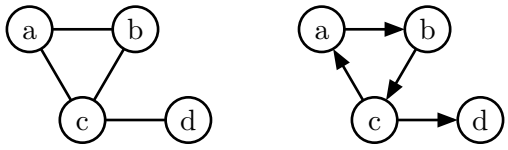
A.1.5 Graphs

Graphs consist of a set of nodes (points/vertices), V, and a set of edges, E, where the graph is commonly referred to as a pair of sets (V, E).

Each edge in E joins two nodes in V, where two nodes connected by an edge are **adjacent** or **neighbors**

Edges can have direction, where graphs with these edges are known as **directed graphs**. This is useful when discussing relations between vertices with contexts to their edges. In contrast, edges that can be traversed in both directions are known as **undirected graphs**

Figure A.1.1 draws the same four nodes both ways: on the left the edges have no direction, and on the right each edge has an arrow



Undirected Directed

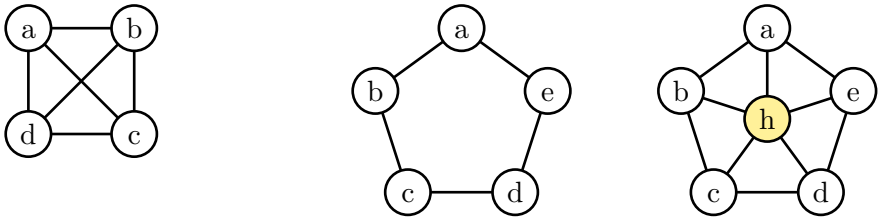
Figure A.1.1: Example: $V = \{a, b, c, d\}$ with four edges; `a` and `b` are adjacent, `a` and `d` are not

A.1.5.1 Additional Graph terminology

- **Loops** - a node where an edge connects to itself
- **Multiple Edges** - two nodes are connected by multiple edges

- **Complete Graphs** - Graphs where each node is connected to every other node. They have $\frac{n(n-1)}{2}$ edges
- **Walks** - A sequence of nodes and edges, where each edge connects the node before and after it and nodes and edges can repeat
- **Paths** - Walks that **cannot repeat nodes and edges**
- **Cycles** - A path that **starts** and **ends** at the same node
  - **Cyclic Graphs** consist of graphs where nodes are connected in a cycle (non cyclic graphs are called **acyclic**)
- **Wheels** - Cycle Graphs where there exists a **single central node** that connects to every other node in the cycle graph
- **Subgraphs** - A smaller graph made from a subset of vertices and edges of a bigger graph

You can see a complete graph, a cycle, and a wheel side by side in **Figure A.1.2**. Notice how the wheel is just the cycle plus a center `h` connected to every node



Complete ($4 \cdot 3/2 = 6$ edges) Cycle Wheel

Figure A.1.2: Example: a complete graph, a cycle graph, and a wheel with center `h`

A graph is **simple** if it does not contain **loops** or **multiple edges**

A.1.5.2 Degree

The degree of a node v, notated as $\deg(v)$, is the number of edges that are connected to v. Loops count **twice** towards the degree of a node.

Each edge contributes to two degrees of a node. Therefore, the **sum of the degrees of all nodes in a graph** is **twice** the number of edges, which can be mathematically expressed as

$$\sum_{v \in V} \deg(v) = 2|E|$$

or

$$\sum_{k=1}^{n} \deg(v_k) = 2|E|, \quad v_k \in V$$

This is known as the **Handshaking Theorem**. Let's check it on the graph in **Figure A.1.3**: the degrees add up to 10, which is twice its 5 edges

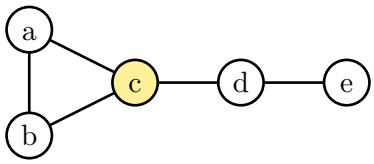


Figure A.1.3: Example: $\deg(c) = 3$, and the degrees $2 + 2 + 3 + 2 + 1 = 10 = 2 \cdot 5$ match the 5 edges

A.1.5.3 Isomorphism

Isomorphism exists when two graphs have the same structure, albeit the drawings may be different

Two graphs, G_1 and G_2 are isomorphic when there exists a bijection of

$$f : V_1 \to V_2$$

Or simply,

- The two graphs must have the same number of nodes and the same number of edges
- For any node degree k, the two graphs must have the same number of nodes of degree k. For example, they must have the same number of nodes with degree 3
- Given a subgraph of G_1, that subgraph must also exist in G_2

In **Figure A.1.4**, G_1 and G_2 are drawn differently but are isomorphic

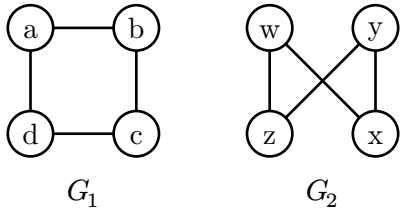


Figure A.1.4: Example: isomorphic graphs; $f(a) = w$, $f(b) = x$, $f(c) = y$, $f(d) = z$ maps every edge to an edge

A.1.5.4 Connectivity

A graph is **connected** when there exists a walk between every pair of nodes in G, or simply you can get from **any node to any other node**, as in the left graph of **Figure A.1.5**

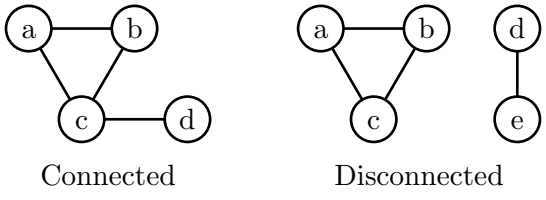


Figure A.1.5: Example: on the right there is no walk from `a` to `d`

Sometimes removing an edge of a graph will completely lose its connectivity, where such edge is known as a **cut edge**. If removed, the graph would be disconnected. Try removing the edge between `c` and `d` in **Figure A.1.6**, and you're left with two separate pieces

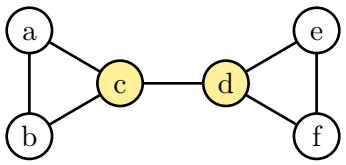


Figure A.1.6: Example: `c-d` is a cut edge; removing it leaves $\{a, b, c\}$ and $\{d, e, f\}$ disconnected

An **Euler Circuit** exists when there is a closed walk (circuit) that uses every edge of the graph once. This can only occur when **all degrees of the nodes are even**. This is because in order to complete the circuit, you must leave each node that you enter. This means that the graph is **connected** and therefore exists a closed walk that uses every edge of the graph once. In **Figure A.1.7**, for example, every node has an even degree, so a circuit starting and ending at a covers each edge exactly once.

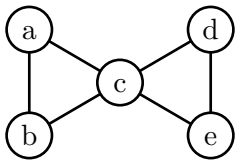


Figure A.1.7: Example: every degree is even ($\deg(c) = 4$, the rest are 2), so the circuit a, b, c, d, e, c, a uses each edge once

A.1.5.5 Bipartite Graph

A graph $G = (V, E)$ is **bipartite**, such that V can be split into two nonoverlapping subsets V_1 and V_2, where every edge connects to an element of V_1 in V_2.

What this simply means is that **a graph can only be bipartite when** $\chi(G) = 2$. This means that a graph can only be colored with **two colors**, like **Figure A.1.8**, where the shaded nodes a, b, c get one color and x, y, z get the other

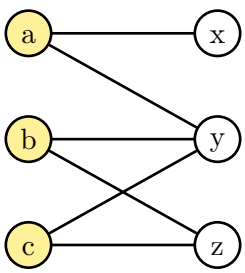


Figure A.1.8: Example: $V_1 = \{a, b, c\}$ (shaded) and $V_2 = \{x, y, z\}$; every edge crosses sides, so two colors suffice

A.1.6 Trees

Trees are undirected graphs with a special node called the **root**, where every node is connected to the root via one exact **path**

They are also **acyclic**, **connected**, and have n **vertices** with $n-1$ **edges**

A.1.6.1 Tree Vocabulary

- **Parent** - A node that is directly above another node in a tree
- **Child** - A node that is directly below another node in a tree
- **Sibling** - Nodes that **share the same parent**
- **Leaf** - A node that has no children
- **Internal** - A node that has at least one child
- **Level** - The number of edges from the **root** to the node (the root has a level of 0)
- **Height** - The longest path from a node to a leaf (in terms of edges)
- **Ancestor** - The ancestor of a node is one that lies on the path from the root to such node
- **Descendant** - The descendant of a node is one that lies on the path from such node to a leaf node

Try matching these terms to the small tree in **Figure A.1.9**. For example, `B` is an internal node at level 1, and `A` is an ancestor of every other node

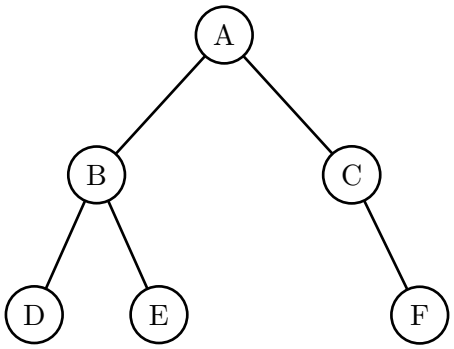


Figure A.1.9: `A` is the root and the parent of siblings `B` and `C`; `D`, `E`, and `F` are leaves; the tree has height 2

A.1.6.2 m-ary Trees

m-ary trees are trees that can have at most m children. A common type of tree is a **Binary Search Tree**, where each node has at most 2 children.

- **Full** trees exist when each node has either 0 or m children
- **Complete** trees exist when all levels are completely filled, except for the lowest one, which is filled from the left
- A tree is **full** and **complete** when the whole bottom level is fully populated with leaves
- **Balanced** trees exist when the heights of its left and right subtrees **differ by at most** 1

Going left to right in **Figure A.1.10**, the first tree is full, the second is complete, and the third is both. For balance, compare the two trees in **Figure A.1.11**: node `1` on the right has a left subtree that is two levels taller than its right

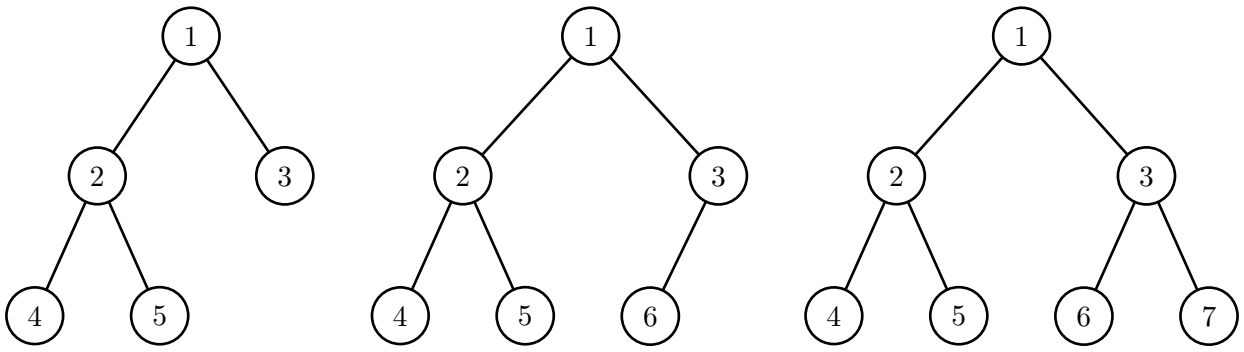
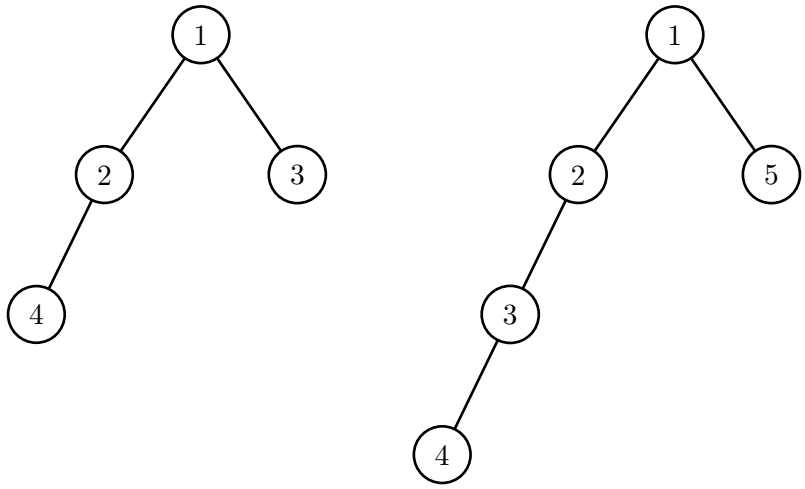


Figure A.1.10: Binary trees that are full, complete, and both full and complete (left to right)



Balanced Not balanced

Figure A.1.11: Example: on the right, node `1` has a left subtree of height 2 and a right subtree of height 0

There are $mi + 1$ nodes total in a m-ary tree. This means a binary tree has $2i + 1$ nodes total. You can count this yourself on the full 3-ary tree in **Figure A.1.12**, which has 2 internal nodes and $3 \cdot 2 + 1 = 7$ nodes

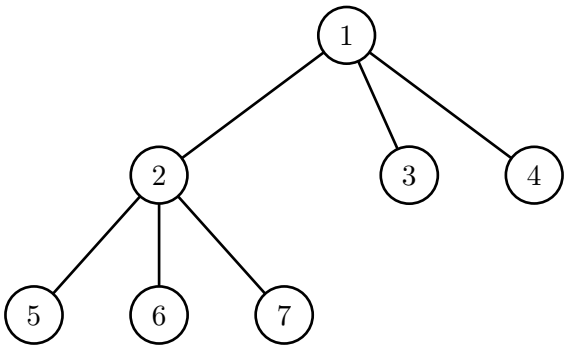


Figure A.1.12: Example: a full 3-ary tree with $i = 2$ internal nodes (`1` and `2`) has $3 \cdot 2 + 1 = 7$ nodes

A.1.6.3 Height and number of nodes

There is no exact formula to find the number of nodes and leaves in an m-ary tree. This is because some trees are more bushier (have more children) than others.

However, there are some patterns **(assuming that a given tree is binary)**

- The number of nodes are maximized when a tree, with height h, is full and complete is 2^h
  - where the number of nodes at level L are 2^L and the total nodes are $\sum_{L=0}^{h} 2^L$, with a closed form of $2^{h+1} - 1$

A.1.6.4 Public vs Private

Each member of a class in C++ has a specific **access level** that determines how that member can be accessed

A member can be one of three types: **public**, **private**, or **protected**

- `private` - Members declared as private can only be accessed from **within the same class**
- `protected` - Members declared as protected can only be accessed by derived classes (subclasses)
- `public` - Members declared as public can be accessed **anywhere** (inside the class, from other classes, or even outside the class)

Access Specifier	Access within Class	Access in Derived Class	Access Outside Class
`public`	Yes	Yes	Yes
`protected`	Yes	Yes	No
`private`	Yes	No	No

A.1.6.4.1 Classes vs Structs

> Members of a `struct` are `public` by default

> Members of a `class` are `private` by default

That is the simple difference between a `class` and a `struct`

The keywords `public protected` and `private` can explicitly define access levels of functions and variables for both a struct or class.

However, it is **best practice** for a struct to not use any access specifiers (all members will anyway default to `public`) and for a class to make member variables `private` (or `protected`) and methods `public`

A `struct` is typically used when
- There is a simple collection of data that doesn't benefit from restricting access
- Aggregate initialization is sufficient (no need for object)
- Structs should be aggregates by convention, where direct initialization of members are allowed through an initializer list `{ }` and there are no **constructors**, no **private/protected access members**, no **virtual functions**, and no **base classes**

Otherwise, just use classes

A.1.6.4.2 Member Functions

Member functions are functions that belong to a class type. This means that they can be called on an instance of that class type (e.g. `e.getData()`, where `getData()` is a member function of object type `e`)

- This means they are declared in the class definition!

> C# and Java refer to these as methods, since they are object oriented languages at the core. They aren't referred to as methods in C++, but the idea still remains the same

Functions that don't belong to a class are **non-member functions** or simply **free functions**.

With free functions, the object must be explicitly passed into the function, while with member functions, the object is implicitly passed and accessed through that object with the dot operator (`.`)

Member functions can sometimes create more clarity on what is doing the action!

A.1.6.5 Constructors

Constructors are special functions automatically called after a non-aggregate class type is created

- If an accessible matching constructor is found, then memory for the object is allocated and the constructor function is called. Otherwise, a compilation error is generated

They can also be **overloaded**, like functions

By convention, constructors **initialize member variables** (via a member initialization list) and perform **other setup functions** (error checking initialization values, opening a file or database, etc.)

Constructors must **have the same name as the class** and have **no return type** (not even `void`)

Class members are initialized in a constructor as follows via an initializer list, which initializes the member variable to the parameter variable

```
class Foo {
    int m_x;
    int m_y;
    public:
        Foo(int x, int y) : m_x{ x }, m_y{ y } { }
};
```

This is the proper way to initialize member variables.

The code below does the same thing, but its technically not initializing, but rather assigning. This means that `m_x` and `m_y` may go uninitialized

```
class Foo {
    int m_x;
    int m_y;

public:
    Foo(int x, int y) {
        m_x = x;
        m_y = y;
    }
};
```

A **default constructor** is one that accepts no arguments, meaning that it has been defined with no parameters. Suppose, class `Foo` exists where the only constructor that exists is the default constructor

```
class Foo {
public:
    Foo() { }
};
```

When creating an object, like `Foo f;` or `Foo f2{};` the **default constructor** is called, which doesn't do anything in this example

If no constructor is given, then the compiler will generate a public default constructor that does nothing, equivalent to the example above. This is known as an **implicit default constructor**

Instead of explicitly stating a default constructor, like `Foo() { }`, we can use the default keyword to have the compiler explicitly generate a default constructor for us. This is shown in the example below

```
class Foo {
public:
    Foo() = default;
};
```

With a user defined constructor, member variables are not automatically initialized (you need to do it yourself). With compiler provided constructors (either `= default` or implicitly provided), the members are also left uninitialized for `Foo f;`. The compiler only zero initializes them when the object is value initialized, like `Foo f{};`

Therefore, only create a default constructor when it makes sense for objects of a class type to be created using all default values

A.1.6.6 Namespaces

Namespaces group together related code (variables, functions, classes, etc.) under a name to avoid naming conflicts.

- The built in `cout` function for console output belongs to the standard `std` namespace, which is why we namespace it with `std::cout`
- This avoids naming conflicts in case we decided to make our own `cout` function (this actually was a thing back in old C++, where `cout` didn't belong to the `std` namespace)
- There is a way to get around this with `using namespace std`, but if we end up defining our own `cout` function, this causes a **namespacing collision** and our code won't compile!

A.1.7 References and Pointers

A **reference** is an alias of an existing object. It refers to the same object in memory with a different **name**. References are notated with an `&`

```
int x = 10;
int& xRef = x;
xRef = 20;

// After x will also become 20 because they refer to the same memory
```

After running the code above, `x` will also become 20 because `x` and `xRef` refer to the same address in memory, which is why **Figure A.1.13** has just one box with both names on it

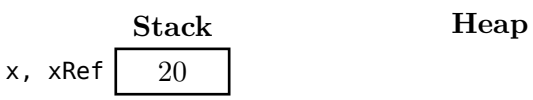


Figure A.1.13: `xRef` is not a new box: it is a second name for the memory of `x`

Note that this is called an lvalue reference in modern C++

A **pointer** is a variable that **stores the memory address** of another variable, thereby "pointing" to that variable's address in memory

- `*` is the syntax used after a type to declare a pointer type and it is used in conjunction with `&` (the address of operator) to get a variable's address in memory!
- `*` (dereferencing operator) is also used to dereference a pointer to access the value that it points to
- Pointers can be **null** (meaning they point to nothing!)
- Dereferencing these pointers can lead to undefined behavior (segmentation fault!)
- `nullptr` keyword can be used to check if a pointer is null! This also avoids dangling pointers!
- `NULL` and `0` can also reference null pointers. `NULL` comes from `C`, while `0` is an address that typically represents a null pointer. However `0` and `NULL` should be avoided in modern C++ in favor of `nullptr`

```
int x = 7;
int* xPtr = &x;
std::cout << *xPtr << std::endl;
```

The code snippet above still prints `7` to the console because it is dereferencing the pointer's value! Unlike the reference above, `xPtr` gets its own box in **Figure A.1.14**, and that box holds the address of `x`

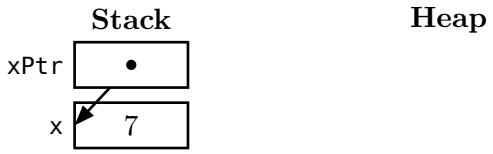


Figure A.1.14: `xPtr` is its own variable holding the address of `x`, so `*xPtr` reads 7

Since a pointer points to the memory address of another variable, it is important to initialize a proper memory address to it. Leaving a pointer **uninitialized** can cause problems!

A pointer is **very similar** to a reference. However, a **reference** is an alias that always refers to one fixed variable, while a **pointer** stores the **memory address** of any variable (or nothing/null). Try to use **references** over pointers because pointers can be dangerous

Dangling Pointers and References are those that happen when they refer to memory that's no longer valid (usually because the object was destroyed or went out of scope)

- **Dangling pointer:** A pointer points to memory that has been freed or deleted
- **Dangling reference:** A reference refers to an object that no longer exists (e.g., a local variable that went out of scope)

These can be **dangerous** because they can lead to **undefined behavior**, **crashes**, or **data corruption** (see **Figure A.1.15** for a pointer left holding freed memory)

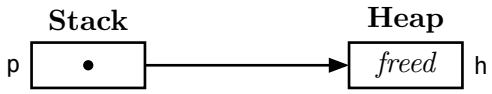


Figure A.1.15: Example: after `delete p;`, the dangling pointer `p` still holds the address of freed memory

A.1.8 Dynamic Memory Allocation

Dynamic Memory Allocation assigns memory space during a program's execution rather than compile time. This memory that the program uses is generally divided into the **stack** and the **heap**.

A.1.8.1 `new` and `delete`

C++ supports static and automatic memory allocation. **Automatic allocation** creates variables that live only while the function runs (on the stack), while **static allocation** creates variables that stay alive for the whole program (in static memory). The period in which a variable stays alive is known as its **lifetime**

Lifetime based on allocation type

- **Automatic objects**: lifetime ends when they go **out of scope** (e.g., when a function/block ends)
- **Static objects**: lifetime ends when the **program terminates**

With these two methods, the size of the variable/array must be known at compile time, and memory allocation and deallocation happen automatically (when a variable is instantiated/destroyed)

However, there may be situations where declaring a fixed size memory may lead to a waste of unused memory or there are more allocated objects of memory than what was assigned.

To fix this problem, memory can be manually managed dynamically, which is known as **dynamic memory allocation**. This is done using two keywords `new` and `delete`

`new` - **Dynamically allocate memory at runtime on the heap for an object/array**. Also calls the **constructor** to initialize the object and returns a typed pointer to the allocated memory

- This is very similar to `malloc` in C (both allocate memory on the heap), but `malloc` does not call a constructor. Also, `malloc` returns a void pointer (`void*`) and needs to be casted to the appropriate pointer type. Therefore in modern C++, `new` is highly recommended.

`delete` - **Frees memory allocated by** `new`. Also calls the **destructor** to clean up the object. Must match the type of allocation (`delete` for single objects and `delete[]` for arrays)

- Ensure that the pointer points to a valid object (has a valid address of memory!). Otherwise, unpredictable behavior can happen!
- Similar to `free` in C, but since there are no destructors, `free` won't call the destructor if there is any
- **Ends the lifetime of the object the pointer points to!**

Note that when you use `delete`, even though the object is freed from memory, its pointer now points to **invalid memory**. This is a **dangling pointer**. Therefore, it is best practice to set the pointer to `nullptr` (null) after using `delete`, as suggested in the code snippet below, which avoids the dangling pointer.

```
int* xPtr = new int(5);
delete xPtr;
xPtr = nullptr;
```

Comparing the two sides of **Figure A.1.16**, the `int` on the heap is gone after the `delete`, and `xPtr` now safely holds `nullptr`

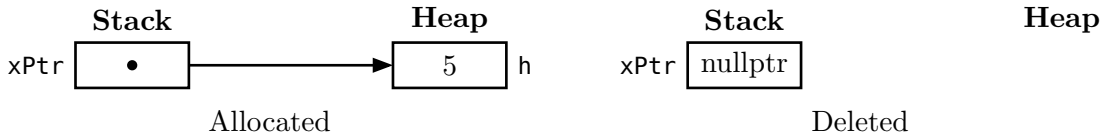


Figure A.1.16: After `new int(5)` (left) and after `delete xPtr; xPtr = nullptr;` (right): the pointer lives on the stack, the `int` on the heap

While static and automatic memory management free variables from memory its lifetime ends or out of scope respectively, memory must be explicitly freed with `delete` when dynamically allocated.

> In other words, when creating memory calls with `new` they **must** be freed with `delete`.

Otherwise, a **memory leak** happens. **Memory leaks** occur when a program allocates memory via `new`, but never frees it via `delete`. This causes unused memory, which can eventually exhaust available memory if it happens repeatedly. In **Figure A.1.17**, for example, `p` gets pointed at a new `int`, so nothing points to the old block `h1` anymore

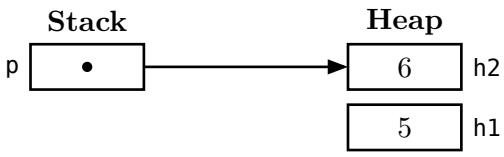


Figure A.1.17: Example: after `int* p = new int(5); p = new int(6);` nothing points to `h1`, so it can never be deleted

A.1.8.2 Destructors

A **destructor** is another special kind of class member function that runs when an object of that class is **destroyed**, using `delete`. While constructors initialize instances of a class, destructors help clean them up

When objects go out of scope or a dynamically allocated object is explicitly deleted using `delete`, the class destructor (if it exists) is automatically called to do any necessary clean up of the object before it is freed from memory

Like constructors, destructors also have specific naming rules

1. Must have **same name** as the class (like a constructor)
2. Cannot have a **return type** (like a constructor)
3. Cannot take **any arguments**

Suppose `Foo` is a class with a default constructor (`Foo() = default;`). The destructor for `Foo` is like the one below

```
~Foo() {

}
```

A.1.8.3 RAII

Resource Acquisition Is Initialization, or RAII, is a C++ programming design pattern that explains how to manage resources safely in a class through a constructor and a destructor. RAII is used heavily in the C++ STL (standard template library) for efficient resource management in objects.

> **RAII** - You acquire a resource (memory, file, lock, etc.) via an object's **constructor** and you automatically release that resource via the object's **destructor**

In other words, an object **acquires** a resource in its **constructor** and **releases** it in its **destructor**

These resources can include anything the class owns or manages
- A file handle
- A pointer to dynamic memory
- A mutex lock
- A network connection

RAII is a basic technique that explains best practices towards writing a class by **tying an object's resources to its lifetime**

- An object acquires a resource in its constructor, and releases it via its destructor when its lifetime ends

A.1.8.4 Heap

The heap, also known as the **free store**, **tracks memory used for dynamic memory allocation**

Recall that `new` allocates memory and returns a pointer that points to that allocated memory. In contrast, `delete` frees the allocated memory towards the operating system to decide what to do with that memory next.

This dynamic memory allocation behavior is all managed via the heap. This means that `new` will allocate memory onto the heap and `delete` will return the allocated memory from the heap, where the operating system can decide what to do with this memory later.

Heap Advantages and Disadvantages

- Allocating memory on the heap is slow
- Allocated memory stays allocated until the programmer deallocates the memory themselves or the application ends (OS cleans it up)
- Even though an OS will clean up dynamically allocated memory at the end of the program, **memory leaks** are still bad practice and you should always `delete` dynamically allocated memory
- This is a problem for bigger programs that handle lots of data, where they can possibly slow down, crash, or cause system instability
- Dynamically allocated memory must be accessed through a pointer. Therefore, dereferencing a pointer is slower than accessing a variable directly
- Because the heap is a big pool of memory, large arrays, structures, or classes can be allocated here

A.1.8.5 Stack

The **stack**, formally referred to as the **call stack**, keeps track of all active functions (ones that have been called and not terminated) from the start of the program to the current point of execution. It also handles allocation of all function parameters and local variables

It stores

- **Local Variables** - function parameters and variables declared inside functions
- **Function call information** - where to return after a function finishes

When a function is called, a stack frame is created on top of the call stack that holds the function's local data. When the function ends, its stack frame is removed, and the memory is automatically reclaimed

Key Details

- The stack is **fast** because elements are accessed in a **LIFO** (Last In First Out) format. This means that both removal and insertion is $O(1)$
- Variables in the stack have automatic storage duration and only exist when the function is running
- Limited size (few MBs), so you can run out of stack space if you create too many local variables or deep recursion

All in all, the **stack** is memory for **local variables** that are automatically managed and exist only when a function is **running**. The **heap** is slower memory for **dynamically allocated variables** (`new`) that must be managed **manually** with `delete`

- **Stack data**: local variables inside functions, function parameters, pointer variables themselves (not what they point to)
  - Used for short-lived, simple variables
- **Heap data**: Any variable dynamically allocated via `new` or `malloc`
  - Use when a variable is needed to outlive the function or for large or dynamically-sized data

Looking at **Figure A.1.18**, notice that the pointer `p` itself is stack data, and only the `int` it points to is on the heap



Figure A.1.18: Example: `int a = 3; int* p = new int(4);` puts `a` and the pointer `p` on the stack and the `int` it points to on the heap

A.1.8.6 Passing Arguments

There are three ways to pass arguments into a function

1. **By Value** - A copy of the argument is made; changes inside the function do **not** affect the original

```
void foo(int x) {
    x = 10;
}
```

   In the example above, assigning `x = 10` only changes the local variable in the function. It does not affect the original `x` parameter being passed into the function. Instead, it creates a copy of `x` and changes that, which is why `n` still holds 5 in **Figure A.1.19**.



Figure A.1.19: Example: after `foo(n)` with `n = 5`, only the copy `x` holds 10

2. **By Reference** - The function receives an alias to the original argument; changes affect the caller's variable

```
void foo(int& x) {
    x = 10;
}
```

   In the example above, assigning `x = 10` changes the original `x` parameter to `10`. Notice in **Figure A.1.20** that `n` and `x` share a single box



Figure A.1.20: Example: inside `foo(n)`, `x` is another name for the caller's `n`, so `n` becomes 10

3. **By Pointer** - The function receives a pointer to the original variable, where you must dereference to modify

```
void foo(int* x) {
    *x = 10;
}
```

In the example above, the function receives a pointer to the original parameter of `x`, which must be dereferenced with `*` to modify the original parameter (see **Figure A.1.21**, where `x` points at `n`)



Figure A.1.21: Example: inside `foo(&n)`, `x` holds the address of `n`, so `*x = 10` sets `n` to 10

A.1.8.7 Returning Values

Same three ways variables are passed into a function, are the same three ways variables are returned in a function

1. **By Value** - Returns a copy of the object

```
int foo() {
    int x = 5;
    return x;
}
```

In the example above, `x` is returned as a copy of the original `x` variable

2. **By Reference** - Returns a reference (alias) to an existing variable, but doesn't refer to a local variable that goes out of scope!

```
int& foo(int& x) {
    return x;
}
```

In the example above, `x` is simply returned as a reference to an existing variable. Be careful! Referencing a local variable that goes out of scope can lead to undefined behaviors!

3. **By Pointer** - Returns a pointer to an object (can be null). The caller can modify the object via the pointer

```
int* foo(int* x) {
    return x;
}
```

In the example above, `x` is simply returned as a pointer to an existing variable. Be careful! Like references, pointing to a local variable that goes out of scope can lead to undefined behaviors!

A.1.9 Copy Semantics

Copy Semantics refers to how objects are copied when they are passed as arguments to functions, returned from functions, or assigned to other objects. Copy semantics dictate the rules for creating new objects that are copies of existing ones. Simply speaking, they **define** how objects are copied

A **copy constructor** is used to initialize an object with an existing object of the same type. After the copy constructor executes, the newly created object becomes a copy of the object passed in the initializer

- **Implicit Copy Constructor** - If a copy constructor is not provided for a class, the compiler will create a public **implicit copy constructor** for you

An explicit copy constructor can also be created, by taking in an referenced object type of that class

```
class Foo {
    public:
    Foo() = default;
    Foo(const Foo& foo) {

    }
};
```

where `Foo a; Foo b = a;` implicitly calls the copy constructor on `b`. Declaring any constructor, including a copy constructor, stops the compiler from generating the implicit default constructor, which is why `Foo() = default;` is needed for `Foo a;` to compile.

A.1.9.1 Shallow and Deep Copies

Shallow and **deep** copies define how objects are copied!

A **shallow copy** occurs where an object's **pointers** are copied, but **not the data they point to**

- Both the object and the copied object share the same underlying data, which means that changing one pointer affects both objects
- If one object deletes or modifies the pointer, the other may be left with invalid data, causing crashes and segmentation faults resulting in undefined behavior or unexpected data, which may contain varied data than what was ideal

A **deep copy** is a completely independent copy of all data, including an object's pointers and the memory they point to

- Unlike a shallow copy, the object and the copied object **do not share the same data**
- Instead, the copied object **allocates new memory** and copies the actual data, so they're fully independent

You can spot the difference in **Figure A.1.22**: with a shallow copy, `a` and `b` point to the same 42, but with a deep copy each one gets its own 42

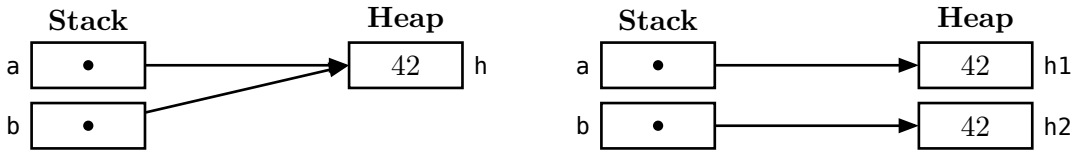


Figure A.1.22: Example: `a` and `b` are the `data` pointers of two objects after `Foo b = a;`

A.1.9.2 Copy Assignment Operator

The Copy Assignment Operator can be overloaded to define copy semantics **after** objects have been constructed. An example, for a `Foo` that owns a dynamically allocated `int* data` and an `int size`, looks like such code below

However, multiple checks that are typically implemented include

- Self assignment check (`this != &other`) to ensure that an object is not copying itself!
- Frees or cleans up resources that the current object owns
- Deep copies data from the source object
- Returns `*this`, a reference to the current object to allow assignment chaining (recall that `this` is an implicit pointer to the current object in a class)

```
Foo& operator=(const Foo& other) {
    // Self assignment check
    if (this != &other) {
        // Free the resource this object currently owns
        delete data;

        // Deep copy the data from the other object
        data = other.data ? new int(*other.data) : nullptr;
        size = other.size;
    }
    return *this;
}
```

Typically, you need to overload it when

- Your class owns dynamically allocated memory or other resources.
- You want to **avoid shallow copies** that lead to double frees or dangling pointers
- You want to implement **deep copying** semantics

A.1.9.3 Default and Delete on Copy constructors

Just like a regular constructor, we can have the compiler explicitly generate a `default` copy constructor, `Foo(const Foo& foo) = default`

- Note that this will perform a **shallow copy**

`= delete` on a copy constructor will prevent objects of a certain class from being copied

A.1.10 Move Semantics

Move semantics allow the resources owned by one object to be **moved** or **transferred** to another object **without copying** the underlying data. They simply define how objects are moved!

Deep copies can be expensive, especially when objects manage dynamic memory or other expensive resources. Therefore, move semantics come in handy and avoid a deep copy when necessary

- Simply use move semantics when you want to avoid copying expensive resources and instead transfer ownership to another object!

They are defined like copy semantics, including a **move constructor** and **move assignment operator**

A.1.10.1 rvalue references

An rvalue is a temporary object or value that **does not** have a persistent memory address. This is the opposite of an lvalue, which has a persistent memory address

```
int x = 1
```

In the code snippet above, `x` is an lvalue (left value) because it is stored in memory with a persistent memory address

- Recall that a pointer will store this memory address and an lvalue reference - sometimes simply called a reference - is a reference or alias to an existing value in memory

In contrast, `1` is an rvalue. It is a temporary value used to initialize `x`

rvalue references (right-value references) are references that bind to **rvalues**, temporary objects or literals that are about to expire. They enable **move semantics** by allowing the program to **take ownership** of resources from these temporary objects, avoiding expensive copies

Understanding these make understanding move semantics much more easier because ultimately what happens is the rvalue of one object is being moved to another object via a rvalue reference of the other object

A.1.10.2 Move Constructor

A **move constructor** defines how objects should be moved once initialized

The code below defines a move constructor. Suppose class `Foo` contains two variables `data` and `size` and we want to move it to an rvalue reference called `other`

- `data` is a dynamically allocated int and `size` is an int
- An example of moving the data from `other` to the current object `this` is shown below by using an object's rvalue reference, `other`

```
Foo(Foo&& other) {
    // Move the data of the other object to this object
    this->data = other.data;
    this->size = other.size;

    // Set data pointer to null and size to 0
```

```
    other.data = nullptr;
    other.size = 0;
}
```

A.1.10.3 Move Assignment Operator

The **move assignment operator** defines how objects should be moved after being initialized

The code below defines a move assignment operator, but typical operations that are implemented include

- **Self-assignment check** (`this != &other`) - ensures that an object is not moving into itself
- Free or clean up any resources that the current object owns (like deleting dynamically allocated memory)
- **Stealing resources from the source** - transfer the ownership of the resources from the `other` rvalue object to the current object by copying pointers or handles
- **Leave the source in a valid, but empty state** - Set the `other` object pointers or resource handles to `nullptr` or equivalent so that destructor won't free the resources again (double free - free what has already been freed)!
- Return `*this`, a reference to the current object!

```
Foo& operator=(Foo&& other) {
    // Self assignment check
    if (this != &other) {
        // Free the resource this object currently owns
        delete data;

        // Steal the resources from the other object
        data = other.data;
        size = other.size;

        // Leave the other object in a valid, but empty state
        other.data = nullptr;
        other.size = 0;
    }
    return *this;
}
```

`std::move()` will trigger move semantics! For example, in the code below

```
Foo a;
Foo b = std::move(a);
```

`b` is move constructed from `a`, effectively moving `a`'s resources into `b`. Looking at **Figure A.1.23**, the 42 on the heap never gets copied: `b` just takes over the pointer, and `a` is left with `nullptr`

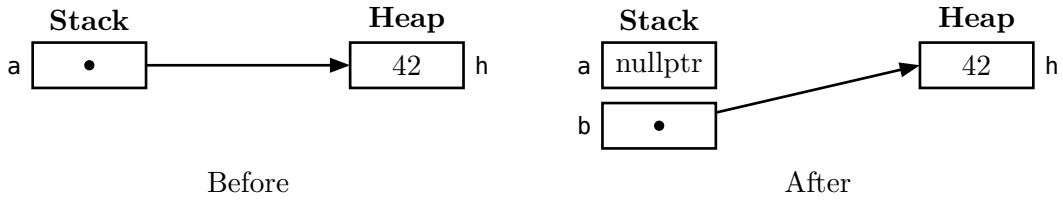


Figure A.1.23: Example: `Foo b = std::move(a);` hands the heap block of `a` to `b` without copying it and leaves `a.data` as `nullptr`

Simply speaking, move semantics are triggered when an object is treated as an rvalue (like the example above), either because it's a temporary (like `b = Foo();`, which move assigns into an existing `b`) or explicitly calling `std::move` (which converts an `lvalue` into an `rvalue`)

A.1.11 Rule of Zero/Three/Five

When a class manages resources (**dynamic memory**, file handles, etc.), it **must** define how it is **copied**, **moved**, and **destroyed** in order to avoid issues like memory leaks or double deletes

The rule of zero/three/five are rules widely adopted in C++ related to memory/resource management and ownership, specifically about when and how you should define or avoid defining special member functions in a class

Rule of Zero - If your class doesn't manage resources directly, **DO NOT** define the **big five**

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Rule of Three - If you define one of a **destructor**, **copy constructor**, or **copy assignment operator**, **you should define ALL three of them!**

- This is because handling one implies your class is managing a resource that needs careful copying and cleanup

Rule of Five - Applies to modern C++, where if you plan on managing resources with move semantics, define **all 5 of the following** (this is the opposite of the **rule of zero**)

- Destructor
- Copy Constructor
- Copy Assignment
- Move Constructor
- Move Assignment

Use this rule when you need to **avoid deep copying** and allow your class to be efficiently moved

A.2 Midterm 1: Lists

Midterm 1 covers modules 1-7, which discuss mostly about Lists. The topics covered are listed below

A.2.1 Topics Covered

- Lists
  - ADT
  - Linked
  - Array

Use these assignments to reference when studying as well

- `lab_debug`
- `mp_stickers`

A.2.2 Lists

ADTs, or Abstract Data Types, are templates/descriptions used to explain how a data structure works and how any user should use it.

In a **`LinkedList`**, each node is stored independently in memory and contains
1. Data - the actual value stored in the node
2. Pointer - a pointer to the next node in the list (or previous in a doubly linked list)

From a visual standpoint a `LinkedList` looks like the one in **Figure A.2.1**

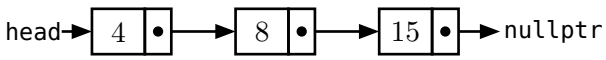


Figure A.2.1: A singly linked list: each node holds data and a pointer to the next node

Each node holds data and then a pointer to the next node!

The table below shows the core functionalities of a `LinkedList` and their runtimes

Operation	**Singly Linked List**
Insert/Remove at front	`O(1)`
Insert/Remove at back	`O(1)` for insert, `O(n)` for remove
Insert/Remove given a pointer	`O(1)`
Insert/Remove at a given index	`O(n)`
Find a value in a list	`O(n)`
Find a value in a sorted list	`O(n)`

- Inserting at the front of a `LinkedList` is `O(1)`. This is because we only have to do 3 steps. Create a new node with its given data, point its next to the original head, set the new node as `head`

You can see these steps in **Figure A.2.2**, where the new node 2 (shaded) points to the old head 4

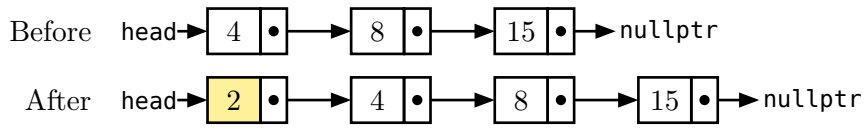


Figure A.2.2: Example: inserting 2 at the front only rewires the new node's `next` and `head`

- Removing at the front of a `LinkedList` is also `O(1)`. This is because to remove the front node, we only need to do 3 constant operations
  1. Create a temporary node pointing to the head
  2. set `head` to its next pointer (`head = head->next`)
  3. Free temp from memory (`delete temp`)

  Notice in **Figure A.2.3** how `head` moves on to 8 while `temp` hangs on to 4 so it can be deleted

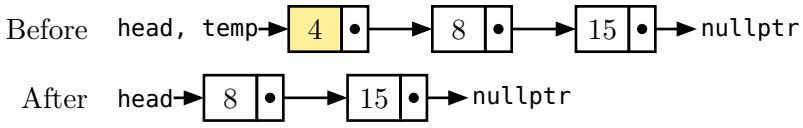


Figure A.2.3: Example: `temp` saves the node holding 4, `head` moves to 8, then `delete temp` frees 4

- Inserting at the back of a `LinkedList` is also `O(1)`, but this assumes that there is a `tail` pointer
  - Otherwise it would be `O(n)` because we would need to traverse `n` times through the entire list from `head` to find the last node
- Removing from the back of a `LinkedList` is `O(n)` because we have to traverse through the list `n` times in order to find the second to last node and set its `next` pointer to `nullptr`

  In **Figure A.2.4**, the walk from `head` stops at the second to last node, the shaded 8

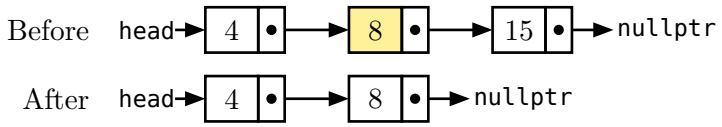


Figure A.2.4: Example: removing 15 means walking from `head` to 8, setting its `next` to `nullptr`, then deleting 15